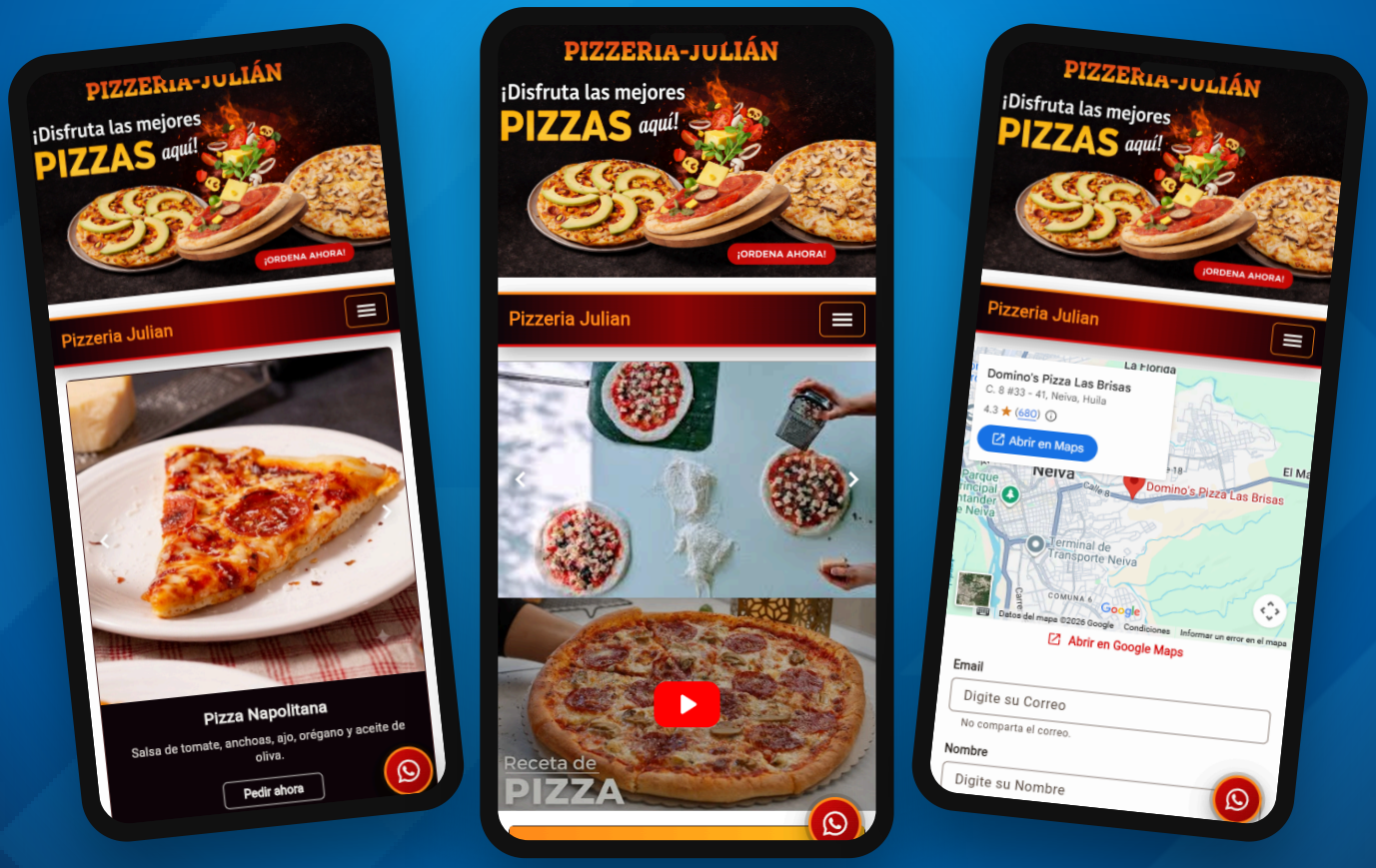
UNIVERSIDAD
SURCOLOMBIANA
Neiva – Huila, Colombia



APP MÓVIL DESARROLLADA CON FLUTTER

MANUAL TÉCNICO

App móvil "Pizzería Julián"

Arquitectura · Diagramas UML · Base de datos · Codificación · Normas aplicables

Flutter 3.47.9 | Dart 3.13.7 | PostgreSQL 16 | Docker Desktop

Ingeniero desarrollador: ALBER DELFIN PEÑA ORTIGOZA

UNIVERSIDAD SURCOLOMBIANA · Versión 1.0 · Noviembre de 2026

Información del documento

Campo	Detalle
Documento	Manual Técnico – App móvil "Pizzería Julián" en Flutter
Ingeniero desarrollador	ALBER DELFIN PEÑA ORTIGOZA
Institución	Universidad Surcolombiana (Neiva, Huila, Colombia)
Sistema documentado	App móvil multiplataforma (Android, iOS y web) de la pizzería "Pizzería Julián", con servidor API en Dart y base de datos PostgreSQL. Versión móvil del Aplicativo Web "Pizzería Julián" (carpeta pizza-julian).
Ubicación del código	D:\FLUTTER\flutter (app: pizza_julian_app · servidor: pizza_julian_api · base de datos: base_de_datos)
Repositorio	github.com/delfinalber/FLUTTER (rama main)
Base de datos	pizza_julian (PostgreSQL 16, contenedor Docker pizza_julian_db)
Destinatarios	Desarrolladores, estudiantes, docentes, personal de soporte y administradores de infraestructura.
Documento relacionado	Manual de Usuario – App móvil "Pizzería Julián" en Flutter (v1.0, noviembre de 2026).
Versión / fecha	1.0 – 8 de noviembre de 2026

Control de cambios

Versión	Fecha	Descripción	Autor
1.0	08/11/2026	Elaboración del manual técnico de la app Flutter: normas, requisitos, arquitectura, diagramas UML, base de datos PostgreSQL, codificación, seguridad, pruebas, instalación y mantenimiento.	Ing. ALBER DELFIN PEÑA ORTIGOZA

> **NOTA**
> Este manual se elaboró a partir del código fuente real del proyecto y de pruebas ejecutadas el 8 de noviembre de 2026 sobre la app y el servidor en funcionamiento (Flutter 3.47.9, Dart 3.13.7, PostgreSQL 16 en Docker Desktop). Los resultados de las pruebas se presentan en el capítulo 11.

Contenido

1. Introducción 6
1.1 Propósito 6
1.2 Alcance del sistema 6
1.3 Definiciones y siglas 6
1.4 Referencias 7
2. Marco normativo y estándares aplicables 8
2.1 Normas de ingeniería y calidad de software 8
2.2 Seguridad de la información 8
2.3 Accesibilidad y estándares técnicos 9
2.4 Estándares de codificación 9
2.5 Legislación colombiana aplicable 10
2.6 Evaluación de calidad según ISO/IEC 25010:2023 10
3. Descripción general del sistema 12
3.1 Contexto 12
3.2 Tecnologías y versiones 12
3.3 Perfiles de usuario 13
3.4 Restricciones y supuestos 13
4. Especificación de requisitos 14
4.1 Requisitos funcionales 14
4.2 Requisitos no funcionales 15
5. Arquitectura del software 16
5.1 Estilo arquitectónico 16
5.2 Estructura de carpetas 16
5.3 Vista de componentes 17
5.4 Vista de despliegue 18
5.5 Patrones y decisiones de diseño 18
6. Modelado UML 20
6.1 Diagrama de casos de uso general 21
6.2 Diagrama de casos de uso del administrador 23
6.3 Especificación de casos de uso 23
6.4 Diagrama de clases (estructura del código) 25
6.5 Diagramas de secuencia 26
6.6 Diagramas de actividad 29
6.7 Diagrama de estados de la sesión 31
6.8 Diagrama de navegación 32
7. Desarrollo de la base de datos 33
7.1 Características generales 33
7.2 Modelo entidad-relación 34

7.3 Diccionario de datos 34
7.4 Script DDL de creación 35
7.5 Consultas SQL utilizadas por la API 39
7.6 Análisis de normalización 39
7.7 Recomendaciones para la base de datos 40
7.8 Copias de seguridad 40
8. Explicación de la codificación 41
8.1 Ciclo de una petición 41
8.2 bin/server.dart y config.dart – arranque y configuración 41
8.3 Autenticación – POST /api/login 42
8.4 Autorización y cierre de sesión 43
8.5 imagenes.dart – carga segura de imágenes 43
8.6 Editores de contenido (CRUD) 45
8.7 paleta.dart y TemaSitio – banner y paleta dinámica 46
8.8 Formulario de contacto – registro y correo 47
8.9 Cliente de la API en la app (lib/api.dart) 48
8.10 Interfaz (widgets, tema y navegación) 49
9. Seguridad de la aplicación 50
9.1 Controles implementados frente a OWASP Top 10:2025 50
9.2 Hallazgos y recomendaciones 50
9.3 Código sugerido 51
10. Instalación, configuración y despliegue 52
10.1 Requisitos del equipo 52
10.2 Instalación en el entorno local 52
10.3 Archivo .env 52
10.4 Creación de administradores 53
10.5 Compilación de la app 53
10.6 Despliegue en producción (lista de verificación) 53
11. Plan y resultados de pruebas 54
11.1 Estrategia 54
11.2 Pruebas automatizadas y análisis estático 56
11.3 Pruebas de compatibilidad 57
12. Mantenimiento 58
12.1 Control de versiones 58
12.2 Hoja de ruta 58
Anexo A. Código fuente 59
A.1 pizza_julian_api/lib/config.dart 60
A.2 pizza_julian_api/lib/api.dart 60
A.3 pizza_julian_api/lib/imagenes.dart 70
A.4 pizza_julian_api/lib/paleta.dart 71

A.5 pizza_julian_app/lib/main.dart 74
A.6 pizza_julian_app/lib/config.dart 75
A.7 pizza_julian_app/lib/api.dart 75
A.8 pizza_julian_app/lib/tema.dart 77
A.9 pizza_julian_app/lib/navegacion.dart 79
A.10 pizza_julian_app/lib/widgets/login_dialogo.dart 79

1. Introducción

1.1 Propósito

Este documento describe la construcción interna de la **App móvil "Pizzería Julián"** desarrollada en Flutter: normas que la rigen, requisitos, arquitectura, modelos UML, base de datos, código fuente, seguridad, pruebas, instalación y mantenimiento. Su objetivo es que cualquier desarrollador pueda comprender, instalar, mantener y evolucionar el software sin depender de su autor. La estructura sigue las recomendaciones de ISO/IEC/IEEE 15289 (contenido de los elementos de información del ciclo de vida) e ISO/IEC/IEEE 26514 (documentación de software).

1.2 Alcance del sistema

El sistema lleva al celular el Aplicativo Web "Pizzería Julián". Está formado por tres piezas: (1) la **app Flutter**, con tres pantallas públicas (Inicio, Productos y Contacto) y tres editores de contenido protegidos por inicio de sesión; (2) un **servidor API REST en Dart** que reemplaza los archivos PHP del sitio web, y (3) la **base de datos PostgreSQL** `pizza_julian`, migrada desde la base MariaDB `tecnico-2026-pagina`. El administrador puede modificar sin programar el banner y la paleta de colores, el carrusel, el video, los textos promocionales, el número de WhatsApp, las tarjetas del catálogo y el mapa de ubicación. El formulario de contacto guarda los mensajes y los envía por correo.

Fuera del alcance de la versión 1.0: carrito de compras, pagos en línea, registro de clientes, gestión de pedidos, notificaciones push, gestión de usuarios desde la interfaz y lectura de mensajes desde los editores. Estas funciones forman parte de la visión a futuro del proyecto (tienda virtual inteligente).

1.3 Definiciones y siglas

Término	Definición
API REST	Interfaz de programación por HTTP que intercambia datos en formato JSON.
Bearer token	Código secreto enviado en la cabecera `Authorization` que identifica la sesión (RFC 6750).
CORS	Cross-Origin Resource Sharing: permite que la versión web de la app consulte el servidor desde otro origen.
CRUD	Create, Read, Update, Delete: operaciones básicas sobre datos.
DDL / DML	Lenguaje de definición / manipulación de datos (SQL).
Flutter / Dart	Kit de desarrollo de Google para apps multiplataforma y su lenguaje de programación.
pgcrypto	Extensión de PostgreSQL con funciones criptográficas (`crypt` , `gen_salt`) para bcrypt.
PK / FK	Llave primaria / llave foránea.
SMTP	Protocolo de envío de correo electrónico.
Widget	Componente visual de Flutter (StatelessWidget o StatefulWidget).

Término	Definición
UML	Unified Modeling Language (OMG UML 2.5.1).

1.4 Referencias

- Manual de Usuario de la App móvil "Pizzería Julián" en Flutter, v1.0.
- Manual Técnico y Manual de Usuario del Aplicativo Web "Pizzería Julián", v1.0 (octubre de 2026).
- Código fuente en `D:\FLUTTER\flutter` y script `base_de_datos/pizza_julian_postgres.sql`.
- Normas y documentos listados en el capítulo 2.

2. Marco normativo y estándares aplicables

Las siguientes tablas reúnen las normas técnicas, los estándares de la industria y las disposiciones legales colombianas vigentes a noviembre de 2026 que aplican al desarrollo, la documentación, la seguridad y la operación de la app, junto con su aplicación en el proyecto.

2.1 Normas de ingeniería y calidad de software

Norma	Objeto	Aplicación en el proyecto
ISO/IEC/IEEE 12207:2017	Procesos del ciclo de vida del software.	Organización del desarrollo en análisis, diseño, construcción, pruebas, despliegue y mantenimiento (capítulos 4 a 12).
ISO/IEC/IEEE 29148:2018	Ingeniería de requisitos.	Requisitos funcionales y no funcionales con identificador, prioridad e implementación (cap. 4).
ISO/IEC/IEEE 42010:2022	Descripción de arquitectura.	Vistas de componentes, despliegue y datos (cap. 5 y 6).
ISO/IEC 25010:2023	Modelo de calidad del producto (SQuaRE).	Evaluación de calidad (numeral 2.6) y requisitos no funcionales.
ISO/IEC 25012:2008	Modelo de calidad de datos.	Integridad y consistencia de la base de datos (cap. 7).
ISO/IEC/IEEE 29119 (partes 1–5)	Pruebas de software.	Diseño y registro de los 30 casos de prueba (cap. 11).
ISO/IEC/IEEE 15289:2019 y 26514:2022	Documentación del ciclo de vida y de usuario.	Estructura de este manual y del Manual de Usuario.
OMG UML 2.5.1 (ISO/IEC 19505)	Lenguaje Unificado de Modelado.	Diagramas de casos de uso, clases, componentes, despliegue, secuencia, actividad y estados (cap. 6).
ISO/IEC 9075 (SQL)	Lenguaje de base de datos SQL.	Script DDL y consultas de PostgreSQL (cap. 7).
ISO/IEC 14764:2022	Mantenimiento de software.	Plan de mantenimiento (cap. 12).

Tabla 1. Normas de ingeniería y calidad de software.

2.2 Seguridad de la información

Norma / guía	Objeto	Aplicación
ISO/IEC 27001:2022 y 27002:2022	Sistema de gestión y controles de seguridad.	Control de acceso, criptografía, desarrollo seguro (8.25–8.28) y copias de respaldo.
OWASP Top 10:2025	Riesgos más críticos de aplicaciones web.	Matriz de controles del capítulo 9.
OWASP API Security Top 10:2023	Riesgos de las API.	Autorización por token, validación de datos y límites de consumo (cap. 9).

Norma / guía	Objeto	Aplicación
OWASP MASVS 2.1 / MASTG	Seguridad de aplicaciones móviles.	Almacenamiento de credenciales, comunicación de red y permisos de la app (cap. 9).
OWASP ASVS 5.0	Requisitos verificables de seguridad.	Autenticación, sesión, validación y carga de archivos.

Tabla 2. Normas de seguridad.

2.3 Accesibilidad y estándares técnicos

Estándar	Objeto	Aplicación
W3C WCAG 2.2 (nivel AA)	Accesibilidad del contenido.	Contraste de la paleta (luminancia relativa WCAG en `paleta.dart`), etiquetas semánticas en el botón de WhatsApp, textos legibles.
NTC 5854 / Resolución MinTIC 1519 de 2020	Accesibilidad web en Colombia.	Referente de buenas prácticas para la versión web.
Material Design 3	Sistema de diseño de Google.	Componentes Material de Flutter (`useMaterial3: true`).
RFC 9110 (HTTP), RFC 8259 (JSON), RFC 6750 (Bearer)	Protocolos de la API.	Códigos 200, 400, 401, 404, 422, 429, 500 y 502; cuerpos JSON UTF-8; token en `Authorization` .
RFC 5321 (SMTP)	Envío de correo.	Mensajes del formulario por Gmail (puerto 587, STARTTLS).
Políticas de Google Play y App Store	Publicación de apps.	Permisos declarados (Internet, galería) y política de privacidad requerida para publicar.

Tabla 3. Estándares de accesibilidad y técnicos.

2.4 Estándares de codificación

Estándar	Aplicación
Effective Dart (guía oficial)	Nombres en lowerCamelCase para variables y funciones, UpperCamelCase para clases, archivos en snake_case.
dart format (ancho 160)	Formato automático e uniforme de todo el código Dart.
flutter_lints 4 / lints 4	Reglas de análisis estático; `flutter analyze` y `dart analyze` sin hallazgos.
Null safety de Dart 3	Tipos que no admiten nulos por defecto; los opcionales se marcan con `?` .
Convención del proyecto	Identificadores y comentarios en español (p. ej. `guardarImagen` , `TemaSitio`); columnas SQL en snake_case.

Tabla 4. Estándares de codificación.

2.5 Legislación colombiana aplicable

Norma	Contenido	Implicación para la app
Ley 1581 de 2012 y Decreto 1377 de 2013 (Decreto Único 1074 de 2015)	Protección de datos personales (habeas data).	El formulario recolecta nombre, correo y teléfono: se requiere aviso de privacidad, autorización del titular y política de tratamiento (también exigida por las tiendas de apps).
Ley 1273 de 2009	Delitos informáticos.	Fundamenta los controles de acceso y la protección de credenciales.
Ley 527 de 1999	Comercio electrónico y mensajes de datos.	Validez de los mensajes del formulario y de futuras transacciones.
Ley 1480 de 2011	Estatuto del Consumidor.	Información veraz de productos; aplicable plenamente al evolucionar a tienda virtual.
Ley 23 de 1982, Decisión Andina 351 de 1993 y Decreto 1360 de 1989	Derechos de autor; el software se protege como obra literaria.	Registro ante la Dirección Nacional de Derecho de Autor y respeto de licencias de terceros (Flutter y paquetes: BSD-3, MIT y Apache 2.0; PostgreSQL License).
Ley 1712 de 2014	Transparencia y acceso a la información pública.	La Universidad Surcolombiana es una entidad pública: si la app se publica en su nombre, debe cumplir esta ley.

Tabla 5. Legislación colombiana aplicable.

2.6 Evaluación de calidad según ISO/IEC 25010:2023

Característica	Evidencia en la app	Valoración
Adecuación funcional	Los 12 casos de uso están implementados; 30 de 30 pruebas aprobadas tras corregir el defecto de la prueba P20.	Alta
Eficiencia de desempeño	Respuestas de la API entre 1 y 300 ms en red local; imágenes enviadas en base64 (≈33 % más pesadas).	Media-alta
Compatibilidad	Un solo código para Android, iOS y web; integración con YouTube, Google Maps y WhatsApp.	Alta
Capacidad de interacción	Mismo diseño del sitio web, formularios con contadores y avisos claros, actualización con gesto de deslizar.	Alta
Fiabilidad	Manejo centralizado de errores (middleware), transacción lógica de imágenes (se descartan si falla el guardado).	Media-alta
Seguridad	SQL parametrizado, bcrypt, token de 256 bits con vencimiento, bloqueo por intentos, validación de imágenes por contenido. Tráfico HTTP sin cifrar en red local (H01).	Media
Mantenibilidad	Código modular (pantallas, widgets, API), una sola conexión centralizada, análisis estático sin hallazgos.	Alta

Característica	Evidencia en la app	Valoración
Flexibilidad (portabilidad)	Base de datos en Docker; servidor Dart ejecutable en Windows, Linux o macOS.	Alta
Protección (safety)	No aplica riesgo físico; los datos personales requieren política de tratamiento.	N/A

Tabla 6. Evaluación de calidad del producto.

3. Descripción general del sistema

3.1 Contexto

La app se instala en el celular (Android o iOS) o se abre en Google Chrome. Toda la información la obtiene del **servidor API** (`pizza_julian_api`), que corre en un computador con Dart y se conecta a **PostgreSQL 16** dentro del contenedor `pizza_julian_db` de Docker Desktop. El administrador se autentica desde la misma app; el servidor le entrega un **token** que la app envía en cada operación de los editores. La app se integra con servicios externos: YouTube (video), Google Maps (mapa), WhatsApp (wa.me) y Gmail SMTP (correo).

3.2 Tecnologías y versiones

Componente	Versión	Uso	Observación (nov. 2026)
Flutter SDK	3.47.9 (stable)	Interfaz multiplataforma	Versión estable vigente; instalado en `flutter_sdk`.
Dart	3.13.7	Lenguaje de la app y del servidor	Incluido en Flutter.
PostgreSQL	16 (postgres:16-alpine)	Base de datos	Soporte del proveedor hasta noviembre de 2028.
Docker Desktop	Engine 29.1.3	Contenedor de la base de datos	Requiere WSL 2 en Windows.
shelf / shelf_router / shelf_static	1.4.2 / 1.1.4 / 1.1.3	Servidor HTTP, rutas e imágenes	Paquetes oficiales de Dart.
postgres (Dart)	3.5.7	Conexión a PostgreSQL con pool	Consultas con parámetros nombrados.
image / mailer	4.12.0 / 6.6.0	Validación de imágenes y paleta / correo SMTP	—
http	1.6.0	Cliente HTTP de la app	—
url_launcher	6.3.1	WhatsApp, YouTube y Maps	—
webview_flutter	4.10.0	Mapa de Google en Android/iOS	En web se usa un iframe.
image_picker	1.1.2	Elegir imágenes de la galería	—
font_awesome_flutter	11.0.0	Íconos de redes sociales	La 11 es la compatible con Flutter 3.47.
web / pointer_interceptor	1.1.1 / 0.10.1+3	Iframe del mapa en web y clics sobre él	Solo afectan la versión web.

Tabla 7. Pila tecnológica.

3.3 Perfiles de usuario

Perfil	Descripción	Acceso
Visitante	Cliente o público general.	Pantallas públicas, formulario de contacto, WhatsApp.
Administrador	Persona de la pizzería encargada del contenido; registrada en la tabla `usuario`.	Editores de Inicio, Productos y Contacto; consola de la base de datos.
Desarrollador / soporte	Mantiene el código y la infraestructura.	Código fuente, servidor, Docker, base de datos y archivo `.env`.

3.4 Restricciones y supuestos

- La app depende del servidor API: sin conexión muestra "Error de comunicación con el servidor." y el botón Reintentar.
- El primer registro de la tabla `inicio` define el banner, la paleta y el WhatsApp de toda la app.
- Las imágenes se guardan en `pizza_julian_api/media` y la base de datos solo almacena su ruta relativa.
- La dirección del servidor se fija al compilar con `--dart-define=API_URL=…` (por defecto `http://10.0.2.2:8080`, el computador visto desde el emulador de Android).
- El envío de correo requiere una cuenta de Gmail con verificación en dos pasos y contraseña de aplicación.
- La sesión del editor se guarda solo en memoria: al cerrar la app hay que volver a iniciar sesión.

4. Especificación de requisitos

4.1 Requisitos funcionales

ID	Requisito	Prioridad	Implementación
RF01	Mostrar Inicio con banner, carrusel de 3 imágenes, video de YouTube, acordeón de 5 elementos y 3 secciones desplegables.	Alta	pantallas/inicio.dart · GET /api/inicio
RF02	Mostrar el catálogo en tarjetas (1, 2 o 4 columnas según el ancho) con carrusel de 3 imágenes cada 3 s.	Alta	pantallas/productos.dart · GET /api/productos
RF03	Mostrar el mapa de ubicación y el formulario de contacto.	Alta	pantallas/contacto.dart (WebView / iframe)
RF04	Registrar los mensajes de contacto y enviarlos por correo.	Alta	POST /api/contacto/mensaje
RF05	Abrir WhatsApp desde el botón flotante, el pie de página y el botón de cada producto.	Media	widgets/comunes.dart · abrirWhatsapp()
RF06	Autenticar al administrador con usuario y contraseña en un diálogo con límite de 30 s.	Alta	widgets/login_dialogo.dart · POST /api/login
RF07	Bloquear el inicio de sesión 5 minutos tras 5 intentos fallidos.	Alta	PizzaJulianApi._intentosPorIp
RF08	Crear, editar y eliminar el contenido de Inicio, incluido el número de WhatsApp.	Alta	pantallas/editor_inicio.dart · /api/inicio
RF09	Calcular la paleta de colores a partir del banner y aplicarla a toda la app.	Media	paleta.dart · tema.dart (TemaSitio)
RF10	Crear, editar y eliminar productos con 3 imágenes obligatorias.	Alta	pantallas/editor_productos.dart · /api/productos
RF11	Crear y editar la URL del mapa de Google.	Media	pantallas/editor_contacto.dart · /api/contacto
RF12	Validar las imágenes por su contenido real (JPG, PNG, GIF, WEBP) y tamaño máximo de 5 MB.	Alta	imagenes.dart · decodificarImagen()
RF13	Eliminar del servidor las imágenes reemplazadas o de registros borrados que ya no se usen.	Media	_borrarNoUsadas()
RF14	Cerrar sesión y volver a la pantalla pública sin que "Atrás" regrese al editor.	Alta	Api.logout() · irA() con pushAndRemoveUntil
RF15	Impedir el uso de los editores sin token válido.	Alta	exigirSesion() · _usuarioAutenticado()
RF16	Actualizar banner y colores al volver a la app o al deslizar hacia abajo.	Media	main.dart · RefreshIndicator

ID	Requisito	Prioridad	Implementación
RF17	Rechazar textos más largos que su columna en la base de datos.	Media	_validarLongitudes()

Tabla 8. Requisitos funcionales.

4.2 Requisitos no funcionales

ID	Categoría (ISO 25010)	Requisito	Verificación
RNF01	Seguridad	Todas las consultas con datos externos usan parámetros nombrados (`Sql.named`).	Revisión de código; prueba P10.
RNF02	Seguridad	Contraseñas almacenadas con bcrypt (`crypt` + `gen_salt('bf', 10)`).	Tabla usuario; prueba P13.
RNF03	Seguridad	Token de sesión de 256 bits generado con `Random.secure()` , con vencimiento de 8 horas.	Pruebas P13, P28 y P29.
RNF04	Seguridad	Validación de imágenes y longitudes en el servidor, no solo en la app.	Pruebas P16, P17 y P20.
RNF05	Capacidad de interacción	Interfaz adaptable a celular, tableta y computador, con el diseño del sitio web.	Capturas del Manual de Usuario.
RNF06	Compatibilidad	Un solo código para Android, iOS y web.	Compilación de APK y versión web.
RNF07	Fiabilidad	Errores inesperados del servidor responden JSON 500 sin detalles internos.	Middleware _errores().
RNF08	Mantenibilidad	Análisis estático sin hallazgos y pruebas automatizadas de la app.	flutter analyze, dart analyze, flutter test.
RNF09	Eficiencia	Respuestas de la API menores a 1 s en red local.	Registro del servidor (logRequests).
RNF10	Flexibilidad	Base de datos reproducible en cualquier equipo con Docker.	Contenedor pizza_julian_db.

Tabla 9. Requisitos no funcionales.

5. Arquitectura del software

5.1 Estilo arquitectónico

El sistema sigue una arquitectura **cliente-servidor de tres capas**. La app Flutter es un cliente "grueso" que dibuja toda la interfaz en el dispositivo y se comunica con una **API REST** sin estado; el servidor concentra la lógica de negocio, la autorización y el acceso a datos; PostgreSQL guarda el contenido. Esta separación evita que la contraseña de la base de datos viaje dentro del instalador de la app, que es el riesgo principal de conectar una app móvil directamente a la base de datos.

Capa	Responsabilidad	Elementos
Presentación	Pantallas, navegación, formularios, tema de colores, validación en el cliente.	pizza_julian_app/lib (pantallas, widgets, tema.dart, navegacion.dart).
Lógica de negocio / API	Autenticación, autorización, validación, CRUD, imágenes, paleta y correo.	pizza_julian_api/lib (api.dart, imagenes.dart, paleta.dart, config.dart).
Datos	Persistencia de contenido, productos, mensajes, usuarios y sesiones; archivos de imagen.	PostgreSQL pizza_julian; carpeta media/.

5.2 Estructura de carpetas

```
D:\FLUTTER\flutter
├── base_de_datos\
│   └── pizza_julian_postgres.sql     Script DDL + datos iniciales (PostgreSQL)
├── crear_base_de_datos.ps1           Crea la base en el PostgreSQL de Windows
├── pizza_julian_api\                 Servidor API REST (Dart)
│   ├── bin\server.dart               Punto de arranque: pool de conexiones y puerto 8080
│   ├── lib\api.dart                  Rutas, autenticación, CRUD y formulario de contacto
│   ├── lib\config.dart               Lectura de .env y variables de entorno
│   ├── lib\imagenes.dart             Validación, guardado y borrado de imágenes
│   ├── lib\paleta.dart               Paleta de colores calculada desde el banner
│   ├── media\inicio, media\productos Imágenes servidas en /media/
│   ├── .env / .env.example           Credenciales (no versionar)
│   └── pubspec.yaml
├── pizza_julian_app\                 App Flutter (Android, iOS, web)
│   ├── lib\main.dart                 Arranque, tema Material y ciclo de vida
│   ├── lib\config.dart               API_URL (--dart-define)
│   ├── lib\api.dart                  Cliente HTTP, Sesion y ApiError
│   ├── lib\tema.dart                 Paleta y TemaSitio (ChangeNotifier)
│   ├── lib\navegacion.dart           Secciones y cambio de pantalla
│   ├── lib\pantallas\                inicio, productos, contacto y sus 3 editores
│   ├── lib\widgets\                  comunes, login_dialogo, editor, iframe*
│   ├── test\widget_test.dart         Pruebas automatizadas
│   ├── android\ ios\ web\            Proyectos de cada plataforma
│   └── pubspec.yaml
├── flutter_sdk\                      SDK de Flutter 3.47.9 (no versionado)
└── *.pdf                             Manuales
```

Componente	Archivos	Líneas
Servidor API (lib + bin)	5	884
App Flutter (lib)	16	2.632

Componente	Archivos	Líneas
Pruebas de la app (test)	1	31
Script de base de datos + PowerShell	2	196
Total código propio	**24**	**≈ 3.743**

Tabla 10. Tamaño del código fuente.

5.3 Vista de componentes

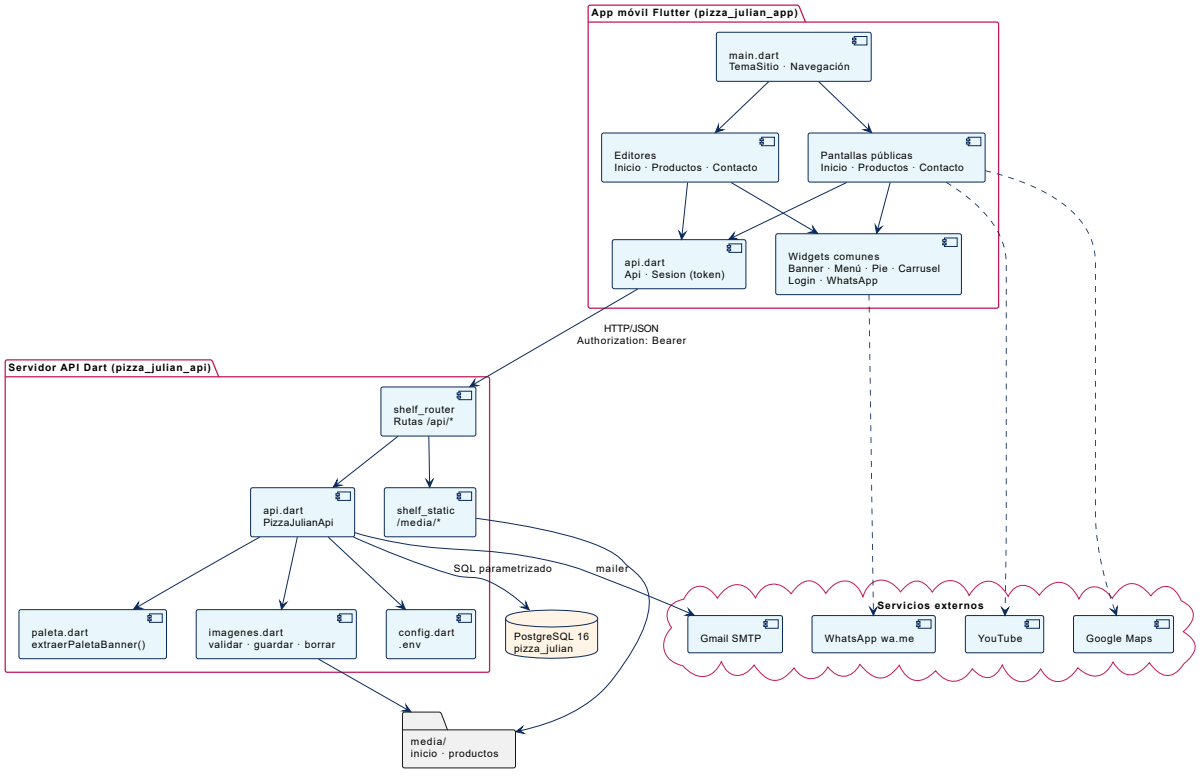


Figura 1. Diagrama de componentes del sistema.

5.4 Vista de despliegue

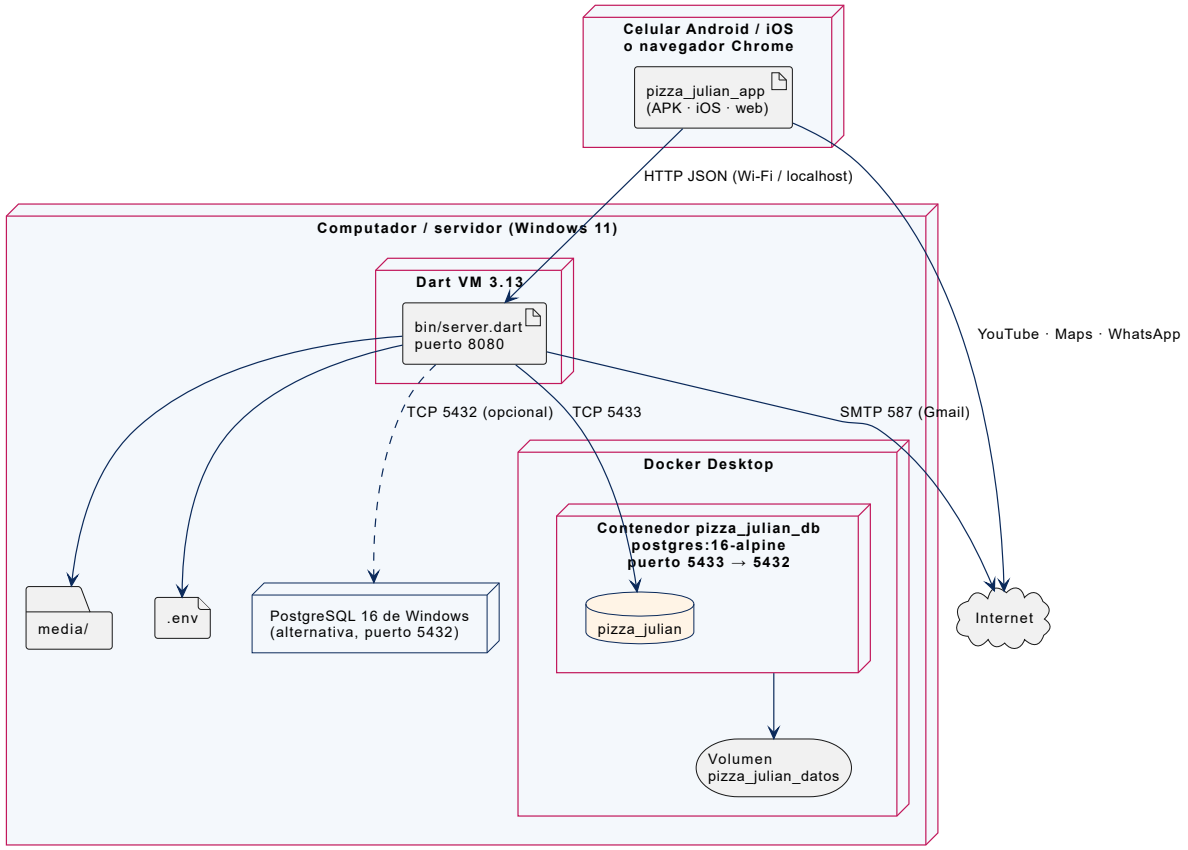


Figura 2. Diagrama de despliegue.

5.5 Patrones y decisiones de diseño

Decisión	Justificación
API intermedia en lugar de conexión directa a PostgreSQL	La app no lleva credenciales de la base de datos; la API valida y autoriza cada operación.
Servidor en Dart (shelf)	Mismo lenguaje que la app: un solo conocimiento para el equipo; ejecución en cualquier sistema operativo.
Token Bearer con vencimiento en la tabla sesion_api	Reemplaza la cookie de sesión de PHP; funciona igual en Android, iOS y web, y no requiere protección CSRF porque el navegador no lo envía solo.
Imágenes en base64 dentro del JSON	Simplifica la API (un solo formato) a cambio de ≈33 % más de tamaño (ver H11).
Paleta calculada en el servidor	Se calcula una sola vez al guardar el banner y todos los clientes la reciben lista (en la web se calculaba en el navegador).
Estado global con ChangeNotifier (TemaSitio, Sesion)	Suficiente para dos estados compartidos, sin dependencias adicionales de gestión de estado.

Decisión	Justificación
Navegación con pushAndRemoveUntil	Igual que `window.location.replace` de la web: el botón Atrás no vuelve a un editor.
bcrypt en la base de datos (pgcrypto)	Conserva los hashes del sitio web (`$2y$` → `$2a$`) sin depender de una librería bcrypt en Dart.
Base de datos en Docker	Instalación reproducible en segundos, sin tocar el PostgreSQL de Windows.

Tabla 11. Decisiones de diseño.

6. Modelado UML

Los diagramas se elaboraron con PlantUML siguiendo la notación OMG UML 2.5.1 y describen la interacción del **visitante** y del **administrador** con el sistema.

6.1 Diagrama de casos de uso general

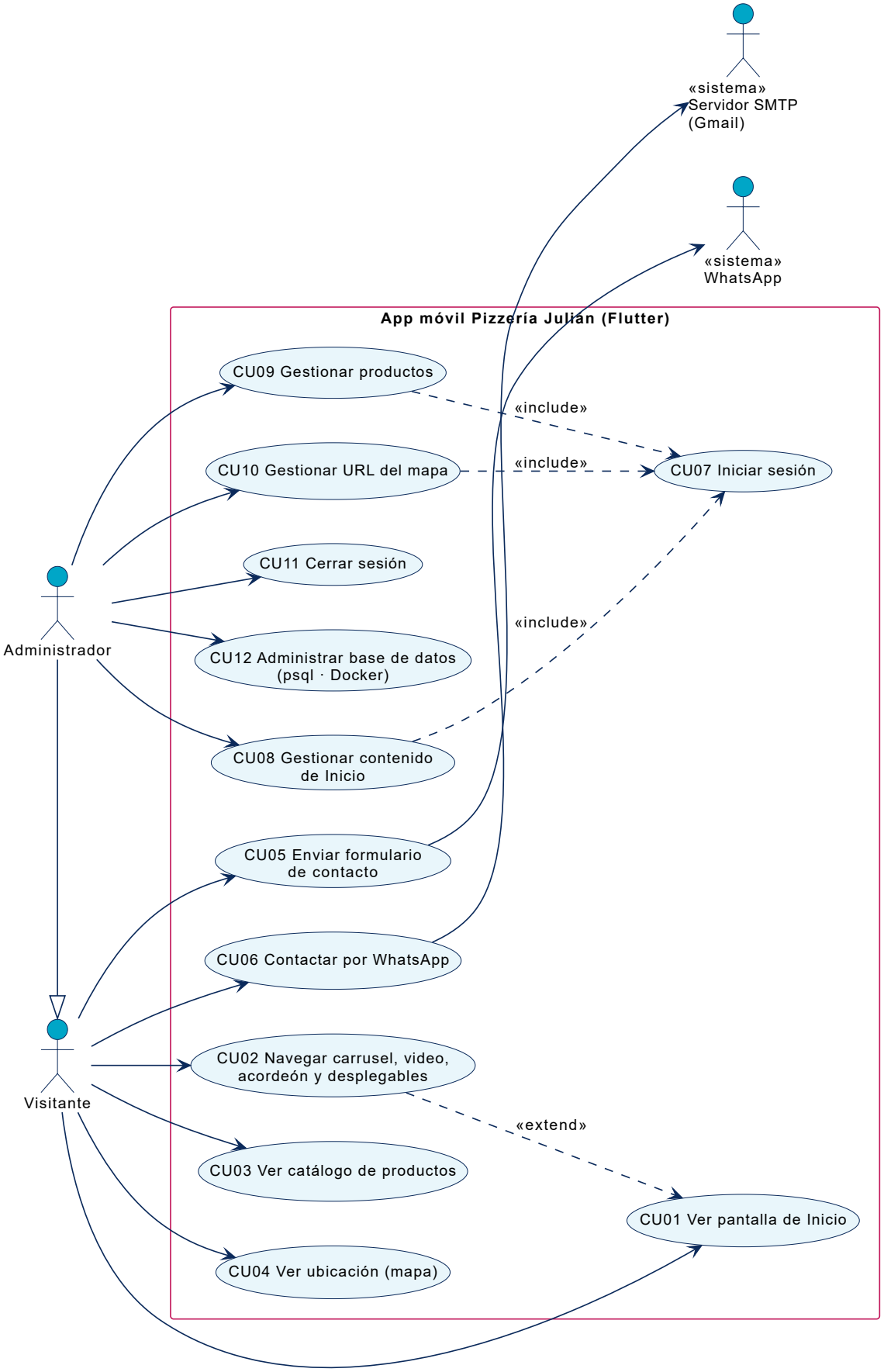

«sistema»
Servidor SMTP
(Gmail)
«sistema»
WhatsApp
App móvil Pizzería Julián (Flutter)
CU09 Gestionar productos
CU10 Gestionar URL del mapa
CU11 Cerrar sesión
CU12 Administrar base de datos
(psql · Docker)
CU08 Gestionar contenido
de Inicio
CU05 Enviar formulario
de contacto
CU06 Contactar por WhatsApp
CU02 Navegar carrusel, video,
acordeón y desplegables
CU03 Ver catálogo de productos
CU04 Ver ubicación (mapa)
CU07 Iniciar sesión
CU01 Ver pantalla de Inicio
«include»
«include»
«include»
«extend»
Administrador
Visitante

Figura 3. Diagrama de casos de uso general.

6.2 Diagrama de casos de uso del administrador

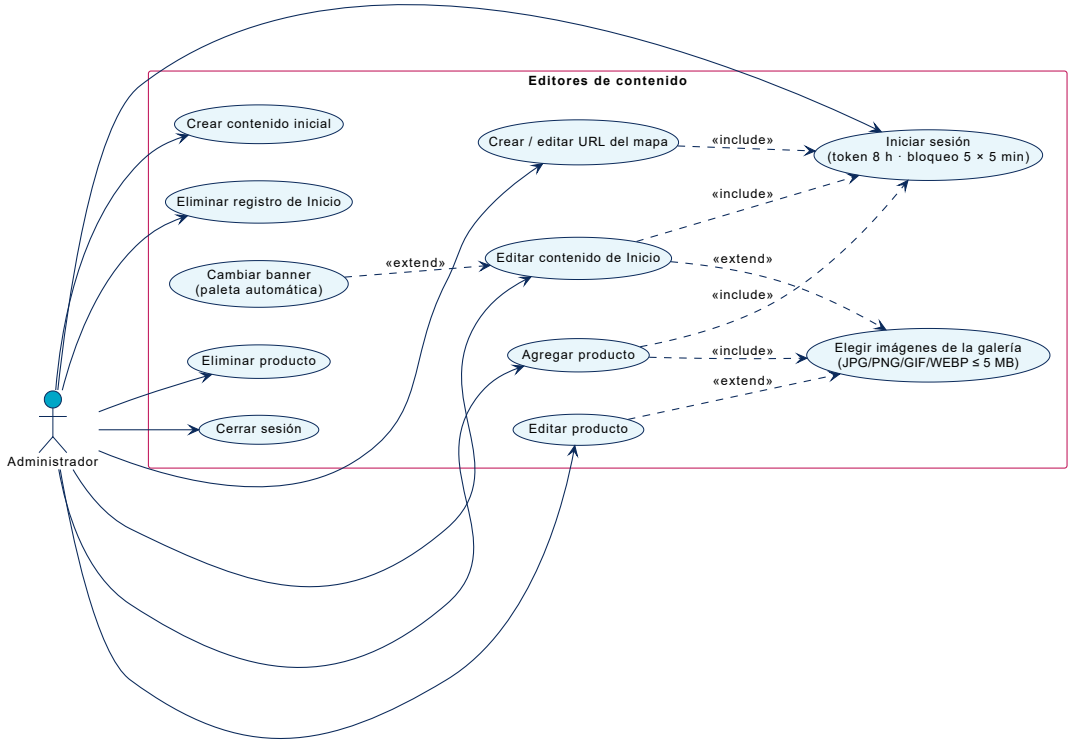


Figura 4. Casos de uso detallados de los editores de contenido.

6.3 Especificación de casos de uso

CU05 – Enviar formulario de contacto

Campo	Descripción
Actor	Visitante (principal); servidor SMTP Gmail (secundario).
Precondición	Servidor API encendido.
Flujo básico	1. El visitante diligencia Email, Nombre, Teléfono y Comentario. 2. Toca Enviar. 3. La app valida los campos obligatorios y el correo. 4. Envía POST /api/contacto/mensaje. 5. El servidor valida y guarda en formulario_contacto. 6. Envía el correo con Reply-To del visitante. 7. La app muestra el aviso verde y limpia el formulario.

Campo	Descripción
Flujos alternos	3a. Datos inválidos: mensajes en rojo debajo de cada campo. 5a. Datos inválidos en el servidor: HTTP 422. 6a. Sin contraseña de Gmail: HTTP 200 "Tu mensaje se guardó correctamente." 6b. Error SMTP: HTTP 502.
Postcondición	Mensaje almacenado y notificado.

CU07 – Iniciar sesión

Campo	Descripción
Actor	Administrador.
Precondición	Usuario registrado en la tabla usuario con contraseña bcrypt.
Flujo básico	1. Toca ☰ → Inicio Sesion. 2. La app abre el diálogo con contador de 30 s. 3. Digita usuario y contraseña. 4. El servidor verifica el bloqueo, los campos y la contraseña con crypt(). 5. Crea un token de 64 caracteres (8 horas) en sesion_api. 6. La app guarda el token en memoria y abre el editor de la sección de origen.
Flujos alternos	2a. Se agotan los 30 s: se cierra el diálogo. 4a. Bloqueo vigente: 429. 4b. Campos vacíos: 400. 4c. Credenciales incorrectas: 401; al 5.º fallo se bloquea la IP 300 s.
Postcondición	Sesión autenticada.

CU08 – Gestionar contenido de Inicio

Campo	Descripción
Actor	Administrador.
Precondición	Sesión iniciada (incluye CU07).
Flujo básico	1. Abre el Editor de Inicio. 2. Toca Editar. 3. Elige imágenes de la galería y modifica video, textos y WhatsApp. 4. Toca Guardar. 5. El servidor valida token, longitudes e imágenes; guarda las imágenes nuevas; calcula la paleta si cambió el banner; actualiza el registro y borra las imágenes reemplazadas. 6. La app muestra "Contenido actualizado correctamente." y sincroniza el tema.
Flujos alternos	5a. Imagen inválida o texto muy largo: 422, no se guarda nada. 1a. Tabla vacía: Crear contenido inicial. 2a. Eliminar con confirmación. 5b. Token vencido: 401 y regreso a la pantalla pública.
Postcondición	Contenido y tema de la app actualizados.

CU09 – Gestionar productos

Campo	Descripción
Actor	Administrador.
Precondición	Sesión iniciada.
Flujo básico	1. Toca Agregar producto. 2. Elige 3 imágenes. 3. Escribe título, texto y texto del botón. 4. Guarda. 5. El servidor inserta el registro y responde "Producto creado correctamente."

Campo	Descripción
Flujos alternos	4a. Falta una imagen o el título: aviso en la app (y 422 en el servidor). Editar: conserva las imágenes no reemplazadas. Eliminar: borra el registro y las imágenes sin uso.
Postcondición	Catálogo actualizado en la pantalla Productos.

CU10 – Gestionar URL del mapa

Campo	Descripción
Actor	Administrador.
Flujo básico	1. Abre el Editor de Contacto. 2. Toca Editar (o Crear URL del mapa). 3. Pega la URL embed o el código HTML de Google Maps (la app extrae el atributo src). 4. Guarda. 5. El servidor verifica que empiece por https:// y actualiza la tabla contacto.
Postcondición	Mapa actualizado en la pantalla Contacto.

6.4 Diagrama de clases (estructura del código)

El diagrama muestra las clases principales de la app (widgets, estado y cliente de la API) y del servidor, con sus dependencias. Los widgets de Flutter se marcan con su estereotipo (StatelessWidget, StatefulWidget, ChangeNotifier).

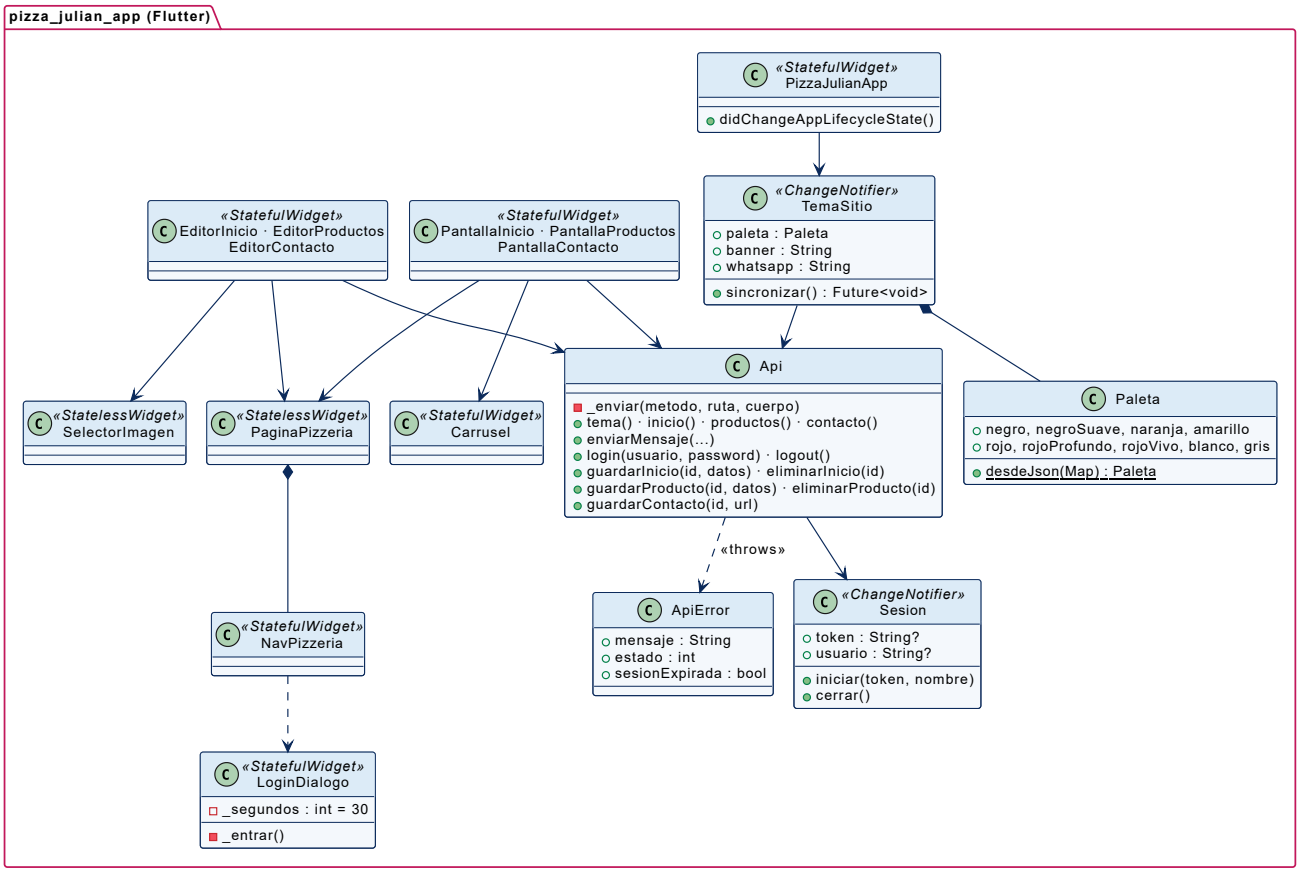


Figura 5. Diagrama de clases de la app Flutter.

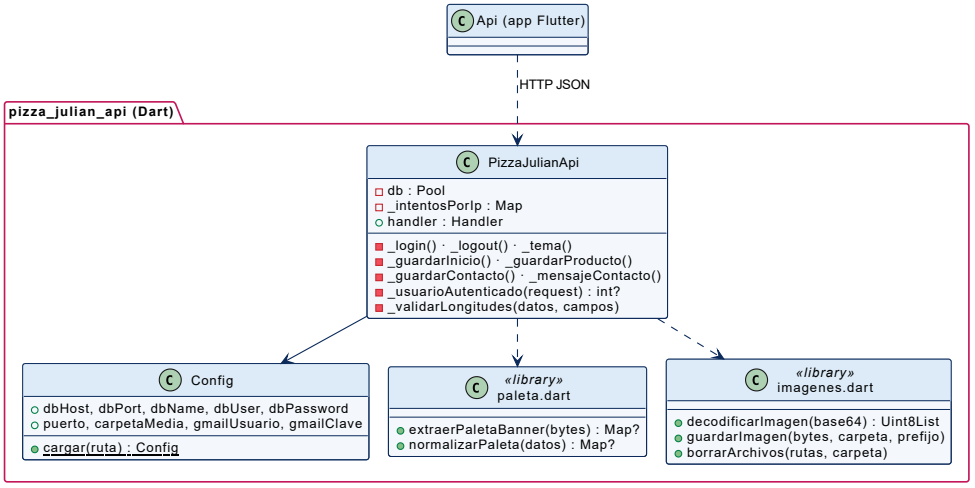


Figura 6. Diagrama de clases del servidor API.

6.5 Diagramas de secuencia

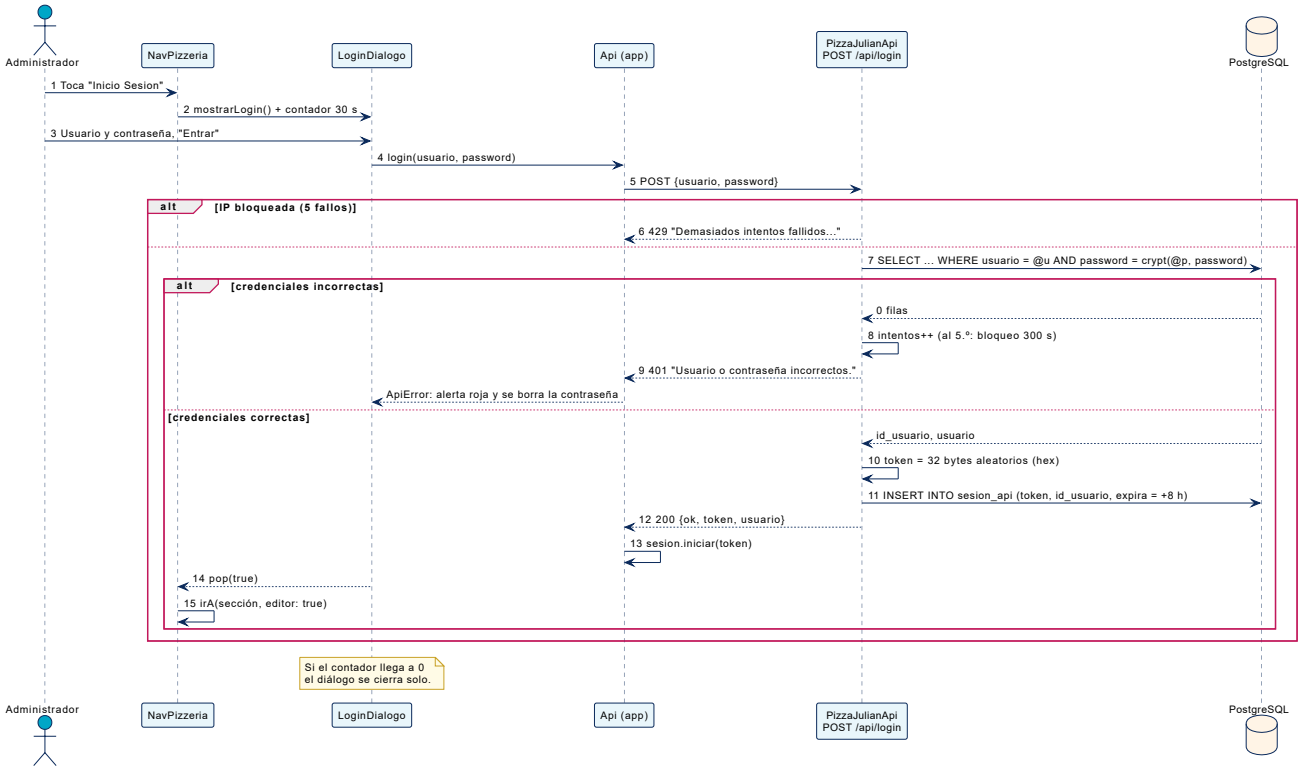


Figura 7. Secuencia: inicio de sesión del administrador.

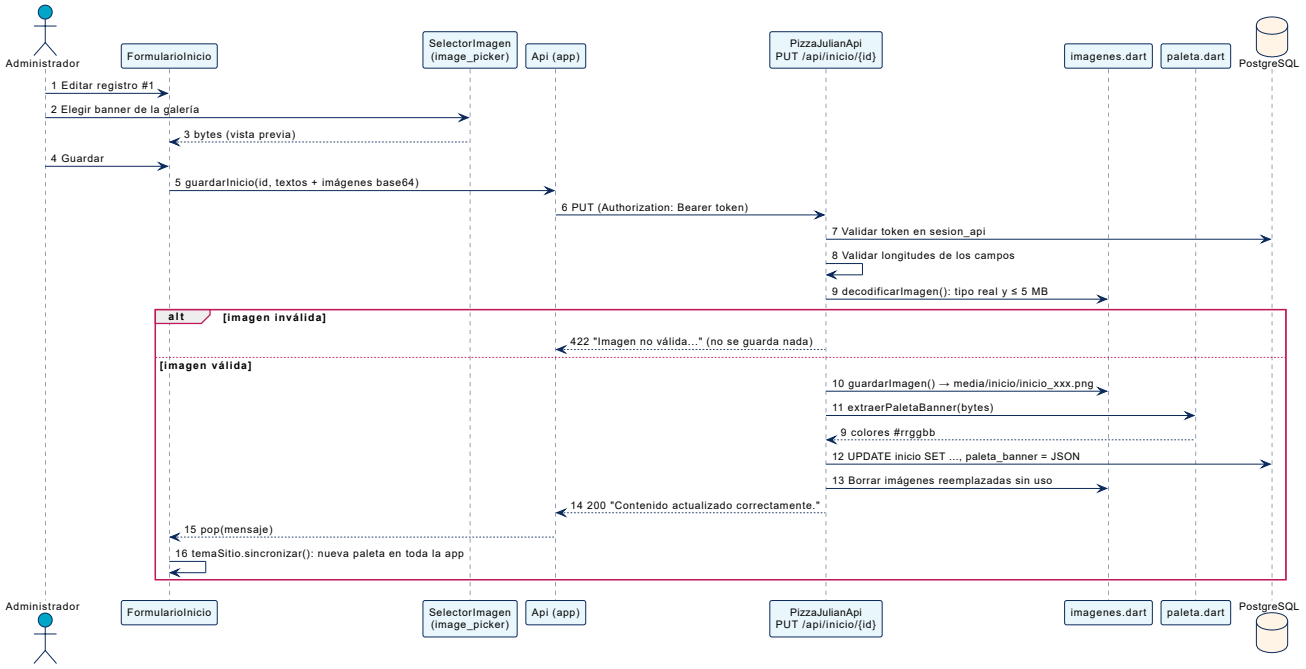


Figura 8. Secuencia: edición del contenido de Inicio con cambio de banner.

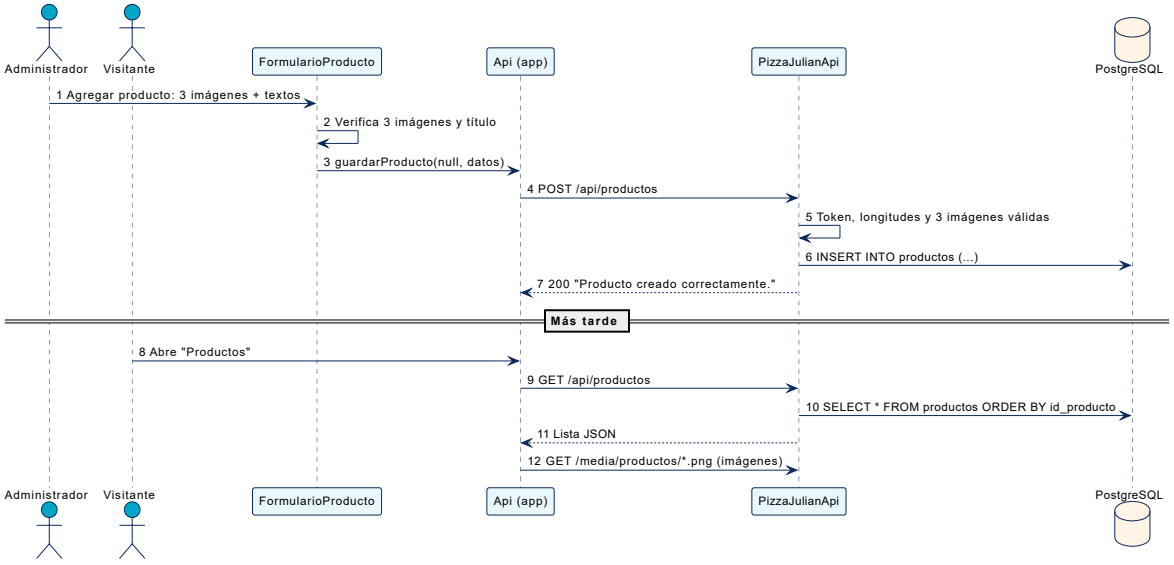


Figura 9. Secuencia: creación de un producto y consulta por el visitante.

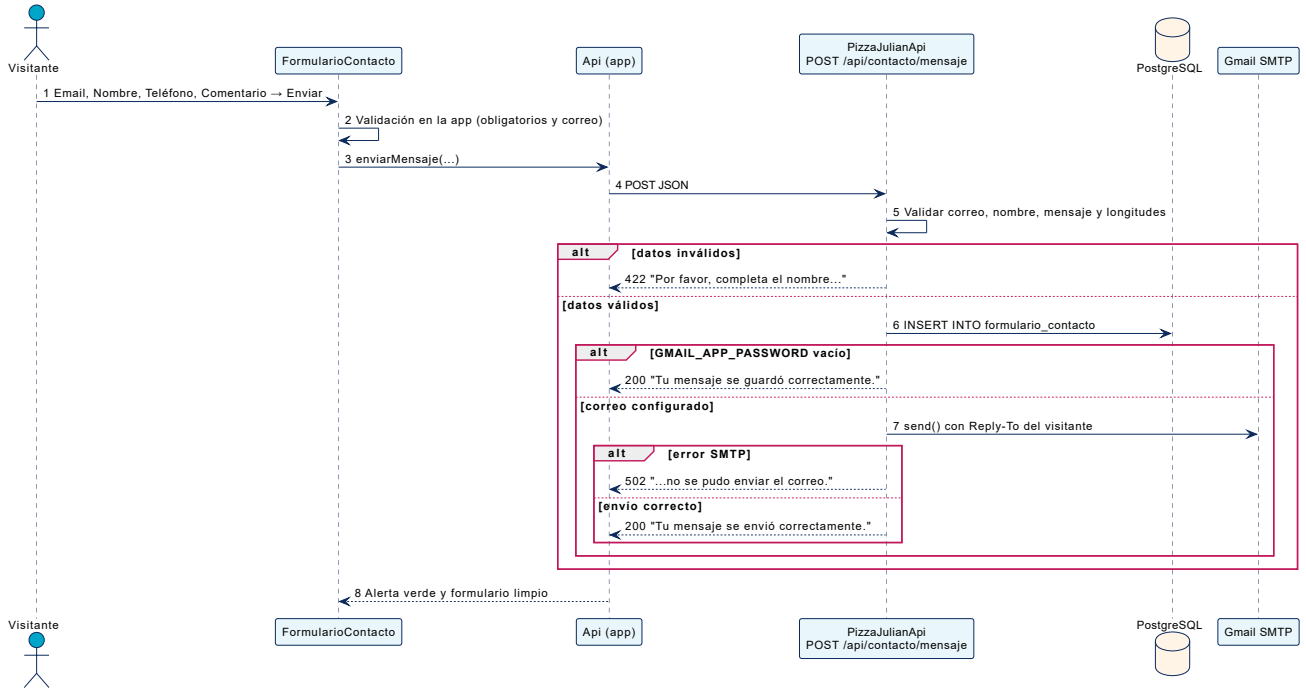


Figura 10. Secuencia: envío del formulario de contacto.

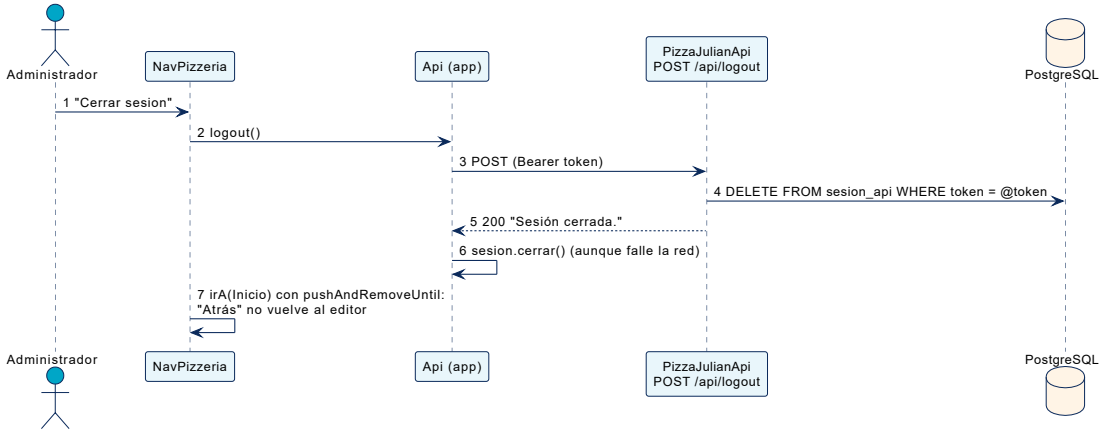


Figura 11. Secuencia: cierre de sesión.

6.6 Diagramas de actividad

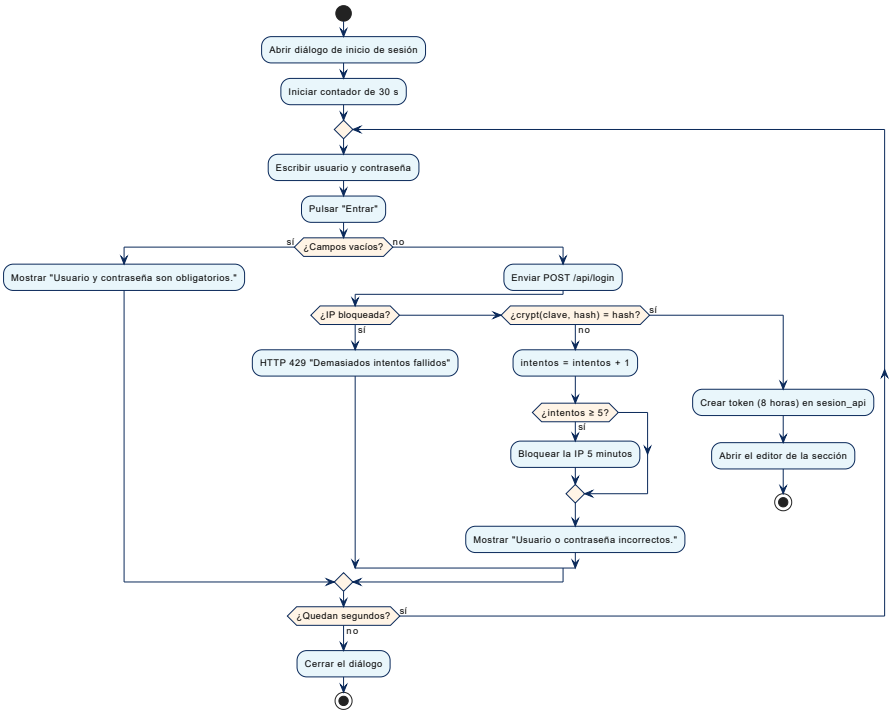


Figura 12. Actividad: proceso de inicio de sesión con bloqueo.

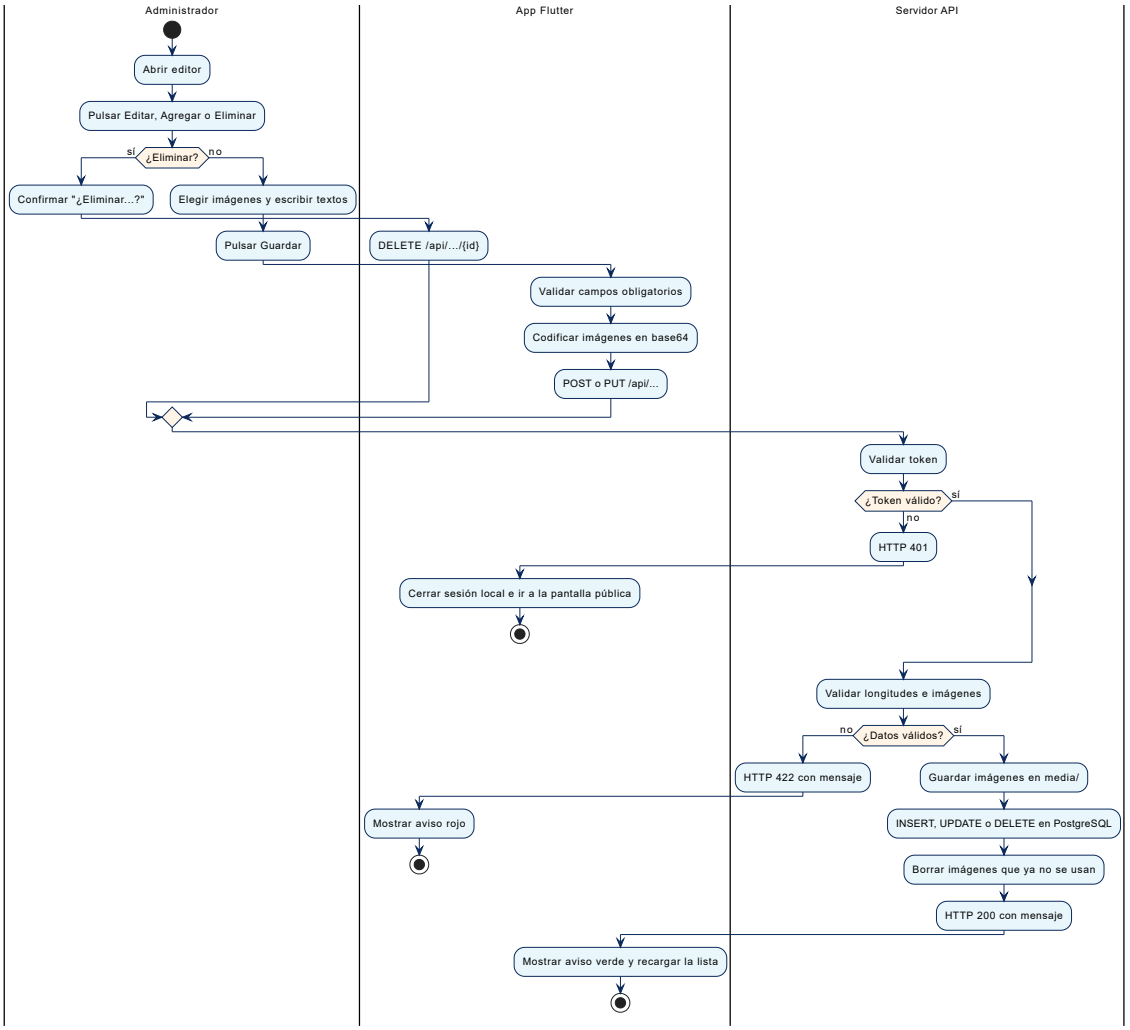


Figura 13. Actividad: operación CRUD en un editor (con particiones).

6.7 Diagrama de estados de la sesión

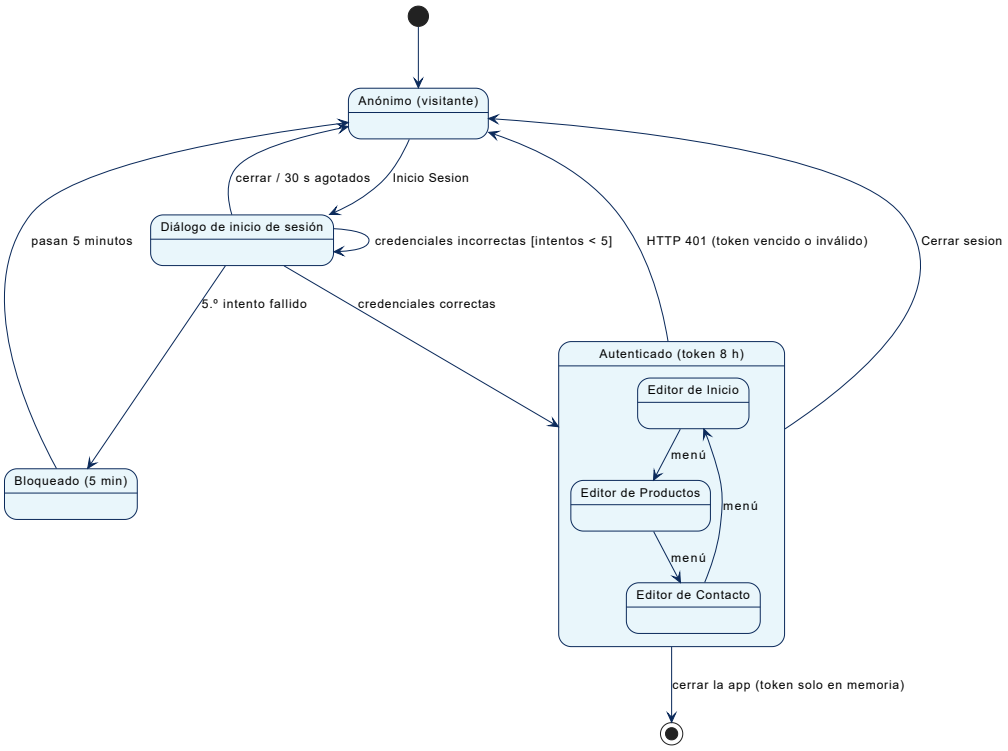


Figura 14. Máquina de estados de la sesión del administrador.

6.8 Diagrama de navegación

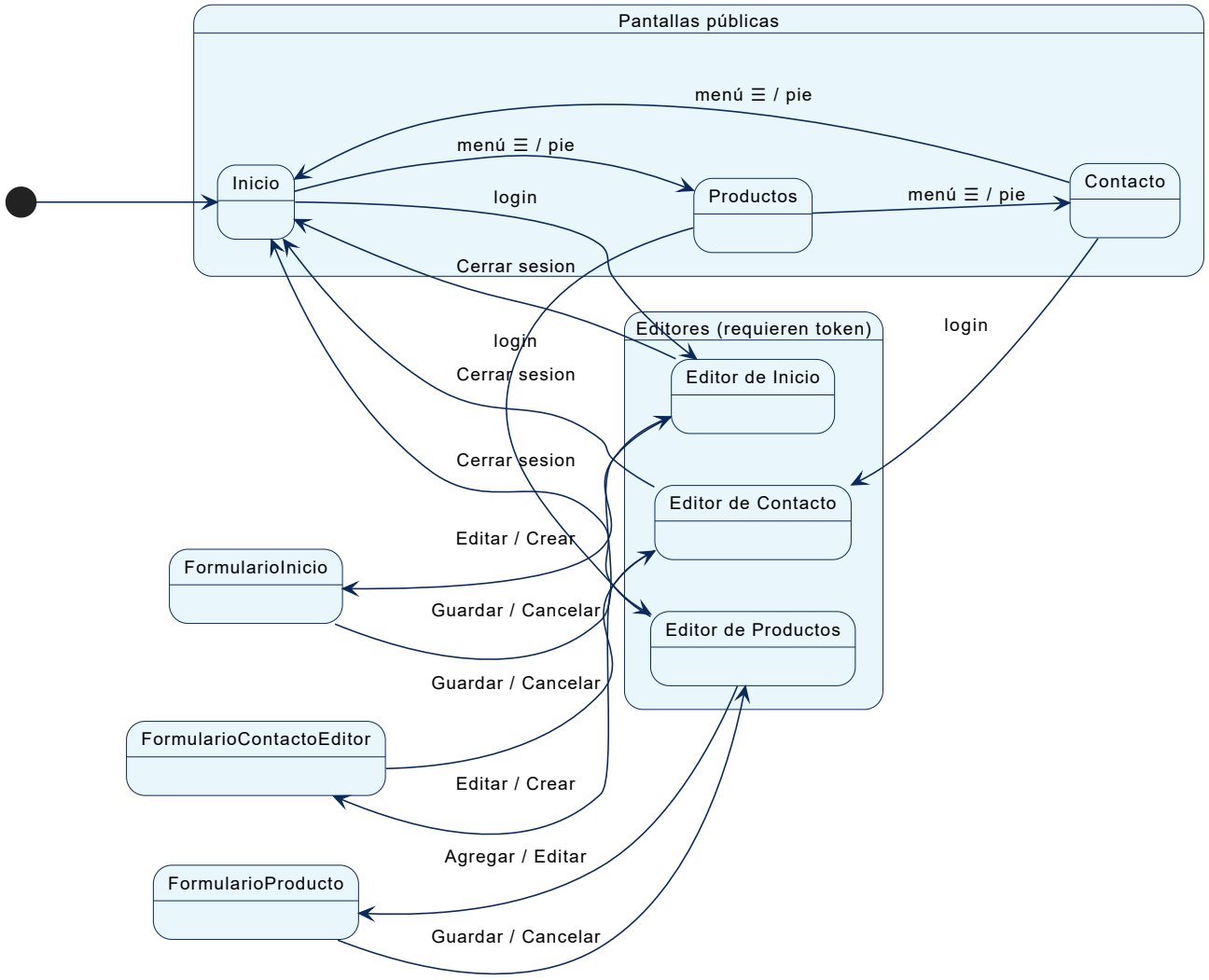


Figura 15. Navegación entre pantallas públicas, editores y formularios.

7. Desarrollo de la base de datos

7.1 Características generales

Parámetro	Valor
Nombre	pizza_julian
Motor	PostgreSQL 16 (imagen oficial postgres:16-alpine en Docker; también funciona en PostgreSQL 16 de Windows)
Codificación	UTF8 (base creada con `ENCODING 'UTF8' TEMPLATE template0`)
Extensiones	pgcrypto (bcrypt con `crypt` y `gen_salt`)
Tablas	6: usuario, sesion_api, inicio, productos, contacto, formulario_contacto
Relaciones	sesion_api.id_usuario → usuario.id_usuario (ON DELETE CASCADE). Las demás tablas son independientes (modelo de contenidos).
Acceso	Paquete `postgres` 3.5.7 de Dart con un pool de hasta 5 conexiones; datos de conexión en `.env`.
Origen	Migrada desde la base MariaDB `tecnico-2026-pagina` del sitio web.

Migración desde MariaDB

MariaDB (sitio web)	PostgreSQL (app)	Motivo
`` `formulario-contacto` ``, `` `carru-1` ``, `` `fecha-formulario` ``	`formulario_contacto`, `carru_1`, `fecha_formulario`	Sin guiones: no requieren comillas (recomendación BD7 del sitio web).
`int(11) AUTO_INCREMENT`	`INTEGER GENERATED ALWAYS AS IDENTITY`	Columna de identidad del estándar SQL.
`telefono_formulario bigint NOT NULL`	`VARCHAR(20) NOT NULL DEFAULT ''`	Corrige el hallazgo H05 del sitio web (teléfono vacío o con espacios).
`numero_whatsapp bigint`	`VARCHAR(20)`	Un teléfono no es una cantidad.
`timestamp ... ON UPDATE current_timestamp()`	`TIMESTAMPTZ DEFAULT now()`	Conserva la fecha original (recomendación BD6).
Tabla `contacto` ausente del respaldo	Incluida en el script	Recomendación BD1.
`usuario.usuario` sin índice único	`UNIQUE`	Recomendación BD4.
Contraseña `$2y$10$…`	`$2a$10$…`	Mismo bcrypt; pgcrypto reconoce el prefijo $2a$.
Sesión PHP (cookie)	Tabla `sesion_api`	Tokens para la app.

Tabla 12. Cambios de la migración.

7.2 Modelo entidad-relación

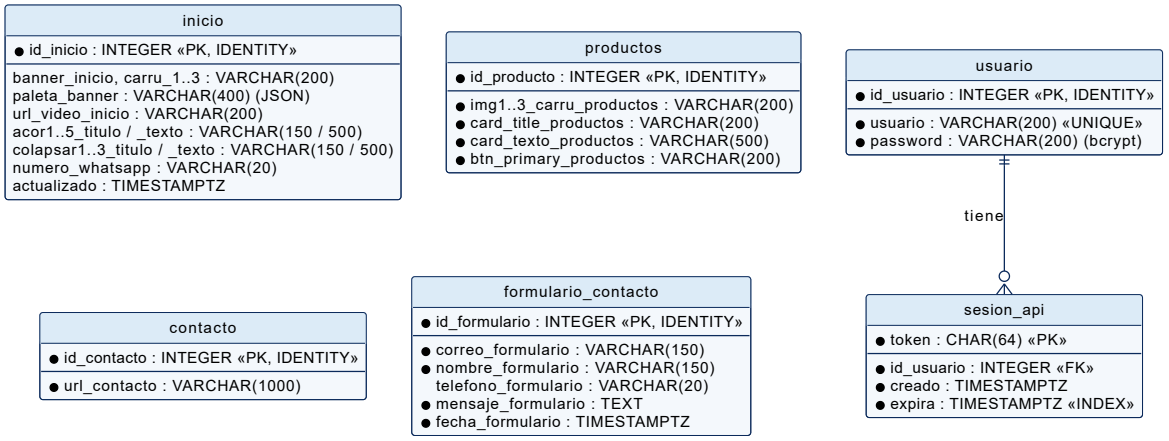


Figura 16. Modelo entidad-relación (físico) de la base de datos.

7.3 Diccionario de datos

Tabla	Campo	Tipo	Restricción	Descripción
usuario	id_usuario	INTEGER	PK, identidad	Identificador.
	usuario	VARCHAR(200)	NOT NULL, UNIQUE	Nombre de inicio de sesión.
	password	VARCHAR(200)	NOT NULL	Hash bcrypt ($2a$10$…, 60 caracteres).
sesion_api	token	CHAR(64)	PK	Token hexadecimal de 256 bits.
	id_usuario	INTEGER	FK → usuario, CASCADE	Dueño de la sesión.
	creado / expira	TIMESTAMPTZ	NOT NULL; índice en expira	Creación y vencimiento (+8 h).
inicio	id_inicio	INTEGER	PK, identidad	Identificador del registro de contenido.
	banner_inicio	VARCHAR(200)	NOT NULL DEFAULT ''	Ruta relativa del banner (inicio/…).
	paleta_banner	VARCHAR(400)	NOT NULL DEFAULT ''	JSON con 9 colores #rrggbb.
	carru_1, carru_2, carru_3	VARCHAR(200)	NOT NULL DEFAULT ''	Rutas de las imágenes del carrusel.
	url_video_inicio	VARCHAR(200)	NOT NULL DEFAULT ''	URL youtube.com/embed/…

Tabla	Campo	Tipo	Restricción	Descripción
	acorN_titulo_inicio (N=1..5)	VARCHAR(150)	NOT NULL DEFAULT ''	Título del elemento N del acordeón.
	acorN_texto_inicio (N=1..5)	VARCHAR(500)	NOT NULL DEFAULT ''	Texto del elemento N.
	colapsarN_titulo / _texto_inicio (N=1..3)	VARCHAR(150) / (500)	NOT NULL DEFAULT ''	Botón y contenido del desplegable N.
	numero_whatsapp	VARCHAR(20)	NOT NULL DEFAULT ''	Número con indicativo (solo dígitos).
	actualizado	TIMESTAMPTZ	DEFAULT now()	Última modificación.
productos	id_producto	INTEGER	PK, identidad	Identificador; define el orden.
	img1/img2/img3_carru_productos	VARCHAR(200)	NOT NULL	Rutas de las 3 imágenes (productos/…).
	card_title_productos	VARCHAR(200)	NOT NULL	Título de la tarjeta.
	card_texto_productos	VARCHAR(500)	NOT NULL	Descripción.
	btn_primary_productos	VARCHAR(200)	NOT NULL	Texto del botón.
contacto	id_contacto	INTEGER	PK, identidad	Identificador.
	url_contacto	VARCHAR(1000)	NOT NULL	URL embed de Google Maps.
formulario_contacto	id_formulario	INTEGER	PK, identidad	Identificador del mensaje.
	correo / nombre _formulario	VARCHAR(150)	NOT NULL	Correo y nombre del visitante.
	telefono_formulario	VARCHAR(20)	NOT NULL DEFAULT ''	Teléfono (opcional).
	mensaje_formulario	TEXT	NOT NULL	Mensaje (máx. 500 validado en la API).
	fecha_formulario	TIMESTAMPTZ	DEFAULT now()	Fecha de registro.

Tabla 13. Diccionario de datos.

7.4 Script DDL de creación

Script completo `base_de_datos/pizza_julian_postgres.sql`. Se ejecuta dentro de una transacción y puede repetirse: borra las tablas si existen y las vuelve a crear con los datos iniciales.

```
-- ======================================================================
-- Base de datos PostgreSQL para la app móvil PIZZA JULIAN (Flutter)
```

```
-- Basada en: pizza-julian/bd/tecnico-2026-pagina.sql (MariaDB)
--
-- Cambios respecto a la versión MariaDB:
--   * Nombres con guion pasan a guion bajo (PostgreSQL no los admite sin comillas):
--       `formulario-contacto` -> formulario_contacto, `carru-1` -> carru_1, `fecha-formulario` -> fecha_formulario
--   * AUTO_INCREMENT -> GENERATED ALWAYS AS IDENTITY
--   * Se agrega la tabla `contacto` (la usa contactos.php pero no estaba en el volcado)
--   * Se agrega la tabla `sesion_api` (tokens de inicio de sesión de la app móvil)
--   * Las contraseñas se verifican en la base con pgcrypto (bcrypt), igual que password_verify de PHP
--   * Las rutas de imágenes son relativas a la carpeta /media del servidor API
--
-- Uso (desde PowerShell):
--   & "C:\Program Files\PostgreSQL\16\bin\psql.exe" -U postgres -c "CREATE DATABASE pizza_julian ENCODING 'UTF8';"
--   & "C:\Program Files\PostgreSQL\16\bin\psql.exe" -U postgres -d pizza_julian -f base_de_datos\pizza_julian_postgres.sql
-- ======================================================================

SET client_encoding = 'UTF8';

BEGIN;

CREATE EXTENSION IF NOT EXISTS pgcrypto;

DROP TABLE IF EXISTS sesion_api;
DROP TABLE IF EXISTS formulario_contacto;
DROP TABLE IF EXISTS contacto;
DROP TABLE IF EXISTS productos;
DROP TABLE IF EXISTS inicio;
DROP TABLE IF EXISTS usuario;

-- ----------------------------------------------------------------------
-- Usuarios que pueden entrar a los editores de contenido
-- ----------------------------------------------------------------------
CREATE TABLE usuario (
  id_usuario  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  usuario     VARCHAR(200) NOT NULL UNIQUE,
  password    VARCHAR(200) NOT NULL -- hash bcrypt
);

-- Mismo usuario y hash del sitio web. PHP genera "$2y$" y pgcrypto usa "$2a$":
-- son el mismo algoritmo, por eso solo se cambia el prefijo y la contraseña sigue siendo la misma.
INSERT INTO usuario (usuario, password) VALUES
('alberdelfin', '$2a$10$eGR.EOb270JBg7t30CyVNO7Z9HK4j3L.APJsD12lsRX8OfGx4FzUG');

-- Para crear otro usuario o cambiar una contraseña:
--   INSERT INTO usuario (usuario, password) VALUES ('nuevo', crypt('su-clave', gen_salt('bf', 10)));
--   UPDATE usuario SET password = crypt('nueva-clave', gen_salt('bf', 10)) WHERE usuario = 'alberdelfin';

-- ----------------------------------------------------------------------
-- Tokens de sesión de la app (reemplazan la sesión PHP con cookie)
-- ----------------------------------------------------------------------
CREATE TABLE sesion_api (
  token       CHAR(64) PRIMARY KEY,
  id_usuario  INTEGER NOT NULL REFERENCES usuario(id_usuario) ON DELETE CASCADE,
  creado      TIMESTAMPTZ NOT NULL DEFAULT now(),
  expira      TIMESTAMPTZ NOT NULL
);
CREATE INDEX idx_sesion_api_expira ON sesion_api (expira);

-- ----------------------------------------------------------------------
-- Contenido de la pantalla Inicio (el primer registro define banner y paleta de toda la app)
-- ----------------------------------------------------------------------
CREATE TABLE inicio (
```

```
  id_inicio                INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  banner_inicio            VARCHAR(200) NOT NULL DEFAULT '',
  paleta_banner            VARCHAR(400) NOT NULL DEFAULT '',
  carru_1                  VARCHAR(200) NOT NULL DEFAULT '',
  carru_2                  VARCHAR(200) NOT NULL DEFAULT '',
  carru_3                  VARCHAR(200) NOT NULL DEFAULT '',
  url_video_inicio         VARCHAR(200) NOT NULL DEFAULT '',
  acor1_titulo_inicio      VARCHAR(150) NOT NULL DEFAULT '',
  acor1_texto_inicio       VARCHAR(500) NOT NULL DEFAULT '',
  acor2_titulo_inicio      VARCHAR(150) NOT NULL DEFAULT '',
  acor2_texto_inicio       VARCHAR(500) NOT NULL DEFAULT '',
  acor3_titulo_inicio      VARCHAR(150) NOT NULL DEFAULT '',
  acor3_texto_inicio       VARCHAR(500) NOT NULL DEFAULT '',
  acor4_titulo_inicio      VARCHAR(150) NOT NULL DEFAULT '',
  acor4_texto_inicio       VARCHAR(500) NOT NULL DEFAULT '',
  acor5_titulo_inicio      VARCHAR(150) NOT NULL DEFAULT '',
  acor5_texto_inicio       VARCHAR(500) NOT NULL DEFAULT '',
  colapsar1_titulo_inicio  VARCHAR(150) NOT NULL DEFAULT '',
  colapsar1_texto_inicio   VARCHAR(500) NOT NULL DEFAULT '',
  colapsar2_titulo_inicio  VARCHAR(150) NOT NULL DEFAULT '',
  colapsar2_texto_inicio   VARCHAR(500) NOT NULL DEFAULT '',
  colapsar3_titulo_inicio  VARCHAR(150) NOT NULL DEFAULT '',
  colapsar3_texto_inicio   VARCHAR(500) NOT NULL DEFAULT '',
  numero_whatsapp          VARCHAR(20)  NOT NULL DEFAULT '',
  actualizado              TIMESTAMPTZ  NOT NULL DEFAULT now()
);

INSERT INTO inicio (
  banner_inicio, carru_1, carru_2, carru_3, url_video_inicio,
  acor1_titulo_inicio, acor1_texto_inicio, acor2_titulo_inicio, acor2_texto_inicio,
  acor3_titulo_inicio, acor3_texto_inicio, acor4_titulo_inicio, acor4_texto_inicio,
  acor5_titulo_inicio, acor5_texto_inicio,
  colapsar1_titulo_inicio, colapsar1_texto_inicio, colapsar2_titulo_inicio, colapsar2_texto_inicio,
  colapsar3_titulo_inicio, colapsar3_texto_inicio, numero_whatsapp
) VALUES (
  'inicio/banner.png', 'inicio/1.jpeg', 'inicio/2.jpeg', 'inicio/3.jpeg', 'https://www.youtube.com/embed/crd
trzZj3fo',
  'Margarita (Margherita)', 'La reina de las pizzas. Salsa de tomate, mozzarella fresca, albahaca y aceite d
e oliva EUGENIO FERRO FALLA.',
  'Cuatro Quesos (Quattro Formaggi)', 'Una mezcla de quesos fundidos, usualmente mozzarella, gorgonzola (azu
l), fontina y parmesano.',
  'Napolitana', 'Salsa de tomate, anchoas, ajo, orégano y aceite de oliva (tradicionalmente no lleva ques
o).',
  'Mexicana', 'Incluye carne molida o chorizo, frijoles negros, jalapeños, cebolla y un toque de aguacate o
cilantro.',
  'Barbacoa (BBQ)', 'Sustituye la salsa de tomate por salsa barbacoa, acompañada de pollo, carne picada, ceb
olla y bacon.',
  'Vegetariana', 'Cubierta de vegetales frescos o salteados, como pimientos, cebolla, champiñones, aceitunas
y tomates cherry.',
  'Cuatro Estaciones (Quattro Stagioni)', 'Dividida en cuatro secciones, cada una representando una estación
con ingredientes distintos (por ejemplo: alcachofas, champiñones, jamón y albahaca).',
  'Funghi (Champiñones)', 'Base de mozzarella con abundantes hongos salteados, a veces con un toque de aceit
e de trufa.',
  '573132345685'
);

-- ------------------------------------------------------------------------
-- Tarjetas de la pantalla Productos (cada una con un carrusel de 3 imágenes)
-- ------------------------------------------------------------------------
CREATE TABLE productos (
  id_producto            INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  img1_carru_productos   VARCHAR(200) NOT NULL,
  img2_carru_productos   VARCHAR(200) NOT NULL,
  img3_carru_productos   VARCHAR(200) NOT NULL,
  card_title_productos   VARCHAR(200) NOT NULL,
```

```
  card_texto_productos   VARCHAR(500) NOT NULL,
  btn_primary_productos  VARCHAR(200) NOT NULL
);

-- Productos que el sitio tenía publicados en productos.html
INSERT INTO productos (img1_carru_productos, img2_carru_productos, img3_carru_productos, card_title_productos, card_texto_productos, btn_primary_productos) VALUES
('productos/pizza1.png',   'productos/pizza2.png',   'productos/pizza3.png',   'Pizza Napolitana', 'Salsa de tomate, anchoas, ajo, orégano y aceite de oliva.', 'Pedir ahora'),
('productos/pizza4.jpeg',  'productos/pizza5.png',   'productos/pizza6.png',   'Pizza Boloñesa', 'Salsa boloñesa de carne, mozzarella y orégano.', 'Pedir ahora'),
('productos/pizza7.jpeg',  'productos/pizza8.png',   'productos/pizza9.png',   'Pizza Hawaiana', 'Jamón, piña, mozzarella y salsa de tomate.', 'Pedir ahora'),
('productos/pizza10.jpeg', 'productos/pizza11.png',  'productos/pizza12.png',  'Pizza Vegetariana', 'Pimientos, cebolla, champiñones, aceitunas y tomates cherry.', 'Pedir ahora'),
('productos/pizza13.jpg',  'productos/pizza14.png',  'productos/pizza15.png',  'Pizza Ranchera', 'Carne, maíz tierno, tocineta y salsa ranchera.', 'Pedir ahora'),
('productos/pizza16.png',  'productos/pizza17.png',  'productos/pizza18.png',  'Pizza Tocineta', 'Tocineta crocante, mozzarella y salsa de la casa.', 'Pedir ahora'),
('productos/pizza19.jpeg', 'productos/pizza20.png',  'productos/pizza21.png',  'Pizza Aceitunas', 'Aceitunas negras y verdes, mozzarella y orégano.', 'Pedir ahora'),
('productos/pizza22.jpeg', 'productos/pizza23.png',  'productos/pizza24.png',  'Pizza de Todito', 'Un poco de todo: carnes, vegetales y mucho queso.', 'Pedir ahora'),
('productos/pizza25.jpeg', 'productos/pizza26.png',  'productos/pizza27.png',  'Pizza Trufa Andina', 'Salsa blanca, queso mozzarella, champinones y toque de trufa.', 'Pedir ahora'),
('productos/pizza28.jpeg', 'productos/pizza29.png',  'productos/pizza30.jpeg', 'Pizza Mediterranea Fina', 'Salsa pomodoro, aceitunas negras, tomate seco y hojas de albahaca.', 'Pedir ahora'),
('productos/pizza31.jpeg', 'productos/pizza32.png',  'productos/pizza33.jpeg', 'Pizza BBQ Ahumada', 'Salsa BBQ casera, pollo ahumado, cebolla morada y queso cheddar.', 'Pedir ahora'),
('productos/pizza43.png',  'productos/pizza44.png',  'productos/pizza45.png',  'Pizza Serrana', 'Jamón serrano, rúcula fresca, queso parmesano y aceite de oliva.', 'Pedir ahora'),
('productos/pizza34.jpeg', 'productos/pizza35.png',  'productos/pizza36.png',  'Pizza Cuatro Quesos Deluxe', 'Mozzarella, parmesano, provolone y azul con toque de oregano.', 'Pedir ahora'),
('productos/pizza37.png',  'productos/pizza38.png',  'productos/pizza39.png',  'Pizza Carbonara', 'Base cremosa, tocineta crocante, cebolla salteada y pimienta negra.', 'Pedir ahora'),
('productos/pizza40.png',  'productos/pizza41.png',  'productos/pizza42.png',  'Pizza Criolla Picante', 'Chorizo artesanal, pimenton rojo, maiz tierno y salsa picante de la casa.', 'Pedir ahora'),
('productos/pizza46.jpeg', 'productos/pizza47.jpeg', 'productos/pizza48.png',  'Pizza Marinera', 'Camarones al ajillo, salsa de tomate, mozzarella y perejil fresco.', 'Pedir ahora');

-- ------------------------------------------------------------------------
-- Mapa de la pantalla Contacto
-- ------------------------------------------------------------------------
CREATE TABLE contacto (
  id_contacto   INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  url_contacto  VARCHAR(1000) NOT NULL
);

INSERT INTO contacto (url_contacto) VALUES
('https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d18953.883843598873!2d-75.27853089699 08!3d2.941021717 2291766!2m3!1f0!2f0!3f0!3m2!1i1024!2i768!4f13.1!3m3!1m2!1s0x8e3b7464ac59aa5b%3A0x7e513d68543b6eec!2sDomino''s%20Pizza%20Las%20Brisas!5e0!3m2!1ses-419!2sco!4v1784748693148!5m2!1ses-419!2sco');

-- ------------------------------------------------------------------------
-- Mensajes que llegan desde el formulario de contacto
-- ------------------------------------------------------------------------
CREATE TABLE formulario_contacto (
  id_formulario        INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  correo_formulario    VARCHAR(150) NOT NULL,
  nombre_formulario    VARCHAR(150) NOT NULL,
  telefono_formulario  VARCHAR(20)  NOT NULL DEFAULT '',
  mensaje_formulario   TEXT         NOT NULL,
  fecha_formulario     TIMESTAMPTZ  NOT NULL DEFAULT now()
);

```

```
  167 INSERT INTO formulario_contacto (correo_formulario, nombre_formulario, telefono_formulario, mensaje_formular
io, fecha_formulario) VALUES
  168 ('delfin.alber@gmail.com', 'Prueba Automatica', '3132345685', 'Mensaje de prueba desde agente', '2026-08-28
14:37:37+00'),
  169 ('delfin.alber@gmail.com', 'DANNA LISETH PULIDO ZABALETA', '3132345685', 'Hola', '2026-08-28 14:40:29+00'),
  170 ('delfin.alber@gmail.com', 'ALBER DELFIN PEÑA ORTIGOZA', '3132345685', 'Hola un saludo', '2026-08-28 14:48:5
2+00'),
  171 ('g46113026@gmail.com', 'CUALQUIERA', '3213386504', 'Hola checho estamos en el Tecnico Programando con PH
P.', '2026-09-02 18:30:40+00');
  172
  173 COMMIT;
```

7.5 Consultas SQL utilizadas por la API

Ruta	Operación	Sentencia (resumen)
GET /api/tema	Leer tema	SELECT banner_inicio, paleta_banner, numero_whatsapp FROM inicio ORDER BY id_inicio LIMIT 1
GET /api/inicio	Leer contenido	SELECT * FROM inicio ORDER BY id_inicio LIMIT 1
POST/PUT/DELETE /api/inicio	CRUD	INSERT INTO inicio (…) VALUES (@…); UPDATE inicio SET … , actualizado = now() WHERE id_inicio = @id_inicio; DELETE FROM inicio WHERE id_inicio = @id
GET /api/productos	Listar	SELECT * FROM productos ORDER BY id_producto
POST/PUT/DELETE /api/productos	CRUD	INSERT / UPDATE / DELETE … productos … WHERE id_producto = @id
GET/POST/PUT /api/contacto	Leer / crear / editar	SELECT * FROM contacto LIMIT 1; INSERT INTO contacto (url_contacto) VALUES (@url); UPDATE contacto SET url_contacto = @url WHERE id_contacto = @id
POST /api/contacto/mensaje	Registrar mensaje	INSERT INTO formulario_contacto (correo…, nombre…, telefono…, mensaje…) VALUES (@correo, @nombre, @telefono, @mensaje)
POST /api/login	Autenticar	SELECT id_usuario, usuario FROM usuario WHERE usuario = @usuario AND password = crypt(@password, password); DELETE FROM sesion_api WHERE expira <= now(); INSERT INTO sesion_api …
Todas las rutas protegidas	Validar token	SELECT id_usuario FROM sesion_api WHERE token = @token AND expira > now()
POST /api/logout	Cerrar sesión	DELETE FROM sesion_api WHERE token = @token

Tabla 14. Consultas de la API (todas con parámetros nombrados).

7.6 Análisis de normalización

- **Primera forma normal:** los valores son atómicos, salvo `paleta_banner` (JSON), que se trata como un único valor de configuración.
- **Tabla inicio:** los grupos `acor1..5` y `colapsar1..3` son grupos repetidos de columnas; se mantuvieron para conservar la compatibilidad con el sitio web. En una versión futura conviene una tabla `inicio_item(id, tipo, orden, titulo, texto)`.

- **Tabla productos:** las 3 columnas de imagen podrían pasar a `producto_imagen(id_producto FK, orden, ruta)` para admitir N imágenes.
- **Segunda y tercera forma normal:** se cumplen (llaves simples y sin dependencias transitivas).

7.7 Recomendaciones para la base de datos

N.º	Recomendación	Prioridad
BD1	Crear un usuario de base de datos exclusivo para la API con permisos SELECT, INSERT, UPDATE y DELETE, en lugar de `postgres`.	Alta
BD2	Cambiar la contraseña del contenedor (`pizzajulian`) por una robusta en producción.	Alta
BD3	Guardar en `sesion_api` el hash SHA-256 del token y no el token (H07).	Media
BD4	Programar la limpieza periódica de sesiones vencidas y de mensajes antiguos según la política de datos.	Baja
BD5	Normalizar `inicio` y `productos` (numeral 7.6) cuando se requieran más elementos.	Baja

Tabla 15. Recomendaciones para la base de datos.

```
-- Usuario de base de datos con privilegios mínimos (BD1)
CREATE USER pizza_app WITH PASSWORD 'Contraseña-Segura-2026';
GRANT CONNECT ON DATABASE pizza_julian TO pizza_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO pizza_app;
-- y en pizza_julian_api/.env:  DB_USER=pizza_app  DB_PASSWORD=Contraseña-Segura-2026
```

7.8 Copias de seguridad

```
# Respaldo completo (PowerShell)
docker exec pizza_julian_db pg_dump -U postgres -d pizza_julian --clean --if-exists -f /tmp/respaldo.sql
docker cp pizza_julian_db:/tmp/respaldo.sql .\base_de_datos\respaldo_2026-11-08.sql
# Restauración
docker cp .\base_de_datos\respaldo_2026-11-08.sql pizza_julian_db:/tmp/respaldo.sql
docker exec pizza_julian_db psql -U postgres -d pizza_julian -v ON_ERROR_STOP=1 -f /tmp/respaldo.sql
```

Respaldar también la carpeta `pizza_julian_api/media`, porque la base de datos solo guarda las rutas de las imágenes. Los datos del contenedor persisten en el volumen `pizza_julian_datos` aunque el contenedor se borre. Frecuencia recomendada: semanal y antes de cada cambio.

8. Explicación de la codificación

8.1 Ciclo de una petición

Toda operación sigue el mismo camino: (1) un widget de la app llama a un método de `Api` (`lib/api.dart`); (2) `_enviar()` arma la petición HTTP con JSON UTF-8 y, si hay sesión, la cabecera `Authorization: Bearer <token>`; (3) en el servidor, los middleware registran la petición (`logRequests`), agregan las cabeceras CORS y capturan los errores; (4) `shelf_router` dirige la petición a su método; (5) el método valida el token si la ruta es protegida, valida los datos, consulta PostgreSQL con parámetros nombrados y responde `{"ok": true|false, "mensaje": …}`; (6) la app convierte las respuestas con error en `ApiError` y las muestra como avisos.

8.2 bin/server.dart y config.dart – arranque y configuración

```dart
import 'dart:io';

import 'package:pizza_julian_api/api.dart';
import 'package:pizza_julian_api/config.dart';
import 'package:postgres/postgres.dart';
import 'package:shelf/shelf_io.dart' as io;

Future<void> main() async {
  final config = Config.cargar();

  final db = Pool.withEndpoints(
    [
      Endpoint(
        host: config.dbHost,
        port: config.dbPort,
        database: config.dbName,
        username: config.dbUser,
        password: config.dbPassword,
      ),
    ],
    settings: const PoolSettings(maxConnectionCount: 5, sslMode: SslMode.disable),
  );

  // Verifica la conexión antes de aceptar peticiones
  try {
    await db.execute('SELECT 1');
  } catch (e) {
    stderr.writeln('No fue posible conectar con PostgreSQL (${config.dbHost}:${config.dbPort}/${config.dbNam
e}): $e');
    stderr.writeln('Revise el archivo .env (DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD).');
    exit(1);
  }

  final api = PizzaJulianApi(config, db);
  // 0.0.0.0 para que el celular o el emulador puedan llegar al servidor por la red local
  final servidor = await io.serve(api.handler, InternetAddress.anyIPv4, config.puerto);
  servidor.autoCompress = true;
  print('API Pizza Julian escuchando en http://${servidor.address.host}:${servidor.port}');
}
```

Elemento	Descripción
Config.cargar()	Lee el archivo `.env` del directorio actual; las variables de entorno del sistema tienen prioridad.
Pool.withEndpoints	Pool de hasta 5 conexiones reutilizables a PostgreSQL (sin SSL en red local).
SELECT 1	Verifica la conexión antes de aceptar peticiones; si falla, explica qué revisar en `.env` y termina.
InternetAddress.anyIPv4	Escucha en todas las interfaces para que el celular llegue por la red Wi-Fi.

8.3 Autenticación – POST /api/login

```
  Future<Response> _login(Request request) async {
    final info = request.context['shelf.io.connection_info'] as HttpConnectionInfo?;
    final ip = info?.remoteAddress.address ?? 'desconocida';
    final intentos = _intentosPorIp.putIfAbsent(ip, _Intentos.new);

    // Bloqueo temporal tras varios intentos fallidos (mitiga fuerza bruta)
    if (DateTime.now().isBefore(intentos.bloqueoHasta)) {
      return _error('Demasiados intentos fallidos. Intente de nuevo en unos minutos.', estado: 429);
    }

    final datos = await _cuerpo(request);
    final usuario = _texto(datos, 'usuario');
    final password = (datos['password'] ?? '').toString();
    if (usuario.isEmpty || password.isEmpty || usuario.length > 200 || password.length > 200) {
      return _error('Usuario y contraseña son obligatorios.');
    }

    // crypt() de pgcrypto compara contra el hash bcrypt guardado, nunca en texto plano
    final filas = await _consulta(
      'SELECT id_usuario, usuario FROM usuario WHERE usuario = @usuario AND password = crypt(@password, pass
word) LIMIT 1',
      {'usuario': usuario, 'password': password},
    );
    if (filas.isEmpty) {
      intentos.fallidos++;
      if (intentos.fallidos >= 5) {
        intentos.bloqueoHasta = DateTime.now().add(const Duration(minutes: 5));
        intentos.fallidos = 0;
      }
      return _error('Usuario o contraseña incorrectos.', estado: 401);
    }
    _intentosPorIp.remove(ip);

    final token = List.generate(32, (_) => _aleatorio.nextInt(256).toRadixString(16).padLeft(2, '0')).join
();
    await db.execute('DELETE FROM sesion_api WHERE expira <= now()');
    await db.execute(
      Sql.named('INSERT INTO sesion_api (token, id_usuario, expira) VALUES (@token, @id, @expira)'),
      parameters: {'token': token, 'id': filas.first['id_usuario'], 'expira': DateTime.now().toUtc().add(_du
racionSesion)},
    );
    return _json({'ok': true, 'mensaje': 'Inicio de sesión correcto.', 'token': token, 'usuario': filas.firs
t['usuario']});
  }
```

Puntos clave: bloqueo de 300 s por IP tras 5 fallos (429); límite de 200 caracteres; la contraseña se compara **dentro de PostgreSQL** con `crypt(@password, password)`, que recalcula el hash bcrypt con la sal guardada;

el token de 32 bytes se genera con `Random.secure()` y se guarda con vencimiento de 8 horas; las sesiones vencidas se borran en cada inicio de sesión.

8.4 Autorización y cierre de sesión

```
  186   // Devuelve el id del usuario si el token "Authorization: Bearer ..." es válido
  187   Future<int?> _usuarioAutenticado(Request request) async {
  188     final cabecera = request.headers['authorization'] ?? '';
  189     if (!cabecera.startsWith('Bearer ')) {
  190       return null;
  191     }
  192     final token = cabecera.substring(7).trim();
  193     if (!RegExp(r'^[0-9a-f]{64}$').hasMatch(token)) {
  194       return null;
  195     }
  196     final filas = await _consulta('SELECT id_usuario FROM sesion_api WHERE token = @token AND expira > now
()', {'token': token});
  197     return filas.isEmpty ? null : filas.first['id_usuario'] as int;
  198   }
  199
  200   Response _noAutorizado() => _error('La sesión expiró, inicie sesión nuevamente.', estado: 401);
```

```
  273   Future<Response> _logout(Request request) async {
  274     final cabecera = request.headers['authorization'] ?? '';
  275     if (cabecera.startsWith('Bearer ')) {
  276       await db.execute(Sql.named('DELETE FROM sesion_api WHERE token = @token'), parameters: {'token': cabec
era.substring(7).trim()});
  277     }
  278     return _json({'ok': true, 'mensaje': 'Sesión cerrada.'});
  279   }
```

Cada ruta de los editores comienza con `if (await _usuarioAutenticado(request) == null) return _noAutorizado();`. En la app, `Api._enviar()` cierra la sesión local al recibir un 401 y los editores regresan a la pantalla pública (`manejarErrorEditor`).

8.5 imagenes.dart – carga segura de imágenes

```
   12 class ErrorImagen implements Exception {
   13   ErrorImagen(this.mensaje);
   14   final String mensaje;
   15 }
   16
   17 final _aleatorio = Random.secure();
   18
   19 String _extension(img.ImageFormat formato) {
   20   switch (formato) {
   21     case img.ImageFormat.jpg:
   22       return 'jpg';
   23     case img.ImageFormat.png:
   24       return 'png';
   25     case img.ImageFormat.gif:
   26       return 'gif';
   27     case img.ImageFormat.webp:
   28       return 'webp';
   29     default:
   30       throw ErrorImagen('Imagen no válida: usa archivos JPG, PNG, GIF o WEBP de máximo 5 MB. No se guardaron
cambios.');
   31   }
   32 }
   33
```

```dart
// Decodifica la imagen en base64 y valida tipo y tamaño (no se confía en la extensión que envía la app)
Uint8List decodificarImagen(String base64Imagen) {
  final Uint8List bytes;
  try {
    bytes = base64Decode(base64Imagen.contains(',') ? base64Imagen.split(',').last : base64Imagen);
  } on FormatException {
    throw ErrorImagen('Imagen no válida: usa archivos JPG, PNG, GIF o WEBP de máximo 5 MB. No se guardaron cambios.');
  }
  if (bytes.isEmpty || bytes.length > tamanoMaximoImagen) {
    throw ErrorImagen('Imagen no válida: usa archivos JPG, PNG, GIF o WEBP de máximo 5 MB. No se guardaron cambios.');
  }
  _extension(img.findFormatForData(bytes));
  return bytes;
}

// Guarda la imagen en media/<carpeta>/ con nombre aleatorio y devuelve la ruta relativa ("productos/producto_xxx.png")
Future<String> guardarImagen(Uint8List bytes, String carpetaMedia, String carpeta, String prefijo) async {
  final extension = _extension(img.findFormatForData(bytes));
  final sufijo = List.generate(8, (_) => _aleatorio.nextInt(256).toRadixString(16).padLeft(2, '0')).join();
  final nombre = '$prefijo${DateTime.now().microsecondsSinceEpoch.toRadixString(16)}$sufijo.$extension';
  final directorio = Directory('$carpetaMedia/$carpeta');
  await directorio.create(recursive: true);
  await File('${directorio.path}/$nombre').writeAsBytes(bytes, flush: true);
  return '$carpeta/$nombre';
}

// Ruta relativa segura dentro de una carpeta de media (evita "../")
bool rutaValida(String ruta, String carpeta) => RegExp('^$carpeta/[A-Za-z0-9._-]+\$').hasMatch(ruta);

Future<void> borrarArchivos(Iterable<String> rutas, String carpetaMedia, String carpeta, {Set<String> protegidas = const {}}) async {
  for (final ruta in rutas) {
    if (!rutaValida(ruta, carpeta) || protegidas.contains(ruta.split('/').last)) {
      continue;
    }
    final archivo = File('$carpetaMedia/$ruta');
    if (await archivo.exists()) {
      await archivo.delete();
    }
  }
}
```

Control	Descripción
Tamaño	Se rechaza si está vacía o supera 5 MB (después de decodificar el base64).
Tipo real	`findFormatForData` lee la firma binaria del archivo: un texto renombrado como .jpg se rechaza (prueba P16).
Nombre aleatorio	Prefijo + marca de tiempo + 8 bytes aleatorios: evita colisiones y que el nombre original llegue al servidor.
rutaValida()	Solo acepta rutas `carpeta/archivo` con caracteres seguros: impide borrar archivos fuera de `media` con `../`.

8.6 Editores de contenido (CRUD)

El guardado de productos ilustra el patrón usado en los tres editores: validar todo antes de escribir, guardar las imágenes, ejecutar el SQL y, si algo falla, borrar las imágenes recién guardadas (transacción lógica). Tras editar, se borran las imágenes reemplazadas que ningún registro usa.

```
  421   Future<Response> _guardarProducto(Request request, int? id) async {
  422     if (await _usuarioAutenticado(request) == null) return _noAutorizado();
  423     final datos = await _cuerpo(request);
  424     final errorLongitud = _validarLongitudes(datos, _camposTextoProductos);
  425     if (errorLongitud != null) return errorLongitud;
  426 
  427     Map<String, dynamic> actual = {};
  428     if (id != null) {
  429       final filas = await _consulta('SELECT * FROM productos WHERE id_producto = @id', {'id': id});
  430       if (filas.isEmpty) return _error('El producto no existe.', estado: 404);
  431       actual = filas.first;
  432     }
  433 
  434     final nuevas = _imagenesNuevas(datos, _camposImagenProductos);
  435     for (final campo in _camposImagenProductos) {
  436       if (!nuevas.containsKey(campo) && (actual[campo] ?? '').toString().isEmpty) {
  437         return _error('Cada producto necesita las 3 imágenes del carrusel. No se guardaron cambios.', estad
o: 422);
  438       }
  439     }
  440     if (_texto(datos, 'card_title_productos').isEmpty) {
  441       return _error('El título del producto es obligatorio.', estado: 422);
  442     }
  443 
  444     final valores = <String, dynamic>{};
  445     final guardadas = <String>[];
  446     try {
  447       for (final campo in _camposImagenProductos) {
  448         valores[campo] = nuevas.containsKey(campo)
  449             ? (guardadas..add(await guardarImagen(nuevas[campo]!, config.carpetaMedia, 'productos', 'product
o_'))).last
  450             : (actual[campo] ?? '').toString();
  451       }
  452       for (final campo in _camposTextoProductos) {
  453         valores[campo] = _texto(datos, campo);
  454       }
  455       final columnas = valores.keys.toList();
  456       if (id == null) {
  457         await db.execute(
  458           Sql.named('INSERT INTO productos (${columnas.join(', ')}) VALUES (${columnas.map((c) => '@$c').joi
n(', ')})'),
  459           parameters: valores,
  460         );
  461       } else {
  462         await db.execute(
  463           Sql.named('UPDATE productos SET ${columnas.map((c) => '$c = @$c').join(', ')} WHERE id_producto =
@id_producto'),
  464           parameters: {...valores, 'id_producto': id},
  465         );
  466       }
  467     } catch (_) {
  468       await borrarArchivos(guardadas, config.carpetaMedia, 'productos');
  469       rethrow;
  470     }
  471 
  472     if (id != null) {
  473       await _borrarNoUsadas('productos', _camposImagenProductos, _camposImagenProductos.map((c) => (actual
```

```
[c] ?? '').toString()), 'productos', const {});
  474     }
  475     return _json({'ok': true, 'mensaje': id == null ? 'Producto creado correctamente.' : 'Producto actualiza
do correctamente.'});
  476   }
```

8.7 paleta.dart y TemaSitio – banner y paleta dinámica

`extraerPaletaBanner()` reduce el banner a 160 píxeles de ancho, arma un histograma de tonos (36 grupos de 10°) con los píxeles de color vivo, toma el tono dominante y deriva nueve colores ajustando su luminosidad para mantener el contraste (luminancia relativa WCAG). Es la misma lógica de `tema.js` del sitio web, trasladada al servidor. En la app, `TemaSitio.sincronizar()` consulta `/api/tema` y, si cambió la versión, notifica a todos los widgets para que se redibujen con los colores nuevos.

```
  103 // Devuelve la paleta con los tonos del banner, o null si la imagen no tiene colores definidos
  104 Map<String, String>? extraerPaletaBanner(Uint8List bytes) {
  105   final original = img.decodeImage(bytes);
  106   if (original == null || original.width == 0) {
  107     return null;
  108   }
  109   const ancho = 160;
  110   final alto = math.max(1, (original.height * ancho / original.width).round());
  111   final imagen = img.copyResize(original, width: ancho, height: alto);
  112
  113   // histograma de tonos (36 grupos de 10°) de los píxeles con color vivo
  114   final pesos = List<double>.filled(36, 0);
  115   for (final pixel in imagen) {
  116     final alfa = pixel.aNormalized.toDouble();
  117     final hsl = _rgbAHsl(pixel.rNormalized * alfa, pixel.gNormalized * alfa, pixel.bNormalized * alfa);
  118     if (hsl[1] > 0.45 && hsl[2] > 0.25 && hsl[2] < 0.75) {
  119       pesos[(hsl[0] / 10).floor() % 36] += hsl[1];
  120     }
  121   }
  122   final maximo = pesos.reduce(math.max);
  123   if (maximo < ancho * alto * 0.01) {
  124     return null; // imagen casi en blanco y negro: se usa la paleta por defecto
  125   }
  126
  127   // tono principal: promedio circular del grupo dominante y sus vecinos
  128   final principal = pesos.indexOf(maximo);
  129   var sumaX = 0.0, sumaY = 0.0;
  130   for (final desplazamiento in [-1, 0, 1]) {
  131     final grupo = (principal + desplazamiento + 36) % 36;
  132     final angulo = (grupo * 10 + 5) * math.pi / 180;
  133     sumaX += math.cos(angulo) * pesos[grupo];
  134     sumaY += math.sin(angulo) * pesos[grupo];
  135   }
  136   final tonoNaranja = (math.atan2(sumaY, sumaX) * 180 / math.pi + 360) % 360;
  137
  138   // entre los tonos importantes, el más oscuro hace de "rojo" y el más claro de "amarillo"
  139   double luzDeTono(double tono) => _luminancia(_hslAHex(tono, 1, 0.5));
  140   var tonoRojo = tonoNaranja, tonoAmarillo = tonoNaranja;
  141   for (var grupo = 0; grupo < 36; grupo++) {
  142     if (pesos[grupo] < maximo * 0.15) {
  143       continue;
  144     }
  145     final tono = grupo * 10 + 5.0;
  146     if (luzDeTono(tono) < luzDeTono(tonoRojo)) tonoRojo = tono;
  147     if (luzDeTono(tono) > luzDeTono(tonoAmarillo)) tonoAmarillo = tono;
  148   }
  149   if (_distanciaTono(tonoRojo, tonoNaranja) < 15) {
  150     tonoRojo = luzDeTono(tonoNaranja - 25) < luzDeTono(tonoNaranja + 25) ? tonoNaranja - 25 : tonoNaranja +
```

```
25;
  151    }
  152    if (_distanciaTono(tonoAmarillo, tonoNaranja) < 10) {
  153      tonoAmarillo = luzDeTono(tonoNaranja - 15) > luzDeTono(tonoNaranja + 15) ? tonoNaranja - 15 : tonoNaranj
a + 15;
  154    }
  155
  156    // mismas saturaciones y luces que el diseño original: solo cambian los tonos
  157    final naranja = _colorConLuminancia(tonoNaranja, 1, 0.55, 0.02, (lum) => lum >= 0.38);
  158    return {
  159      'negro': _hslAHex(tonoRojo, 0.25, 0.047),
  160      'negro-suave': _hslAHex(tonoRojo, 0.21, 0.084),
  161      'naranja': naranja,
  162      'amarillo': _colorConLuminancia(tonoAmarillo, 1, 0.55, 0.02, (lum) => lum >= math.max(0.55, _luminancia
(naranja) + 0.1)),
  163      'rojo': _colorConLuminancia(tonoRojo, 0.91, 0.445, -0.02, (lum) => lum <= 0.16),
  164      'rojo-profundo': _colorConLuminancia(tonoRojo, 0.88, 0.3, -0.02, (lum) => lum <= 0.07),
  165      'rojo-vivo': _colorConLuminancia(tonoRojo, 1, 0.59, -0.02, (lum) => lum <= 0.3),
  166      'blanco': _hslAHex(tonoNaranja, 0.2, 0.96),
  167      'gris': _hslAHex(tonoNaranja, 0.17, 0.89),
  168    };
  169  }
```

8.8 Formulario de contacto – registro y correo

```
  516    Future<Response> _mensajeContacto(Request request) async {
  517      final datos = await _cuerpo(request);
  518      final correo = _texto(datos, 'correo');
  519      final nombre = _texto(datos, 'nombre');
  520      final telefono = _texto(datos, 'telefono');
  521      final mensaje = _texto(datos, 'mensaje');
  522
  523      final correoValido = RegExp(r'^[^@\s]+@[^@\s]+\.[^@\s]+$').hasMatch(correo);
  524      if (!correoValido || nombre.isEmpty || mensaje.isEmpty) {
  525        return _error('Por favor, completa el nombre, un correo válido y el mensaje.', estado: 422);
  526      }
  527      if (correo.length > 150 || nombre.length > 150 || telefono.length > 20 || mensaje.length > 500) {
  528        return _error('Algún campo supera la longitud permitida.', estado: 422);
  529      }
  530
  531      await db.execute(
  532        Sql.named('INSERT INTO formulario_contacto (correo_formulario, nombre_formulario, telefono_formulario,
mensaje_formulario) '
  533            'VALUES (@correo, @nombre, @telefono, @mensaje)'),
  534        parameters: {'correo': correo, 'nombre': nombre, 'telefono': telefono, 'mensaje': mensaje},
  535      );
  536
  537      if (config.gmailClave.isEmpty) {
  538        stderr.writeln('No está configurada GMAIL_APP_PASSWORD: el mensaje se guardó pero no se envió por corr
eo.');
  539        return _json({'ok': true, 'mensaje': 'Tu mensaje se guardó correctamente.'});
  540      }
  541
  542      try {
  543        final correoSalida = Message()
  544          ..from = Address(config.gmailUsuario, 'Formulario PIZZERIA-JULIAN')
  545          ..recipients.add(config.correoDestino)
  546          ..headers = {'Reply-To': '$nombre <$correo>'}
  547          ..subject = 'Nuevo mensaje del formulario de contacto'
  548          ..text = 'Nombre: $nombre\nCorreo: $correo\nTeléfono: $telefono\n\nMensaje:\n$mensaje';
  549        await send(correoSalida, gmail(config.gmailUsuario, config.gmailClave));
  550      } catch (e) {
  551        stderr.writeln('Error SMTP del formulario de contacto: $e');
  552        return _error('La información se guardó, pero no se pudo enviar el correo.', estado: 502);
```

```
    }
    return _json({'ok': true, 'mensaje': 'Tu mensaje se envió correctamente.'});
  }
}
```

8.9 Cliente de la API en la app (lib/api.dart)

```
class Api {
  static const _tiempoEspera = Duration(seconds: 30);

  Map<String, String> get _cabeceras => {
        'content-type': 'application/json; charset=utf-8',
        if (sesion.token != null) 'authorization': 'Bearer ${sesion.token}',
      };

  Future<Map<String, dynamic>> _enviar(String metodo, String ruta, [Map<String, dynamic>? cuerpo]) async {
    final uri = Uri.parse('$apiUrl/api/$ruta');
    final http.Response respuesta;
    try {
      final peticion = http.Request(metodo, uri)..headers.addAll(_cabeceras);
      if (cuerpo != null) {
        peticion.body = jsonEncode(cuerpo);
      }
      respuesta = await http.Response.fromStream(await peticion.send().timeout(_tiempoEspera));
    } catch (_) {
      throw ApiError('Error de comunicación con el servidor.');
    }

    Map<String, dynamic> datos;
    try {
      datos = Map<String, dynamic>.from(jsonDecode(utf8.decode(respuesta.bodyBytes)) as Map);
    } catch (_) {
      throw ApiError('Respuesta no válida del servidor.', respuesta.statusCode);
    }
    if (respuesta.statusCode >= 400 || datos['ok'] != true) {
      if (respuesta.statusCode == 401 && sesion.activa) {
        sesion.cerrar();
      }
      throw ApiError((datos['mensaje'] ?? 'No fue posible completar la operación.').toString(), respuesta.st
atusCode);
    }
    return datos;
  }

  List<Map<String, dynamic>> _lista(Object? valor) => (valor as List? ?? []).map((e) => Map<String, dynamic
>.from(e as Map)).toList();

  // ---- público
  Future<Map<String, dynamic>> tema() => _enviar('GET', 'tema');

  Future<Map<String, String>> inicio() async {
    final datos = await _enviar('GET', 'inicio');
    return Map<String, dynamic>.from(datos['datos'] as Map).map((k, v) => MapEntry(k, (v ?? '').toString
()));
  }

  Future<List<Map<String, dynamic>>> productos() async => _lista((await _enviar('GET', 'productos'))['produc
tos']);

  Future<Map<String, dynamic>> contacto() => _enviar('GET', 'contacto');
```

8.10 Interfaz (widgets, tema y navegación)

Archivo	Responsabilidad
main.dart	Crea `MaterialApp` con el tema derivado de la paleta y sincroniza el tema al volver a la app (`WidgetsBindingObserver`).
widgets/comunes.dart	`PaginaPizzeria` (estructura común), `BannerSitio` , `NavPizzeria` (menú ☰), `FooterPizzeria` , `WhatsappFlotante` , `Carrusel` , `BotonGradiente` , `Alerta` .
widgets/login_dialogo.dart	Diálogo de inicio de sesión con contador de 30 s (`Timer.periodic`).
widgets/editor.dart	Piezas de los editores: `PanelEditor` , `FormularioEditor` , `SelectorImagen` (image_picker, máx. 5 MB), `CampoEditor` , `confirmar()` .
widgets/iframe*.dart	Importación condicional: en web el mapa es un `<iframe>` (HtmlElementView); en Android/iOS se usa WebView. `PointerInterceptor` evita que el iframe capture los clics de los diálogos.
pantallas/*.dart	Cada pantalla carga sus datos en `initState` , muestra `EstadoCarga` mientras llegan y permite actualizar deslizando hacia abajo.
navegacion.dart	Enum `Seccion` y `irA()` , que reemplaza la pila de pantallas con una transición de desvanecido.

Tabla 16. Organización de la interfaz.

9. Seguridad de la aplicación

9.1 Controles implementados frente a OWASP Top 10:2025

Riesgo OWASP 2025	Control en la app y la API	Estado
A01 Control de acceso roto	Token obligatorio en todas las rutas de escritura (P07–P09); la app impide abrir editores sin sesión; `rutaValida()` evita recorrer directorios (P26).	Cumple
A02 Configuración de seguridad incorrecta	Errores sin detalles internos. Tráfico HTTP sin TLS permitido en Android/iOS; usuario `postgres` con contraseña débil en Docker; CORS abierto.	Parcial
A03 Fallas en la cadena de suministro	Dependencias con versión fijada en `pubspec.lock`, paquetes oficiales o verificados de pub.dev.	Cumple
A04 Fallas criptográficas	bcrypt para contraseñas; token con `Random.secure()`. Sin HTTPS en red local (H01); tokens guardados sin hash (H07).	Parcial
A05 Inyección	Parámetros nombrados en todas las consultas (P10); Flutter dibuja los textos como texto, no como HTML (P19).	Cumple
A06 Diseño inseguro	Validación en el servidor de imágenes, longitudes y campos (P15–P17, P20, P22, P25). Bloqueo de intentos en memoria (H03).	Parcial
A07 Fallas de autenticación	Mensajes genéricos, bloqueo 5×5 min (P30), vencimiento de 8 h, cierre de sesión que invalida el token (P29).	Cumple
A08 Fallas de integridad	No se usan cookies, así que no aplica CSRF; el formulario público no tiene límite de envíos (H05).	Parcial
A09 Fallas de registro y alertas	Solo el registro de peticiones en consola (`logRequests`); sin auditoría de inicios de sesión.	Pendiente
A10 Manejo de condiciones excepcionales	Middleware `_errores()` convierte excepciones en JSON 400/422/500; la app muestra avisos y botón Reintentar.	Cumple

Tabla 17. Matriz OWASP Top 10:2025.

9.2 Hallazgos y recomendaciones

ID	Hallazgo	Riesgo	Recomendación
H01	La app permite tráfico HTTP sin cifrar (`usesCleartextTraffic` y `NSAllowsArbitraryLoads`): contraseña y token viajan en claro por la Wi-Fi.	Alto	Publicar la API detrás de HTTPS (Caddy o Nginx con Let's Encrypt) y quitar esos permisos.
H02	La API se conecta como `postgres` (superusuario) y la contraseña del contenedor es débil y figura en la documentación.	Alto	Usuario de privilegios mínimos (BD1) y contraseña robusta (BD2).

ID	Hallazgo	Riesgo	Recomendación
H03	El bloqueo de intentos vive en la memoria del servidor por IP: se pierde al reiniciar y bloquea a todos los equipos detrás de la misma IP pública.	Medio	Registrar intentos por usuario e IP en una tabla.
H04	CORS permite cualquier origen (`*`).	Medio	Limitar al dominio de la versión web en producción.
H05	El formulario de contacto no limita envíos ni tiene captcha.	Medio	Límite por IP y campo trampa (honeypot).
H06	Imágenes en base64: hasta ≈ 27 MB por petición con 4 imágenes de 5 MB.	Bajo	Subida multipart y compresión previa en la app.
H07	Los tokens se guardan sin hash en `sesion_api`.	Bajo	Guardar SHA-256 del token (BD3).
H08	No hay auditoría de inicios de sesión ni de cambios.	Bajo	Tabla de auditoría (usuario, acción, fecha, IP).
H09	La URL del mapa solo se valida como https://, no como dominio de Google Maps; en el celular la WebView abriría cualquier página.	Bajo	Aceptar solo `https://www.google.com/maps/embed`.
H10	El usuario de prueba `administrador` / `Pizza2026*` está documentado.	Medio	Eliminarlo o cambiar su clave antes de publicar.
H11	Íconos de redes sociales sin enlace.	Bajo	Agregar las URL reales en `comunes.dart`.
H12	**Corregido:** la API no validaba la longitud de los textos y respondía 500 (prueba P20).	—	Se agregó `_validarLongitudes()`; ahora responde 422.

Tabla 18. Hallazgos de la revisión técnica.

> **IMPORTANTE**
> El hallazgo H01 debe atenderse antes de publicar la app en las tiendas o usarla fuera de la red local: Google Play y la App Store exigen tráfico cifrado salvo excepciones justificadas.

9.3 Código sugerido

```
# Caddyfile (H01): HTTPS automático delante de la API
api.pizzeriajulian.com {
    reverse_proxy localhost:8080
}

// api.dart (H09): aceptar solo mapas de Google
if (!url.startsWith('https://www.google.com/maps/embed')) {
  return _error('Use la URL de "Insertar un mapa" de Google Maps.', estado: 422);
}

// api.dart (H07): guardar el hash del token (import 'package:crypto/crypto.dart')
final hashToken = sha256.convert(utf8.encode(token)).toString();
```

10. Instalación, configuración y despliegue

10.1 Requisitos del equipo

Recurso	Mínimo	Recomendado
Sistema operativo	Windows 10/11 (64 bits), Linux o macOS	Windows 11 o Linux
Flutter SDK	3.38 (exigido por las dependencias)	3.47.9
Docker Desktop	4.x con WSL 2	Última versión
PostgreSQL	16 (Docker o Windows)	16 en Docker
Android	Android SDK 36 y Android Studio para compilar el APK	—
Navegador	Google Chrome actualizado	—
Disco	6 GB (SDK, imágenes de Docker, compilaciones)	15 GB

10.2 Instalación en el entorno local

1. Clonar el repositorio: `git clone https://github.com/delfinalber/FLUTTER.git D:\FLUTTER`.
2. Instalar Flutter en `D:\FLUTTER\flutter\flutter_sdk` y agregar `flutter_sdk\bin` al PATH; verificar con `flutter doctor`.
3. Abrir Docker Desktop y crear la base de datos:

```
docker run -d --name pizza_julian_db --restart unless-stopped -p 5433:5432 -e
POSTGRES_PASSWORD=pizzajulian -e POSTGRES_DB=pizza_julian -v
pizza_julian_datos:/var/lib/postgresql/data -v D:/FLUTTER/flutter/base_de_datos:/sql:ro
postgres:16-alpine
docker exec pizza_julian_db psql -U postgres -d pizza_julian -f /sql/pizza_julian_postgres.sql
```

   (Con PostgreSQL de Windows: `.\crear_base_de_datos.ps1`).
4. Configurar el servidor: copiar `pizza_julian_api\.env.example` como `.env` y completar los datos (10.3).
5. Encender el servidor: `cd pizza_julian_api; dart pub get; dart run bin/server.dart`.
6. Ejecutar la app: `cd pizza_julian_app; flutter pub get; flutter run -d chrome --dart-define=API_URL=http://localhost:8080`.
7. Crear el primer administrador (10.4) e iniciar sesión desde la app.

10.3 Archivo .env

```
# Copie este archivo como .env y complete los datos
# Conexión a PostgreSQL
DB_HOST=localhost
DB_PORT=5432
DB_NAME=pizza_julian
DB_USER=postgres
DB_PASSWORD=

# Puerto del servidor API
```

```
PORT=8080

# Correo del formulario de contacto (contraseña de aplicación de Gmail)
GMAIL_USERNAME=delfin.alber@gmail.com
GMAIL_APP_PASSWORD=
CORREO_DESTINO=delfin.alber@gmail.com
```

10.4 Creación de administradores

```
docker exec -it pizza_julian_db psql -U postgres -d pizza_julian -c "INSERT INTO usuario (usuario, password) VALUES ('admin', crypt('ClaveSegura2026*', gen_salt('bf', 10)));"
```

10.5 Compilación de la app

Destino	Comando	Resultado
Android (pruebas)	`flutter build apk --debug`	build/app/outputs/flutter-apk/app-debug.apk
Android (publicar)	`flutter build apk --release --dart-define=API_URL=https://…` o `flutter build appbundle`	app-release.apk / app-release.aab (Google Play)
iOS	`flutter build ipa --dart-define=API_URL=https://…` (en macOS con Xcode)	Archivo .ipa (App Store)
Web	`flutter build web --release --dart-define=API_URL=https://…`	Carpeta build/web para cualquier servidor estático

10.6 Despliegue en producción (lista de verificación)

Hecho	Actividad
[ ]	Publicar la API detrás de HTTPS y quitar `usesCleartextTraffic` / `NSAllowsArbitraryLoads` (H01).
[ ]	Usuario de base de datos con privilegios mínimos y contraseña robusta (BD1, BD2).
[ ]	Eliminar el usuario de prueba `administrador` (H10).
[ ]	Limitar CORS al dominio de la versión web (H04).
[ ]	Ejecutar el servidor como servicio (NSSM en Windows o systemd en Linux) para que se reinicie solo.
[ ]	Firmar el APK/AAB con una llave propia y guardar la llave en un lugar seguro.
[ ]	Publicar la política de tratamiento de datos (Ley 1581 de 2012) y enlazarla en las tiendas.
[ ]	Programar copias de seguridad de la base de datos y de la carpeta media.

11. Plan y resultados de pruebas

11.1 Estrategia

Se aplicaron pruebas funcionales de caja negra y de seguridad automatizadas con **curl** contra el servidor API en ejecución (Dart 3.13.7, PostgreSQL 16 en Docker), siguiendo la estructura de casos de prueba de ISO/IEC/IEEE 29119-3. Además se recorrieron manualmente todas las pantallas para elaborar el Manual de Usuario y se ejecutaron las pruebas automatizadas de la app.

ID	Caso de prueba	Esperado	Resultado obtenido	Estado
P01	Servicio de salud de la API	HTTP 200	HTTP 200 {"ok":true}	Aprobada
P02	Contenido público de Inicio	HTTP 200	HTTP 200 {"ok":true,"datos": {"banner_inicio":"inicio/banner.png","carru_1":"inicio/1.jpeg","carru_2":"inicio/2.jpeg","ca	Aprobada
P03	Listado público de Productos	HTTP 200	HTTP 200 {"ok":true,"productos": [{"id_producto":1,"img1_carru_productos":"productos/pizza1.png","img2_carru_productos":"	Aprobada
P04	Mapa público de Contacto	HTTP 200	HTTP 200 {"ok":true,"id_contacto":1,"url_contacto":"https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d18953.88384359	Aprobada
P05	Tema (banner, paleta y WhatsApp)	HTTP 200	HTTP 200 {"ok":true,"banner":"inicio/banner.png","paleta": {"negro":"#0f090a","negro-suave":"#1a1113","naranja":"#ff8b1a"	Aprobada
P06	Imagen servida desde /media	HTTP 200	HTTP 200	Aprobada
P07	Editor de Inicio sin token	HTTP 401	HTTP 401 {"ok":false,"mensaje":"La sesión expiró, inicie sesión nuevamente."}	Aprobada
P08	Crear producto sin token	HTTP 401	HTTP 401 {"ok":false,"mensaje":"La sesión expiró, inicie sesión nuevamente."}	Aprobada
P09	Token con formato inválido	HTTP 401	HTTP 401 {"ok":false,"mensaje":"La sesión expiró, inicie sesión nuevamente."}	Aprobada
P10	Inyección SQL en el usuario del login	HTTP 401	HTTP 401 {"ok":false,"mensaje":"Usuario o contraseña incorrectos."}	Aprobada
P11	Login con campos vacíos	HTTP 400	HTTP 400 {"ok":false,"mensaje":"Usuario y contraseña son obligatorios."}	Aprobada
P12	Cuerpo JSON mal formado	HTTP 400	HTTP 400 {"ok":false,"mensaje":"Datos enviados no válidos."}	Aprobada

ID	Caso de prueba	Esperado	Resultado obtenido	Estado
P13	Login con credenciales válidas (token de 64 caracteres)	HTTP 200	HTTP 200 {"ok":true,"mensaje":"Inicio de sesión correcto.","token":"f3b5bce299b2ccb524c579b31fed115010a6cbc2569b235c1d9c	Aprobada
P14	Editor de Inicio con token válido	HTTP 200	HTTP 200 {"ok":true,"registros":[{"id_inicio":1,"banner_inicio":"inicio/banner.png","paleta_banner":"","carru_1":"inicio	Aprobada
P15	Crear producto sin imágenes	HTTP 422	HTTP 422 {"ok":false,"mensaje":"Cada producto necesita las 3 imágenes del carrusel. No se guardaron cambios."}	Aprobada
P16	Archivo de texto enviado como imagen	HTTP 422	HTTP 422 {"ok":false,"mensaje":"Imagen no válida: usa archivos JPG, PNG, GIF o WEBP de máximo 5 MB. No se guardaron camb	Aprobada
P17	Imagen mayor a 5 MB	HTTP 422	HTTP 422 {"ok":false,"mensaje":"Imagen no válida: usa archivos JPG, PNG, GIF o WEBP de máximo 5 MB. No se guardaron camb	Aprobada
P18	Crear producto con 3 imágenes válidas (título con <script>)	HTTP 200	HTTP 200 {"ok":true,"mensaje":"Producto creado correctamente."}	Aprobada
P19	El título con <script> se guarda como texto (Flutter no ejecuta HTML)	HTTP 200	HTTP 200 {"ok":true,"productos":[{"id_producto":1,"img1_carru_productos":"productos/pizza1.png","img2_carru_productos":"	Aprobada
P20	Título de producto de 250 caracteres (límite 200)	HTTP 422	HTTP 422 {"ok":false,"mensaje":"El campo \"card_title_productos\" supera 200 caracteres. No se guardaron cambios."}	Aprobada
P21	Eliminar el producto de prueba	HTTP 200	HTTP 200 {"ok":true,"mensaje":"Producto eliminado correctamente."}	Aprobada
P22	Formulario de contacto con correo inválido	HTTP 422	HTTP 422 {"ok":false,"mensaje":"Por favor, completa el nombre, un correo válido y el mensaje."}	Aprobada
P23	Mensaje válido sin cuenta SMTP configurada	HTTP 200	HTTP 200 {"ok":true,"mensaje":"Tu mensaje se guardó correctamente."}	Aprobada
P24	Mensaje con teléfono escrito con espacios	HTTP 200	HTTP 200 {"ok":true,"mensaje":"Tu mensaje se guardó correctamente."}	Aprobada
P25	URL del mapa que no empieza por https://	HTTP 422	HTTP 422 {"ok":false,"mensaje":"Escriba una URL de Google Maps válida (https://...)."}	Aprobada
P26	Ruta de imagen con ../ (recorrido de directorios)	HTTP 404	HTTP 404	Aprobada
P27	Petición CORS de verificación (OPTIONS)	HTTP 200	HTTP 200	Aprobada

ID	Caso de prueba	Esperado	Resultado obtenido	Estado
P28	Cerrar sesión	HTTP 200	HTTP 200 {"ok":true,"mensaje":"Sesión cerrada."}	Aprobada
P29	Usar el token después de cerrar sesión	HTTP 401	HTTP 401 {"ok":false,"mensaje":"La sesión expiró, inicie sesión nuevamente."}	Aprobada
P30	Sexto intento (con clave correcta) tras 5 fallos: bloqueo 5 min	HTTP 429	HTTP 429 {"ok":false,"mensaje":"Demasiados intentos fallidos. Intente de nuevo en unos minutos."}	Aprobada

Tabla 19. Resultados de las pruebas de la API: 30 de 30 aprobadas.

En la primera ejecución el caso **P20** falló: un título de 250 caracteres provocaba **HTTP 500** porque la API no validaba la longitud antes de guardar en la columna `VARCHAR(200)`. Se corrigió con `_validarLongitudes()` (hallazgo H12) y la segunda ejecución, presentada en la tabla, fue aprobada por completo.

> **NOTA**
> Los casos P23 y P24 se ejecutaron sin cuenta SMTP: el mensaje se guardó y la API respondió "Tu mensaje se guardó correctamente.", que es el comportamiento esperado. A diferencia del sitio web (casos P24 y P25 fallidos), el teléfono vacío o con espacios ya no produce error, porque la columna ahora es VARCHAR(20).

11.2 Pruebas automatizadas y análisis estático

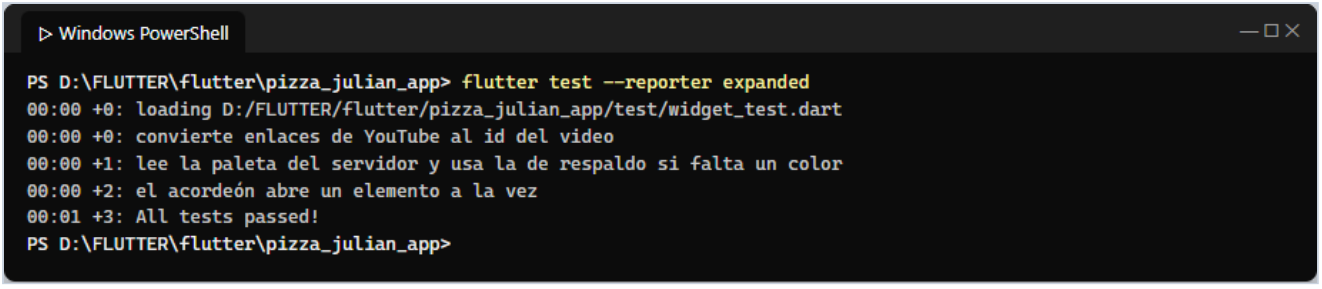


Figura 17. flutter test: 3 pruebas automatizadas de la app aprobadas.

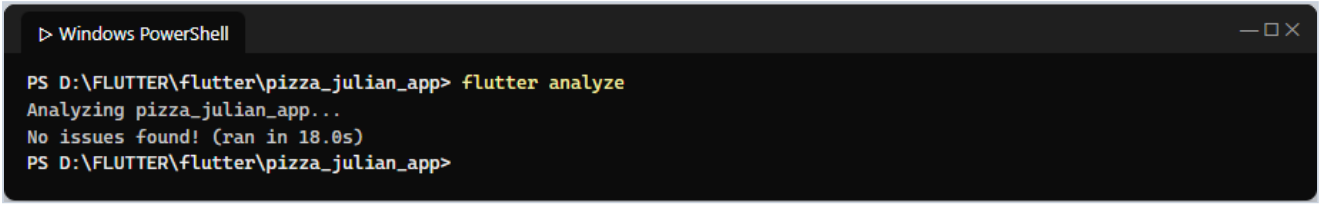


Figura 18. flutter analyze: sin hallazgos.

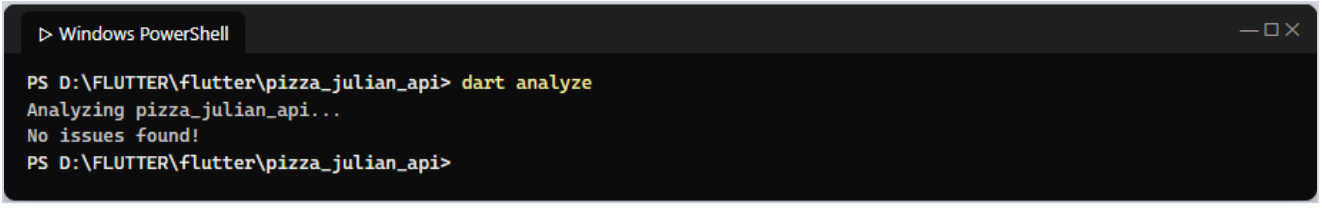


Figura 19. dart analyze del servidor: sin hallazgos.

Prueba automatizada	Qué verifica
convierte enlaces de YouTube al id del video	`idYoutube()` reconoce enlaces embed, youtu.be y watch?v=, y rechaza textos que no son video.
lee la paleta del servidor y usa la de respaldo si falta un color	`Paleta.desdeJson()` toma los colores válidos y reemplaza los inválidos por los predeterminados.
el acordeón abre un elemento a la vez	Prueba de widget: al tocar un título se abre su texto y se cierra el anterior.

11.3 Pruebas de compatibilidad

Escenario	Resultado
Google Chrome 157, tamaño de celular 412 × 915 (modo dispositivo)	Correcto (capturas del Manual de Usuario).
Google Chrome, escritorio 1280 × 760	Correcto; los productos se organizan en 4 columnas.
Compilación de APK de Android (debug)	Correcta (compilada con Flutter 3.24.5 en un contenedor Docker).
Flutter 3.47.9 (versión instalada)	Correcto tras actualizar font_awesome_flutter a la 11.0.0.
PostgreSQL 16 en Docker y script en base vacía	Correcto: 6 tablas, 16 productos, 4 mensajes.

12. Mantenimiento

Tipo (ISO/IEC 14764)	Actividad	Frecuencia
Preventivo	Copia de seguridad de la base de datos y de la carpeta media.	Semanal
Preventivo	Actualizar Flutter (`flutter upgrade`), dependencias (`flutter pub upgrade` , `dart pub upgrade`) e imagen de PostgreSQL.	Trimestral
Correctivo	Atender los hallazgos H01–H11 según su prioridad.	Inmediato / planificado
Adaptativo	Ajustar a nuevas versiones de Android/iOS y a los requisitos de las tiendas (nivel de API objetivo).	Anual
Perfectivo	Carrito de compras, pedidos, pagos, notificaciones y recomendaciones con IA.	Según hoja de ruta

Tabla 20. Plan de mantenimiento.

12.1 Control de versiones

El proyecto se gestiona con Git en **github.com/delfinalber/FLUTTER** (rama `main`). El `.gitignore` excluye los archivos `.env` , el SDK de Flutter (`flutter_sdk/`), las carpetas `build/` y `.dart_tool/` . Se recomienda trabajar en ramas por funcionalidad, con mensajes de commit descriptivos y revisión antes de unir a `main` .

12.2 Hoja de ruta

- **Versión 1.1:** corrección de los hallazgos H01–H11, pantalla de mensajes recibidos en los editores y publicación en Google Play.
- **Versión 2.0:** catálogo con precios, carrito de compras, gestión de pedidos y notificaciones push.
- **Versión 3.0:** pagos en línea, cuentas de clientes y recomendaciones de productos con inteligencia artificial.

ANEXO A

Código fuente del servidor API y de la app Flutter

Anexo A. Código fuente

Se incluye el código del servidor API completo y los archivos principales de la app. Por extensión se omiten las pantallas y widgets restantes, que se encuentran en la carpeta del proyecto y en el repositorio.

A.1 pizza_julian_api/lib/config.dart

```dart
import 'dart:io';

// Configuración del servidor: se lee de .env y las variables de entorno del sistema tienen prioridad
class Config {
  Config._(this._valores);

  final Map<String, String> _valores;

  static Config cargar([String ruta = '.env']) {
    final valores = <String, String>{};
    final archivo = File(ruta);
    if (archivo.existsSync()) {
      for (var linea in archivo.readAsLinesSync()) {
        linea = linea.trim();
        if (linea.isEmpty || linea.startsWith('#') || !linea.contains('=')) {
          continue;
        }
        final indice = linea.indexOf('=');
        final clave = linea.substring(0, indice).trim();
        var valor = linea.substring(indice + 1).trim();
        if (valor.length >= 2 && valor.startsWith('"') && valor.endsWith('"')) {
          valor = valor.substring(1, valor.length - 1);
        }
        valores[clave] = valor;
      }
    }
    valores.addAll(Platform.environment);
    return Config._(valores);
  }

  String texto(String clave, [String porDefecto = '']) {
    final valor = _valores[clave];
    return (valor == null || valor.isEmpty) ? porDefecto : valor;
  }

  int entero(String clave, int porDefecto) => int.tryParse(texto(clave)) ?? porDefecto;

  String get dbHost => texto('DB_HOST', 'localhost');
  int get dbPort => entero('DB_PORT', 5432);
  String get dbName => texto('DB_NAME', 'pizza_julian');
  String get dbUser => texto('DB_USER', 'postgres');
  String get dbPassword => texto('DB_PASSWORD');
  int get puerto => entero('PORT', 8080);
  String get gmailUsuario => texto('GMAIL_USERNAME', 'delfin.alber@gmail.com');
  String get gmailClave => texto('GMAIL_APP_PASSWORD');
  String get correoDestino => texto('CORREO_DESTINO', 'delfin.alber@gmail.com');
  String get carpetaMedia => texto('MEDIA_DIR', 'media');
}
```

A.2 pizza_julian_api/lib/api.dart

```dart
import 'dart:convert';
import 'dart:io';
import 'dart:math';
import 'dart:typed_data';

import 'package:mailer/mailer.dart';
import 'package:mailer/smtp_server.dart';
import 'package:postgres/postgres.dart';
import 'package:shelf/shelf.dart';
import 'package:shelf_router/shelf_router.dart';
```

```
import 'package:shelf_static/shelf_static.dart';

import 'config.dart';
import 'imagenes.dart';
import 'paleta.dart';

// API REST que usa la app Flutter. Replica lo que hacen los .php de pizza-julian:
//   inicio/index.php + index-formulario.php, productos/*.php, contacto/*.php, login.php, logout.php y tema.
php

const _bannerPorDefecto = 'inicio/banner.png';
const _urlMapaPorDefecto =
    "https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d18953.883843598873!2d-75.2785308969908!3d2.941021
7172291766!2m3!1f0!2f0!3f0!3m2!1i1024!2i768!4f13.1!3m3!1m2!1s0x8e3b7464ac59aa5b%3A0x7e513d68543b6eec!2sDomino's%20
Pizza%20Las%20Brisas!5e0!3m2!1ses-419!2sco!4v1784748693148!5m2!1ses-419!2sco";
const _duracionSesion = Duration(hours: 8);

// Valores de respaldo si la tabla `inicio` aún no tiene registros (igual que index.php)
const _datosInicioPorDefecto = <String, String>{
  'banner_inicio': _bannerPorDefecto,
  'carru_1': 'inicio/1.jpeg',
  'carru_2': 'inicio/2.jpeg',
  'carru_3': 'inicio/3.jpeg',
  'url_video_inicio': 'https://www.youtube.com/embed/crdtrzZj3fo',
  'acor1_titulo_inicio': 'Margarita (Margherita)',
  'acor1_texto_inicio': 'La reina de las pizzas. Salsa de tomate, mozzarella fresca, albahaca y aceite de ol
iva.',
  'acor2_titulo_inicio': 'Cuatro Quesos (Quattro Formaggi)',
  'acor2_texto_inicio': 'Una mezcla de quesos fundidos, usualmente mozzarella, gorgonzola (azul), fontina y
parmesano.',
  'acor3_titulo_inicio': 'Napolitana',
  'acor3_texto_inicio': 'Salsa de tomate, anchoas, ajo, orégano y aceite de oliva (tradicionalmente no lleva
queso).',
  'acor4_titulo_inicio': 'Mexicana',
  'acor4_texto_inicio': 'Incluye carne molida o chorizo, frijoles negros, jalapeños, cebolla y un toque de a
guacate o cilantro.',
  'acor5_titulo_inicio': 'Barbacoa (BBQ)',
  'acor5_texto_inicio': 'Sustituye la salsa de tomate por salsa barbacoa, acompañada de pollo, carne picada,
cebolla y bacon.',
  'colapsar1_titulo_inicio': 'Vegetariana',
  'colapsar1_texto_inicio': 'Cubierta de vegetales frescos o salteados, como pimientos, cebolla, champiñone
s, aceitunas y tomates cherry.',
  'colapsar2_titulo_inicio': 'Funghi (Champiñones)',
  'colapsar2_texto_inicio': 'Base de mozzarella con abundantes hongos salteados, a veces con un toque de ace
ite de trufa.',
  'colapsar3_titulo_inicio': 'Cuatro Estaciones (Quattro Stagioni)',
  'colapsar3_texto_inicio':
      'Dividida en cuatro secciones, cada una representando una estación con ingredientes distintos (por eje
mplo: alcachofas, champiñones, jamón y albahaca).',
  'numero_whatsapp': '573132345685',
};

const _camposImagenInicio = ['banner_inicio', 'carru_1', 'carru_2', 'carru_3'];
const _camposTextoInicio = [
  'url_video_inicio',
  'acor1_titulo_inicio',
  'acor1_texto_inicio',
  'acor2_titulo_inicio',
  'acor2_texto_inicio',
  'acor3_titulo_inicio',
  'acor3_texto_inicio',
  'acor4_titulo_inicio',
  'acor4_texto_inicio',
  'acor5_titulo_inicio',
  'acor5_texto_inicio',
```

```
    'colapsar1_titulo_inicio',
    'colapsar1_texto_inicio',
    'colapsar2_titulo_inicio',
    'colapsar2_texto_inicio',
    'colapsar3_titulo_inicio',
    'colapsar3_texto_inicio',
    'numero_whatsapp',
];
// Imágenes que la app usa como respaldo: nunca se borran
const _imagenesProtegidasInicio = {'banner.png', '1.jpeg', '2.jpeg', '3.jpeg'};

const _camposImagenProductos = ['img1_carru_productos', 'img2_carru_productos', 'img3_carru_productos'];
const _camposTextoProductos = ['card_title_productos', 'card_texto_productos', 'btn_primary_productos'];

// Longitud máxima de cada campo de texto (igual a la columna VARCHAR de PostgreSQL)
int _limiteCampo(String campo) {
  if (campo == 'numero_whatsapp') return 20;
  if (campo == 'url_video_inicio' || campo == 'card_title_productos' || campo == 'btn_primary_productos') re
turn 200;
  if (campo.contains('_titulo_')) return 150;
  return 500;
}

// Convierte enlaces de YouTube (watch, youtu.be, shorts) al formato /embed/
String normalizarUrlYoutubeEmbed(String url) {
  url = url.trim();
  final coincidencia = RegExp(r'(?:youtu\.be/|youtube\.com/(?:embed/|watch\?v=|shorts/))([A-Za-z0-9_-]
{6,})', caseSensitive: false).firstMatch(url);
  return coincidencia == null ? url : 'https://www.youtube.com/embed/${coincidencia.group(1)}';
}

class _Intentos {
  int fallidos = 0;
  DateTime bloqueoHasta = DateTime.fromMillisecondsSinceEpoch(0);
}

class PizzaJulianApi {
  PizzaJulianApi(this.config, this.db);

  final Config config;
  final Pool db;
  final _intentosPorIp = <String, _Intentos>{};
  final _aleatorio = Random.secure();

  Handler get handler {
    final router = Router()
      ..get('/api/salud', (Request r) => _json({'ok': true}))
      ..get('/api/tema', _tema)
      ..post('/api/login', _login)
      ..post('/api/logout', _logout)
      ..get('/api/sesion', _sesion)
      ..get('/api/inicio', _inicioPublico)
      ..get('/api/inicio/registros', _inicioRegistros)
      ..post('/api/inicio', (Request r) => _guardarInicio(r, null))
      ..put('/api/inicio/<id>', (Request r, String id) => _guardarInicio(r, int.tryParse(id) ?? 0))
      ..delete('/api/inicio/<id>', _eliminarInicio)
      ..get('/api/productos', _productos)
      ..post('/api/productos', (Request r) => _guardarProducto(r, null))
      ..put('/api/productos/<id>', (Request r, String id) => _guardarProducto(r, int.tryParse(id) ?? 0))
      ..delete('/api/productos/<id>', _eliminarProducto)
      ..get('/api/contacto', _contacto)
      ..post('/api/contacto', (Request r) => _guardarContacto(r, null))
      ..put('/api/contacto/<id>', (Request r, String id) => _guardarContacto(r, int.tryParse(id) ?? 0))
      ..post('/api/contacto/mensaje', _mensajeContacto)
      ..mount('/media/', createStaticHandler(config.carpetaMedia));
```

```
128
129     return const Pipeline().addMiddleware(logRequests()).addMiddleware(_cors()).addMiddleware(_errores()).addHandler(router.call);
130   }
131
132   // ------------------------------------------------------------------ utilidades
133
134   Response _json(Object datos, {int estado = 200}) => Response(
135         estado,
136         body: jsonEncode(datos),
137         headers: {'content-type': 'application/json; charset=utf-8', 'cache-control': 'no-store'},
138       );
139
140   Response _error(String mensaje, {int estado = 400}) => _json({'ok': false, 'mensaje': mensaje}, estado: estado);
141
142   Middleware _cors() => (inner) => (request) async {
143         const cabeceras = {
144           'access-control-allow-origin': '*',
145           'access-control-allow-methods': 'GET, POST, PUT, DELETE, OPTIONS',
146           'access-control-allow-headers': 'content-type, authorization',
147         };
148         if (request.method == 'OPTIONS') {
149           return Response.ok('', headers: cabeceras);
150         }
151         final respuesta = await inner(request);
152         return respuesta.change(headers: cabeceras);
153       };
154
155   Middleware _errores() => (inner) => (request) async {
156         try {
157           return await inner(request);
158         } on ErrorImagen catch (e) {
159           return _error(e.mensaje, estado: 422);
160         } on FormatException {
161           return _error('Datos enviados no válidos.');
162         } catch (e, pila) {
163           stderr.writeln('Error en ${request.method} ${request.url}: $e\n$pila');
164           return _error('No fue posible completar la operación.', estado: 500);
165         }
166       };
167
168   Future<Map<String, dynamic>> _cuerpo(Request request) async {
169     final texto = await request.readAsString();
170     final datos = texto.isEmpty ? <String, dynamic>{} : jsonDecode(texto);
171     if (datos is! Map<String, dynamic>) {
172       throw const FormatException();
173     }
174     return datos;
175   }
176
177   String _texto(Map<String, dynamic> datos, String clave) => (datos[clave] ?? '').toString().trim();
178
179   Map<String, dynamic> _fila(ResultRow fila) => fila.toColumnMap().map((clave, valor) => MapEntry(clave, valor is DateTime ? valor.toIso8601String() : valor));
180
181   Future<List<Map<String, dynamic>>> _consulta(String sql, [Map<String, dynamic>? parametros]) async {
182     final resultado = await db.execute(Sql.named(sql), parameters: parametros);
183     return resultado.map(_fila).toList();
184   }
185
186   // Devuelve el id del usuario si el token "Authorization: Bearer ..." es válido
187   Future<int?> _usuarioAutenticado(Request request) async {
188     final cabecera = request.headers['authorization'] ?? '';
189     if (!cabecera.startsWith('Bearer ')) {
```

```
190       return null;
191     }
192     final token = cabecera.substring(7).trim();
193     if (!RegExp(r'^[0-9a-f]{64}$').hasMatch(token)) {
194       return null;
195     }
196     final filas = await _consulta('SELECT id_usuario FROM sesion_api WHERE token = @token AND expira > now
()', {'token': token});
197     return filas.isEmpty ? null : filas.first['id_usuario'] as int;
198   }
199
200   Response _noAutorizado() => _error('La sesión expiró, inicie sesión nuevamente.', estado: 401);
201
202   // ---------------------------------------------------------------- tema (banner y paleta)
203
204   Future<Map<String, dynamic>> _datosTema() async {
205     var banner = _bannerPorDefecto;
206     var whatsapp = _datosInicioPorDefecto['numero_whatsapp']!;
207     Map<String, String> paleta = paletaPorDefecto;
208     final filas = await _consulta('SELECT banner_inicio, paleta_banner, numero_whatsapp FROM inicio ORDER BY
id_inicio ASC LIMIT 1');
209     if (filas.isNotEmpty) {
210       final numero = (filas.first['numero_whatsapp'] as String).replaceAll(RegExp(r'\D'), '');
211       if (numero.isNotEmpty) {
212         whatsapp = numero;
213       }
214       final valor = (filas.first['banner_inicio'] as String).trim();
215       if (rutaValida(valor, 'inicio')) {
216         banner = valor;
217       }
218       try {
219         paleta = normalizarPaleta(jsonDecode(filas.first['paleta_banner'] as String)) ?? paletaPorDefecto;
220       } on FormatException {
221         paleta = paletaPorDefecto;
222       }
223     }
224     final version = (banner + jsonEncode(paleta) + whatsapp).hashCode.toRadixString(16);
225     return {'banner': banner, 'paleta': paleta, 'version': version, 'whatsapp': whatsapp};
226   }
227
228   Future<Response> _tema(Request request) async => _json({'ok': true, ...await _datosTema()});
229
230   // ---------------------------------------------------------------- sesión
231
232   Future<Response> _login(Request request) async {
233     final info = request.context['shelf.io.connection_info'] as HttpConnectionInfo?;
234     final ip = info?.remoteAddress.address ?? 'desconocida';
235     final intentos = _intentosPorIp.putIfAbsent(ip, _Intentos.new);
236
237     // Bloqueo temporal tras varios intentos fallidos (mitiga fuerza bruta)
238     if (DateTime.now().isBefore(intentos.bloqueoHasta)) {
239       return _error('Demasiados intentos fallidos. Intente de nuevo en unos minutos.', estado: 429);
240     }
241
242     final datos = await _cuerpo(request);
243     final usuario = _texto(datos, 'usuario');
244     final password = (datos['password'] ?? '').toString();
245     if (usuario.isEmpty || password.isEmpty || usuario.length > 200 || password.length > 200) {
246       return _error('Usuario y contraseña son obligatorios.');
247     }
248
249     // crypt() de pgcrypto compara contra el hash bcrypt guardado, nunca en texto plano
250     final filas = await _consulta(
251       'SELECT id_usuario, usuario FROM usuario WHERE usuario = @usuario AND password = crypt(@password, pass
word) LIMIT 1',
```

```
252       {'usuario': usuario, 'password': password},
253     );
254     if (filas.isEmpty) {
255       intentos.fallidos++;
256       if (intentos.fallidos >= 5) {
257         intentos.bloqueoHasta = DateTime.now().add(const Duration(minutes: 5));
258         intentos.fallidos = 0;
259       }
260       return _error('Usuario o contraseña incorrectos.', estado: 401);
261     }
262     _intentosPorIp.remove(ip);
263
264     final token = List.generate(32, (_) => _aleatorio.nextInt(256).toRadixString(16).padLeft(2, '0')).join
();
265     await db.execute('DELETE FROM sesion_api WHERE expira <= now()');
266     await db.execute(
267       Sql.named('INSERT INTO sesion_api (token, id_usuario, expira) VALUES (@token, @id, @expira)'),
268       parameters: {'token': token, 'id': filas.first['id_usuario'], 'expira': DateTime.now().toUtc().add(_du
racionSesion)},
269     );
270     return _json({'ok': true, 'mensaje': 'Inicio de sesión correcto.', 'token': token, 'usuario': filas.firs
t['usuario']});
271   }
272
273   Future<Response> _logout(Request request) async {
274     final cabecera = request.headers['authorization'] ?? '';
275     if (cabecera.startsWith('Bearer ')) {
276       await db.execute(Sql.named('DELETE FROM sesion_api WHERE token = @token'), parameters: {'token': cabec
era.substring(7).trim()});
277     }
278     return _json({'ok': true, 'mensaje': 'Sesión cerrada.'});
279   }
280
281   Future<Response> _sesion(Request request) async => await _usuarioAutenticado(request) == null ? _noAutoriz
ado() : _json({'ok': true});
282
283   // ---------------------------------------------------------------- inicio
284
285   Future<Response> _inicioPublico(Request request) async {
286     final datos = Map<String, String>.of(_datosInicioPorDefecto);
287     final filas = await _consulta('SELECT * FROM inicio ORDER BY id_inicio ASC LIMIT 1');
288     if (filas.isNotEmpty) {
289       for (final campo in datos.keys.toList()) {
290         final valor = (filas.first[campo] ?? '').toString().trim();
291         if (valor.isNotEmpty) {
292           datos[campo] = valor;
293         }
294       }
295     }
296     datos['url_video_inicio'] = normalizarUrlYoutubeEmbed(datos['url_video_inicio']!);
297     datos['numero_whatsapp'] = datos['numero_whatsapp']!.replaceAll(RegExp(r'\D'), '');
298     return _json({'ok': true, 'datos': datos});
299   }
300
301   Future<Response> _inicioRegistros(Request request) async {
302     if (await _usuarioAutenticado(request) == null) return _noAutorizado();
303     return _json({'ok': true, 'registros': await _consulta('SELECT * FROM inicio ORDER BY id_inicio ASC')});
304   }
305
306   // Rutas de imagen que siguen usándose en alguna fila de la tabla (para no borrarlas)
307   Future<Set<String>> _imagenesEnUso(String tabla, List<String> columnas) async {
308     final filas = await _consulta('SELECT ${columnas.join(', ')} FROM $tabla');
309     return {
310       for (final fila in filas)
311         for (final c in columnas) (fila[c] ?? '').toString()
```

```
312       };
313     }
314
315     Future<void> _borrarNoUsadas(String tabla, List<String> columnas, Iterable<String> anteriores, String carp
eta, Set<String> protegidas) async {
316       final enUso = await _imagenesEnUso(tabla, columnas);
317       await borrarArchivos(anteriores.where((r) => !enUso.contains(r)), config.carpetaMedia, carpeta, protegid
as: protegidas);
318     }
319
320     // Decodifica todas las imágenes nuevas antes de guardar nada (si una falla, no se guarda ninguna)
321     Map<String, Uint8List> _imagenesNuevas(Map<String, dynamic> datos, List<String> campos) {
322       final imagenes = datos['imagenes'];
323       final nuevas = <String, Uint8List>{};
324       if (imagenes is Map) {
325         for (final campo in campos) {
326           final valor = imagenes[campo];
327           if (valor is String && valor.isNotEmpty) {
328             nuevas[campo] = decodificarImagen(valor);
329           }
330         }
331       }
332       return nuevas;
333     }
334
335     // Devuelve un error 422 si algún campo de texto supera la longitud de su columna
336     Response? _validarLongitudes(Map<String, dynamic> datos, List<String> campos) {
337       for (final campo in campos) {
338         if (_texto(datos, campo).length > _limiteCampo(campo)) {
339           return _error('El campo "$campo" supera ${_limiteCampo(campo)} caracteres. No se guardaron cambio
s.', estado: 422);
340         }
341       }
342       return null;
343     }
344
345     Future<Response> _guardarInicio(Request request, int? id) async {
346       if (await _usuarioAutenticado(request) == null) return _noAutorizado();
347       final datos = await _cuerpo(request);
348       final errorLongitud = _validarLongitudes(datos, _camposTextoInicio);
349       if (errorLongitud != null) return errorLongitud;
350
351       Map<String, dynamic> actual = {};
352       if (id != null) {
353         final filas = await _consulta('SELECT * FROM inicio WHERE id_inicio = @id', {'id': id});
354         if (filas.isEmpty) return _error('El registro no existe.', estado: 404);
355         actual = filas.first;
356       }
357
358       final nuevas = _imagenesNuevas(datos, _camposImagenInicio);
359       final valores = <String, dynamic>{};
360       final guardadas = <String>[];
361       try {
362         for (final campo in _camposImagenInicio) {
363           valores[campo] = nuevas.containsKey(campo)
364               ? (guardadas..add(await guardarImagen(nuevas[campo]!, config.carpetaMedia, 'inicio', 'inicio
_'))).last
365               : (actual[campo] ?? '').toString();
366         }
367         // Con un banner nuevo se guarda la paleta calculada de esa imagen; si no, se conserva la actual
368         if (nuevas.containsKey('banner_inicio')) {
369           final paleta = extraerPaletaBanner(nuevas['banner_inicio']!);
370           valores['paleta_banner'] = paleta == null ? '' : jsonEncode(paleta);
371         } else {
372           valores['paleta_banner'] = (actual['paleta_banner'] ?? '').toString();
```

```
373       }
374       for (final campo in _camposTextoInicio) {
375         var valor = _texto(datos, campo);
376         if (campo == 'url_video_inicio') valor = normalizarUrlYoutubeEmbed(valor);
377         if (campo == 'numero_whatsapp') valor = valor.replaceAll(RegExp(r'\D'), '');
378         valores[campo] = valor;
379       }
380
381       final columnas = valores.keys.toList();
382       if (id == null) {
383         await db.execute(
384           Sql.named('INSERT INTO inicio (${columnas.join(', ')}) VALUES (${columnas.map((c) => '@$c').join
('', ')})'),
385           parameters: valores,
386         );
387       } else {
388         await db.execute(
389           Sql.named('UPDATE inicio SET ${columnas.map((c) => '$c = @$c').join(', ')}, actualizado = now() WH
ERE id_inicio = @id_inicio'),
390           parameters: {...valores, 'id_inicio': id},
391         );
392       }
393     } catch (_) {
394       // No se guarda nada: se descartan los archivos que sí alcanzaron a guardarse
395       await borrarArchivos(guardadas, config.carpetaMedia, 'inicio');
396       rethrow;
397     }
398
399     if (id != null) {
400       // La imagen antigua que fue reemplazada se borra del almacenamiento
401       await _borrarNoUsadas('inicio', _camposImagenInicio, _camposImagenInicio.map((c) => (actual[c] ?? '').
toString()), 'inicio', _imagenesProtegidasInicio);
402     }
403     return _json({'ok': true, 'mensaje': id == null ? 'Contenido creado correctamente.' : 'Contenido actuali
zado correctamente.'});
404   }
405
406   Future<Response> _eliminarInicio(Request request, String idTexto) async {
407     if (await _usuarioAutenticado(request) == null) return _noAutorizado();
408     final id = int.tryParse(idTexto) ?? 0;
409     final filas = await _consulta('SELECT * FROM inicio WHERE id_inicio = @id', {'id': id});
410     if (filas.isEmpty) return _error('El registro no existe.', estado: 404);
411     await db.execute(Sql.named('DELETE FROM inicio WHERE id_inicio = @id'), parameters: {'id': id});
412     await _borrarNoUsadas(
413         'inicio', _camposImagenInicio, _camposImagenInicio.map((c) => (filas.first[c] ?? '').toString()), 'i
nicio', _imagenesProtegidasInicio);
414     return _json({'ok': true, 'mensaje': 'Contenido eliminado correctamente.'});
415   }
416
417   // ---------------------------------------------------------------------- productos
418
419   Future<Response> _productos(Request request) async => _json({'ok': true, 'productos': await _consulta('SEL
ECT * FROM productos ORDER BY id_producto ASC')});
420
421   Future<Response> _guardarProducto(Request request, int? id) async {
422     if (await _usuarioAutenticado(request) == null) return _noAutorizado();
423     final datos = await _cuerpo(request);
424     final errorLongitud = _validarLongitudes(datos, _camposTextoProductos);
425     if (errorLongitud != null) return errorLongitud;
426
427     Map<String, dynamic> actual = {};
428     if (id != null) {
429       final filas = await _consulta('SELECT * FROM productos WHERE id_producto = @id', {'id': id});
430       if (filas.isEmpty) return _error('El producto no existe.', estado: 404);
431       actual = filas.first;
```

```
    }

    final nuevas = _imagenesNuevas(datos, _camposImagenProductos);
    for (final campo in _camposImagenProductos) {
      if (!nuevas.containsKey(campo) && (actual[campo] ?? '').toString().isEmpty) {
        return _error('Cada producto necesita las 3 imágenes del carrusel. No se guardaron cambios.', estado: 422);
      }
    }
    if (_texto(datos, 'card_title_productos').isEmpty) {
      return _error('El título del producto es obligatorio.', estado: 422);
    }

    final valores = <String, dynamic>{};
    final guardadas = <String>[];
    try {
      for (final campo in _camposImagenProductos) {
        valores[campo] = nuevas.containsKey(campo)
            ? (guardadas..add(await guardarImagen(nuevas[campo]!, config.carpetaMedia, 'productos', 'producto_'))).last
            : (actual[campo] ?? '').toString();
      }
      for (final campo in _camposTextoProductos) {
        valores[campo] = _texto(datos, campo);
      }
      final columnas = valores.keys.toList();
      if (id == null) {
        await db.execute(
          Sql.named('INSERT INTO productos (${columnas.join(', ')}) VALUES (${columnas.map((c) => '@$c').join(', ')})'),
          parameters: valores,
        );
      } else {
        await db.execute(
          Sql.named('UPDATE productos SET ${columnas.map((c) => '$c = @$c').join(', ')} WHERE id_producto = @id_producto'),
          parameters: {...valores, 'id_producto': id},
        );
      }
    } catch (_) {
      await borrarArchivos(guardadas, config.carpetaMedia, 'productos');
      rethrow;
    }

    if (id != null) {
      await _borrarNoUsadas('productos', _camposImagenProductos, _camposImagenProductos.map((c) => (actual[c] ?? '').toString()), 'productos', const {});
    }
    return _json({'ok': true, 'mensaje': id == null ? 'Producto creado correctamente.' : 'Producto actualizado correctamente.'});
  }

  Future<Response> _eliminarProducto(Request request, String idTexto) async {
    if (await _usuarioAutenticado(request) == null) return _noAutorizado();
    final id = int.tryParse(idTexto) ?? 0;
    final filas = await _consulta('SELECT * FROM productos WHERE id_producto = @id', {'id': id});
    if (filas.isEmpty) return _error('El producto no existe.', estado: 404);
    await db.execute(Sql.named('DELETE FROM productos WHERE id_producto = @id'), parameters: {'id': id});
    await _borrarNoUsadas('productos', _camposImagenProductos, _camposImagenProductos.map((c) => (filas.first[c] ?? '').toString()), 'productos', const {});
    return _json({'ok': true, 'mensaje': 'Producto eliminado correctamente.'});
  }

  // ---------------------------------------------------------------- contacto

```

```
  Future<Response> _contacto(Request request) async {
    final filas = await _consulta('SELECT * FROM contacto ORDER BY id_contacto ASC LIMIT 1');
    final fila = filas.isEmpty ? null : filas.first;
    final url = (fila?['url_contacto'] ?? '').toString().trim();
    return _json({
      'ok': true,
      'id_contacto': fila?['id_contacto'],
      'url_contacto': url.isEmpty ? _urlMapaPorDefecto : url,
    });
  }

  Future<Response> _guardarContacto(Request request, int? id) async {
    if (await _usuarioAutenticado(request) == null) return _noAutorizado();
    final url = _texto(await _cuerpo(request), 'url_contacto');
    if (url.isEmpty || !url.startsWith('https://')) {
      return _error('Escriba una URL de Google Maps válida (https://...).', estado: 422);
    }
    if (id == null) {
      await db.execute(Sql.named('INSERT INTO contacto (url_contacto) VALUES (@url)'), parameters: {'url': u
rl});
      return _json({'ok': true, 'mensaje': 'URL del mapa creada correctamente.'});
    }
    final resultado = await db.execute(Sql.named('UPDATE contacto SET url_contacto = @url WHERE id_contacto
= @id'), parameters: {'url': url, 'id': id});
    if (resultado.affectedRows == 0) return _error('El registro no existe.', estado: 404);
    return _json({'ok': true, 'mensaje': 'URL del mapa actualizada correctamente.'});
  }

  Future<Response> _mensajeContacto(Request request) async {
    final datos = await _cuerpo(request);
    final correo = _texto(datos, 'correo');
    final nombre = _texto(datos, 'nombre');
    final telefono = _texto(datos, 'telefono');
    final mensaje = _texto(datos, 'mensaje');

    final correoValido = RegExp(r'^[^@\s]+@[^@\s]+\.[^@\s]+$').hasMatch(correo);
    if (!correoValido || nombre.isEmpty || mensaje.isEmpty) {
      return _error('Por favor, completa el nombre, un correo válido y el mensaje.', estado: 422);
    }
    if (correo.length > 150 || nombre.length > 150 || telefono.length > 20 || mensaje.length > 500) {
      return _error('Algún campo supera la longitud permitida.', estado: 422);
    }

    await db.execute(
      Sql.named('INSERT INTO formulario_contacto (correo_formulario, nombre_formulario, telefono_formulario,
mensaje_formulario) '
          'VALUES (@correo, @nombre, @telefono, @mensaje)'),
      parameters: {'correo': correo, 'nombre': nombre, 'telefono': telefono, 'mensaje': mensaje},
    );

    if (config.gmailClave.isEmpty) {
      stderr.writeln('No está configurada GMAIL_APP_PASSWORD: el mensaje se guardó pero no se envió por corr
eo.');
      return _json({'ok': true, 'mensaje': 'Tu mensaje se guardó correctamente.'});
    }

    try {
      final correoSalida = Message()
        ..from = Address(config.gmailUsuario, 'Formulario PIZZERIA-JULIAN')
        ..recipients.add(config.correoDestino)
        ..headers = {'Reply-To': '$nombre <$correo>'}
        ..subject = 'Nuevo mensaje del formulario de contacto'
        ..text = 'Nombre: $nombre\nCorreo: $correo\nTeléfono: $telefono\n\nMensaje:\n$mensaje';
      await send(correoSalida, gmail(config.gmailUsuario, config.gmailClave));
    } catch (e) {
```

```
      stderr.writeln('Error SMTP del formulario de contacto: $e');
      return _error('La información se guardó, pero no se pudo enviar el correo.', estado: 502);
    }
    return _json({'ok': true, 'mensaje': 'Tu mensaje se envió correctamente.'});
  }
}
```

A.3 pizza_julian_api/lib/imagenes.dart

```
import 'dart:convert';
import 'dart:io';
import 'dart:math';
import 'dart:typed_data';

import 'package:image/image.dart' as img;

// Manejo de imágenes subidas desde los editores (equivalente a inicio/imagenes-editor.php)

const tamanoMaximoImagen = 5 * 1024 * 1024; // 5 MB por imagen

class ErrorImagen implements Exception {
  ErrorImagen(this.mensaje);
  final String mensaje;
}

final _aleatorio = Random.secure();

String _extension(img.ImageFormat formato) {
  switch (formato) {
    case img.ImageFormat.jpg:
      return 'jpg';
    case img.ImageFormat.png:
      return 'png';
    case img.ImageFormat.gif:
      return 'gif';
    case img.ImageFormat.webp:
      return 'webp';
    default:
      throw ErrorImagen('Imagen no válida: usa archivos JPG, PNG, GIF o WEBP de máximo 5 MB. No se guardaron cambios.');
  }
}

// Decodifica la imagen en base64 y valida tipo y tamaño (no se confía en la extensión que envía la app)
Uint8List decodificarImagen(String base64Imagen) {
  final Uint8List bytes;
  try {
    bytes = base64Decode(base64Imagen.contains(',') ? base64Imagen.split(',').last : base64Imagen);
  } on FormatException {
    throw ErrorImagen('Imagen no válida: usa archivos JPG, PNG, GIF o WEBP de máximo 5 MB. No se guardaron cambios.');
  }
  if (bytes.isEmpty || bytes.length > tamanoMaximoImagen) {
    throw ErrorImagen('Imagen no válida: usa archivos JPG, PNG, GIF o WEBP de máximo 5 MB. No se guardaron cambios.');
  }
  _extension(img.findFormatForData(bytes));
  return bytes;
}

// Guarda la imagen en media/<carpeta>/ con nombre aleatorio y devuelve la ruta relativa ("productos/producto_xxx.png")
Future<String> guardarImagen(Uint8List bytes, String carpetaMedia, String carpeta, String prefijo) async {
  final extension = _extension(img.findFormatForData(bytes));
```

```
  final sufijo = List.generate(8, (_) => _aleatorio.nextInt(256).toRadixString(16).padLeft(2, '0')).join();
  final nombre = '$prefijo${DateTime.now().microsecondsSinceEpoch.toRadixString(16)}$sufijo.$extension';
  final directorio = Directory('$carpetaMedia/$carpeta');
  await directorio.create(recursive: true);
  await File('${directorio.path}/$nombre').writeAsBytes(bytes, flush: true);
  return '$carpeta/$nombre';
}

// Ruta relativa segura dentro de una carpeta de media (evita "../")
bool rutaValida(String ruta, String carpeta) => RegExp('^$carpeta/[A-Za-z0-9._-]+\$').hasMatch(ruta);

Future<void> borrarArchivos(Iterable<String> rutas, String carpetaMedia, String carpeta, {Set<String> protegidas = const {}}) async {
  for (final ruta in rutas) {
    if (!rutaValida(ruta, carpeta) || protegidas.contains(ruta.split('/').last)) {
      continue;
    }
    final archivo = File('$carpetaMedia/$ruta');
    if (await archivo.exists()) {
      await archivo.delete();
    }
  }
}
```

A.4 pizza_julian_api/lib/paleta.dart

```
import 'dart:math' as math;
import 'dart:typed_data';

import 'package:image/image.dart' as img;

// Paleta del sitio: misma lógica que inicio/tema.js (extraerPaletaBanner) de la página web,
// pero calculada en el servidor al guardar un banner nuevo.

const paletaPorDefecto = <String, String>{
  'negro': '#0f090a',
  'negro-suave': '#1a1113',
  'naranja': '#ff8b1a',
  'amarillo': '#ffc21a',
  'rojo': '#d90a0a',
  'rojo-profundo': '#8f0909',
  'rojo-vivo': '#ff2e2e',
  'blanco': '#f7f5f3',
  'gris': '#e7e2de',
};

// Devuelve la paleta solo si trae todos los colores en formato #rrggbb; si no, null
Map<String, String>? normalizarPaleta(Object? datos) {
  if (datos is! Map) {
    return null;
  }
  final paleta = <String, String>{};
  final formato = RegExp(r'^#[0-9a-f]{6}$');
  for (final clave in paletaPorDefecto.keys) {
    final color = (datos[clave] ?? '').toString().trim().toLowerCase();
    if (!formato.hasMatch(color)) {
      return null;
    }
    paleta[clave] = color;
  }
  return paleta;
}

String _hslAHex(double h, double s, double l) {
```

```
  h = ((h % 360) + 360) % 360;
  final c = (1 - (2 * l - 1).abs()) * s;
  final x = c * (1 - (((h / 60) % 2) - 1).abs());
  final m = l - c / 2;
  final List<double> rgb;
  if (h < 60) {
    rgb = [c, x, 0];
  } else if (h < 120) {
    rgb = [x, c, 0];
  } else if (h < 180) {
    rgb = [0, c, x];
  } else if (h < 240) {
    rgb = [0, x, c];
  } else if (h < 300) {
    rgb = [x, 0, c];
  } else {
    rgb = [c, 0, x];
  }
  return '#${rgb.map((v) => ((v + m) * 255).round().clamp(0, 255).toRadixString(16).padLeft(2, '0')).join
()}';
}

double _luminancia(String hex) {
  final canales = [1, 3, 5].map((i) {
    final v = int.parse(hex.substring(i, i + 2), radix: 16) / 255;
    return v <= 0.03928 ? v / 12.92 : math.pow((v + 0.055) / 1.055, 2.4).toDouble();
  }).toList();
  return 0.2126 * canales[0] + 0.7152 * canales[1] + 0.0722 * canales[2];
}

List<double> _rgbAHsl(double r, double g, double b) {
  final maximo = math.max(r, math.max(g, b));
  final minimo = math.min(r, math.min(g, b));
  var h = 0.0, s = 0.0;
  final l = (maximo + minimo) / 2;
  if (maximo != minimo) {
    final d = maximo - minimo;
    s = l > 0.5 ? d / (2 - maximo - minimo) : d / (maximo + minimo);
    if (maximo == r) {
      h = (g - b) / d + (g < b ? 6 : 0);
    } else if (maximo == g) {
      h = (b - r) / d + 2;
    } else {
      h = (r - g) / d + 4;
    }
    h *= 60;
  }
  return [h, s, l];
}

double _distanciaTono(double a, double b) {
  final d = (a - b).abs() % 360;
  return d > 180 ? 360 - d : d;
}

// Ajusta la luminosidad hasta cumplir el contraste que necesita cada rol del diseño
String _colorConLuminancia(double tono, double saturacion, double luz, double paso, bool Function(double) cu
mple) {
  var hex = _hslAHex(tono, saturacion, luz);
  while (!cumple(_luminancia(hex)) && luz > 0.05 && luz < 0.9) {
    luz += paso;
    hex = _hslAHex(tono, saturacion, luz);
  }
  return hex;
}
```

```
102
103 // Devuelve la paleta con los tonos del banner, o null si la imagen no tiene colores definidos
104 Map<String, String>? extraerPaletaBanner(Uint8List bytes) {
105   final original = img.decodeImage(bytes);
106   if (original == null || original.width == 0) {
107     return null;
108   }
109   const ancho = 160;
110   final alto = math.max(1, (original.height * ancho / original.width).round());
111   final imagen = img.copyResize(original, width: ancho, height: alto);
112
113   // histograma de tonos (36 grupos de 10°) de los píxeles con color vivo
114   final pesos = List<double>.filled(36, 0);
115   for (final pixel in imagen) {
116     final alfa = pixel.aNormalized.toDouble();
117     final hsl = _rgbAHsl(pixel.rNormalized * alfa, pixel.gNormalized * alfa, pixel.bNormalized * alfa);
118     if (hsl[1] > 0.45 && hsl[2] > 0.25 && hsl[2] < 0.75) {
119       pesos[(hsl[0] / 10).floor() % 36] += hsl[1];
120     }
121   }
122   final maximo = pesos.reduce(math.max);
123   if (maximo < ancho * alto * 0.01) {
124     return null; // imagen casi en blanco y negro: se usa la paleta por defecto
125   }
126
127   // tono principal: promedio circular del grupo dominante y sus vecinos
128   final principal = pesos.indexOf(maximo);
129   var sumaX = 0.0, sumaY = 0.0;
130   for (final desplazamiento in [-1, 0, 1]) {
131     final grupo = (principal + desplazamiento + 36) % 36;
132     final angulo = (grupo * 10 + 5) * math.pi / 180;
133     sumaX += math.cos(angulo) * pesos[grupo];
134     sumaY += math.sin(angulo) * pesos[grupo];
135   }
136   final tonoNaranja = (math.atan2(sumaY, sumaX) * 180 / math.pi + 360) % 360;
137
138   // entre los tonos importantes, el más oscuro hace de "rojo" y el más claro de "amarillo"
139   double luzDeTono(double tono) => _luminancia(_hslAHex(tono, 1, 0.5));
140   var tonoRojo = tonoNaranja, tonoAmarillo = tonoNaranja;
141   for (var grupo = 0; grupo < 36; grupo++) {
142     if (pesos[grupo] < maximo * 0.15) {
143       continue;
144     }
145     final tono = grupo * 10 + 5.0;
146     if (luzDeTono(tono) < luzDeTono(tonoRojo)) tonoRojo = tono;
147     if (luzDeTono(tono) > luzDeTono(tonoAmarillo)) tonoAmarillo = tono;
148   }
149   if (_distanciaTono(tonoRojo, tonoNaranja) < 15) {
150     tonoRojo = luzDeTono(tonoNaranja - 25) < luzDeTono(tonoNaranja + 25) ? tonoNaranja - 25 : tonoNaranja +
25;
151   }
152   if (_distanciaTono(tonoAmarillo, tonoNaranja) < 10) {
153     tonoAmarillo = luzDeTono(tonoNaranja - 15) > luzDeTono(tonoNaranja + 15) ? tonoNaranja - 15 : tonoNaranj
a + 15;
154   }
155
156   // mismas saturaciones y luces que el diseño original: solo cambian los tonos
157   final naranja = _colorConLuminancia(tonoNaranja, 1, 0.55, 0.02, (lum) => lum >= 0.38);
158   return {
159     'negro': _hslAHex(tonoRojo, 0.25, 0.047),
160     'negro-suave': _hslAHex(tonoRojo, 0.21, 0.084),
161     'naranja': naranja,
162     'amarillo': _colorConLuminancia(tonoAmarillo, 1, 0.55, 0.02, (lum) => lum >= math.max(0.55, _luminancia
(naranja) + 0.1)),
163     'rojo': _colorConLuminancia(tonoRojo, 0.91, 0.445, -0.02, (lum) => lum <= 0.16),
```

```
    'rojo-profundo': _colorConLuminancia(tonoRojo, 0.88, 0.3, -0.02, (lum) => lum <= 0.07),
    'rojo-vivo': _colorConLuminancia(tonoRojo, 1, 0.59, -0.02, (lum) => lum <= 0.3),
    'blanco': _hslAHex(tonoNaranja, 0.2, 0.96),
    'gris': _hslAHex(tonoNaranja, 0.17, 0.89),
  };
}
```

A.5 pizza_julian_app/lib/main.dart

```
import 'package:flutter/material.dart';

import 'navegacion.dart';
import 'tema.dart';

void main() {
  runApp(const PizzaJulianApp());
}

class PizzaJulianApp extends StatefulWidget {
  const PizzaJulianApp({super.key});

  @override
  State<PizzaJulianApp> createState() => _PizzaJulianAppState();
}

class _PizzaJulianAppState extends State<PizzaJulianApp> with WidgetsBindingObserver {
  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addObserver(this);
    temaSitio.sincronizar();
  }

  @override
  void dispose() {
    WidgetsBinding.instance.removeObserver(this);
    super.dispose();
  }

  // Al volver a la app se revisa si otro editor cambió el banner y la paleta
  @override
  void didChangeAppLifecycleState(AppLifecycleState estado) {
    if (estado == AppLifecycleState.resumed) {
      temaSitio.sincronizar();
    }
  }

  @override
  Widget build(BuildContext context) {
    return ListenableBuilder(
      listenable: temaSitio,
      builder: (context, _) => MaterialApp(
        title: 'PIZZA-JULIAN',
        debugShowCheckedModeBanner: false,
        theme: ThemeData(
          colorScheme: ColorScheme.fromSeed(seedColor: pz.naranja, primary: pz.rojo, secondary: pz.naranja),
          scaffoldBackgroundColor: Colors.white,
          useMaterial3: true,
        ),
        home: paginaDe(Seccion.inicio),
      ),
    );
```

```
  }
}
```

A.6 pizza_julian_app/lib/config.dart

```dart
// Dirección del servidor API (carpeta pizza_julian_api).
// - Emulador Android: http://10.0.2.2:8080 (valor por defecto)
// - Celular real: la IP del computador en la red Wi-Fi, por ejemplo http://192.168.1.10:8080
// Se cambia sin tocar el código con:  flutter run --dart-define=API_URL=http://192.168.1.10:8080
const String apiUrl = String.fromEnvironment('API_URL', defaultValue: 'http://10.0.2.2:8080');

// Las imágenes se guardan en el servidor como rutas relativas ("productos/pizza1.png")
String urlMedia(String ruta) => '$apiUrl/media/$ruta';
```

A.7 pizza_julian_app/lib/api.dart

```dart
import 'dart:convert';

import 'package:flutter/foundation.dart';
import 'package:http/http.dart' as http;

import 'config.dart';

class ApiError implements Exception {
  ApiError(this.mensaje, [this.estado = 0]);

  final String mensaje;
  final int estado;

  bool get sesionExpirada => estado == 401;

  @override
  String toString() => mensaje;
}

// Sesión del editor de contenido: el token reemplaza la cookie de sesión de PHP
class Sesion extends ChangeNotifier {
  String? token;
  String? usuario;

  bool get activa => token != null;

  void iniciar(String nuevoToken, String nombre) {
    token = nuevoToken;
    usuario = nombre;
    notifyListeners();
  }

  void cerrar() {
    token = null;
    usuario = null;
    notifyListeners();
  }
}

final sesion = Sesion();

// Cliente del servidor pizza_julian_api
class Api {
  static const _tiempoEspera = Duration(seconds: 30);

  Map<String, String> get _cabeceras => {
```

```
47          'content-type': 'application/json; charset=utf-8',
48          if (sesion.token != null) 'authorization': 'Bearer ${sesion.token}',
49        };
50
51    Future<Map<String, dynamic>> _enviar(String metodo, String ruta, [Map<String, dynamic>? cuerpo]) async {
52      final uri = Uri.parse('$apiUrl/api/$ruta');
53      final http.Response respuesta;
54      try {
55        final peticion = http.Request(metodo, uri)..headers.addAll(_cabeceras);
56        if (cuerpo != null) {
57          peticion.body = jsonEncode(cuerpo);
58        }
59        respuesta = await http.Response.fromStream(await peticion.send().timeout(_tiempoEspera));
60      } catch (_) {
61        throw ApiError('Error de comunicación con el servidor.');
62      }
63
64      Map<String, dynamic> datos;
65      try {
66        datos = Map<String, dynamic>.from(jsonDecode(utf8.decode(respuesta.bodyBytes)) as Map);
67      } catch (_) {
68        throw ApiError('Respuesta no válida del servidor.', respuesta.statusCode);
69      }
70      if (respuesta.statusCode >= 400 || datos['ok'] != true) {
71        if (respuesta.statusCode == 401 && sesion.activa) {
72          sesion.cerrar();
73        }
74        throw ApiError((datos['mensaje'] ?? 'No fue posible completar la operación.').toString(), respuesta.st
atusCode);
75      }
76      return datos;
77    }
78
79    List<Map<String, dynamic>> _lista(Object? valor) => (valor as List? ?? []).map((e) => Map<String, dynamic
>.from(e as Map)).toList();
80
81    // ---- público
82    Future<Map<String, dynamic>> tema() => _enviar('GET', 'tema');
83
84    Future<Map<String, String>> inicio() async {
85      final datos = await _enviar('GET', 'inicio');
86      return Map<String, dynamic>.from(datos['datos'] as Map).map((k, v) => MapEntry(k, (v ?? '').toString
()));
87    }
88
89    Future<List<Map<String, dynamic>>> productos() async => _lista((await _enviar('GET', 'productos'))['produc
tos']);
90
91    Future<Map<String, dynamic>> contacto() => _enviar('GET', 'contacto');
92
93    Future<String> enviarMensaje({required String correo, required String nombre, required String telefono, re
quired String mensaje}) async {
94      final datos = await _enviar('POST', 'contacto/mensaje', {'correo': correo, 'nombre': nombre, 'telefono':
telefono, 'mensaje': mensaje});
95      return datos['mensaje'].toString();
96    }
97
98    // ---- sesión
99    Future<void> login(String usuario, String password) async {
100     final datos = await _enviar('POST', 'login', {'usuario': usuario, 'password': password});
101     sesion.iniciar(datos['token'].toString(), datos['usuario'].toString());
102   }
103
104   Future<void> logout() async {
105     try {
```

```
      await _enviar('POST', 'logout');
    } catch (_) {
      // aunque falle la red, la sesión local se cierra
    }
    sesion.cerrar();
  }

  // ---- editores (requieren sesión). Las imágenes nuevas viajan en base64 dentro de "imagenes".
  Future<List<Map<String, dynamic>>> registrosInicio() async => _lista((await _enviar('GET', 'inicio/registros'))['registros']);

  Future<String> guardarInicio(int? id, Map<String, dynamic> datos) async =>
      (await _enviar(id == null ? 'POST' : 'PUT', id == null ? 'inicio' : 'inicio/$id', datos))['mensaje'].toString();

  Future<String> eliminarInicio(int id) async => (await _enviar('DELETE', 'inicio/$id'))['mensaje'].toString();

  Future<String> guardarProducto(int? id, Map<String, dynamic> datos) async =>
      (await _enviar(id == null ? 'POST' : 'PUT', id == null ? 'productos' : 'productos/$id', datos))['mensaje'].toString();

  Future<String> eliminarProducto(int id) async => (await _enviar('DELETE', 'productos/$id'))['mensaje'].toString();

  Future<String> guardarContacto(int? id, String url) async =>
      (await _enviar(id == null ? 'POST' : 'PUT', id == null ? 'contacto' : 'contacto/$id', {'url_contacto': url}))['mensaje'].toString();
}

final api = Api();
```

A.8 pizza_julian_app/lib/tema.dart

```
import 'package:flutter/material.dart';

import 'api.dart';

// Paleta de colores compartida por todas las pantallas (equivale a las variables --pizza-* de tema.css).
// Sale del banner que se sube en el editor de Inicio.
class Paleta {
  const Paleta({
    required this.negro,
    required this.negroSuave,
    required this.naranja,
    required this.amarillo,
    required this.rojo,
    required this.rojoProfundo,
    required this.rojoVivo,
    required this.blanco,
    required this.gris,
  });

  final Color negro;
  final Color negroSuave;
  final Color naranja;
  final Color amarillo;
  final Color rojo;
  final Color rojoProfundo;
  final Color rojoVivo;
  final Color blanco;
  final Color gris;

  static const porDefecto = Paleta(
```

```
    negro: Color(0xFF0F090A),
    negroSuave: Color(0xFF1A1113),
    naranja: Color(0xFFFF8B1A),
    amarillo: Color(0xFFFFC21A),
    rojo: Color(0xFFD90A0A),
    rojoProfundo: Color(0xFF8F0909),
    rojoVivo: Color(0xFFFF2E2E),
    blanco: Color(0xFFF7F5F3),
    gris: Color(0xFFE7E2DE),
  );

  static Color? _color(Object? hex) {
    if (hex is! String || !RegExp(r'^#[0-9a-fA-F]{6}$').hasMatch(hex)) {
      return null;
    }
    return Color(int.parse('FF${hex.substring(1)}', radix: 16));
  }

  static Paleta desdeJson(Map<String, dynamic> datos) {
    const d = porDefecto;
    return Paleta(
      negro: _color(datos['negro']) ?? d.negro,
      negroSuave: _color(datos['negro-suave']) ?? d.negroSuave,
      naranja: _color(datos['naranja']) ?? d.naranja,
      amarillo: _color(datos['amarillo']) ?? d.amarillo,
      rojo: _color(datos['rojo']) ?? d.rojo,
      rojoProfundo: _color(datos['rojo-profundo']) ?? d.rojoProfundo,
      rojoVivo: _color(datos['rojo-vivo']) ?? d.rojoVivo,
      blanco: _color(datos['blanco']) ?? d.blanco,
      gris: _color(datos['gris']) ?? d.gris,
    );
  }
}

// Igual que color-mix(in srgb, a pct%, b) de CSS
Color mezclar(Color a, double porcentajeA, Color b) => Color.lerp(b, a, porcentajeA / 100)!;

// Banner + paleta actuales. Se vuelve a consultar al abrir cada pantalla y al regresar a la app,
// así cuando el editor cambia el banner, toda la app toma los colores nuevos.
class TemaSitio extends ChangeNotifier {
  Paleta paleta = Paleta.porDefecto;
  String banner = 'inicio/banner.png';
  String whatsapp = '573132345685';
  String _version = '';

  Future<void> sincronizar() async {
    try {
      final datos = await api.tema();
      final version = (datos['version'] ?? '').toString();
      if (version == _version) {
        return;
      }
      _version = version;
      banner = (datos['banner'] ?? banner).toString();
      whatsapp = (datos['whatsapp'] ?? whatsapp).toString();
      paleta = Paleta.desdeJson(Map<String, dynamic>.from(datos['paleta'] as Map? ?? {}));
      notifyListeners();
    } catch (_) {
      // sin conexión: se mantiene la paleta actual
    }
  }
}

final temaSitio = TemaSitio();

```

```
// Atajo para leer la paleta dentro de los widgets
Paleta get pz => temaSitio.paleta;
```

A.9 pizza_julian_app/lib/navegacion.dart

```
import 'package:flutter/material.dart';

import 'pantallas/contacto.dart';
import 'pantallas/editor_contacto.dart';
import 'pantallas/editor_inicio.dart';
import 'pantallas/editor_productos.dart';
import 'pantallas/inicio.dart';
import 'pantallas/productos.dart';

enum Seccion { inicio, productos, contacto }

extension NombreSeccion on Seccion {
  String get titulo => switch (this) {
        Seccion.inicio => 'Inicio',
        Seccion.productos => 'Productos',
        Seccion.contacto => 'Contacto',
      };
}

Widget paginaDe(Seccion seccion, {bool editor = false}) => switch ((seccion, editor)) {
      (Seccion.inicio, false) => const PantallaInicio(),
      (Seccion.productos, false) => const PantallaProductos(),
      (Seccion.contacto, false) => const PantallaContacto(),
      (Seccion.inicio, true) => const EditorInicio(),
      (Seccion.productos, true) => const EditorProductos(),
      (Seccion.contacto, true) => const EditorContacto(),
    };

// Reemplaza la pantalla actual (como window.location.replace en la web): "Atrás" no vuelve a un editor protegido
void irA(BuildContext context, Seccion seccion, {bool editor = false}) {
  Navigator.of(context).pushAndRemoveUntil(
    PageRouteBuilder(
      pageBuilder: (_, __, ___) => paginaDe(seccion, editor: editor),
      transitionsBuilder: (_, animacion, __, hijo) => FadeTransition(opacity: animacion, child: hijo),
      transitionDuration: const Duration(milliseconds: 220),
    ),
    (_) => false,
  );
}
```

A.10 pizza_julian_app/lib/widgets/login_dialogo.dart

```
import 'dart:async';

import 'package:flutter/material.dart';
import 'package:pointer_interceptor/pointer_interceptor.dart';

import '../api.dart';
import '../tema.dart';
import 'comunes.dart';

// Ventana de "Inicio de sesion" con contador de 30 segundos, igual que el modal #loginModal de la web.
// Devuelve true si el usuario entró.
Future<bool?> mostrarLogin(BuildContext context) => showDialog<bool>(
      context: context,
      barrierDismissible: false, // data-bs-backdrop="static"
```

```
      builder: (_) => const LoginDialogo(),
    );

class LoginDialogo extends StatefulWidget {
  const LoginDialogo({super.key});

  @override
  State<LoginDialogo> createState() => _LoginDialogoState();
}

class _LoginDialogoState extends State<LoginDialogo> {
  static const _segundosIniciales = 30;

  final _usuario = TextEditingController();
  final _password = TextEditingController();
  Timer? _contador;
  int _segundos = _segundosIniciales;
  String? _alerta;
  bool _alertaError = true;
  bool _enviando = false;

  @override
  void initState() {
    super.initState();
    _contador = Timer.periodic(const Duration(seconds: 1), (_) {
      setState(() => _segundos--);
      if (_segundos <= 0) {
        _contador?.cancel();
        Navigator.of(context).pop(false);
      }
    });
  }

  @override
  void dispose() {
    _contador?.cancel();
    _usuario.dispose();
    _password.dispose();
    super.dispose();
  }

  Future<void> _entrar() async {
    final usuario = _usuario.text.trim();
    if (usuario.isEmpty || _password.text.isEmpty) {
      setState(() {
        _alerta = 'Usuario y contraseña son obligatorios.';
        _alertaError = true;
      });
      return;
    }
    setState(() {
      _enviando = true;
      _alerta = null;
    });
    try {
      await api.login(usuario, _password.text);
      _contador?.cancel();
      setState(() {
        _alerta = 'Acceso correcto, redirigiendo...';
        _alertaError = false;
      });
      await Future<void>.delayed(const Duration(milliseconds: 500));
      if (mounted) Navigator.of(context).pop(true);
    } on ApiError catch (e) {
      _password.clear();
```

```
      setState(() {
        _alerta = e.mensaje;
        _alertaError = true;
        _enviando = false;
      });
    }
  }

  InputDecoration _decoracion(String sugerencia) => InputDecoration(
        hintText: sugerencia,
        filled: true,
        fillColor: mezclar(pz.naranja, 10, pz.blanco),
        isDense: true,
        border: OutlineInputBorder(borderRadius: BorderRadius.circular(6)),
        enabledBorder: OutlineInputBorder(
          borderRadius: BorderRadius.circular(6),
          borderSide: BorderSide(color: mezclar(mezclar(pz.negroSuave, 60, pz.amarillo), 65, pz.rojoVivo)),
        ),
        focusedBorder: OutlineInputBorder(borderRadius: BorderRadius.circular(6), borderSide: BorderSide(col
or: pz.naranja, width: 2)),
      );

  Widget _etiqueta(String texto) => Padding(
        padding: const EdgeInsets.only(bottom: 6),
        child: Text(texto, style: TextStyle(color: pz.amarillo, fontWeight: FontWeight.w600)),
      );

  @override
  Widget build(BuildContext context) {
    final borde = mezclar(mezclar(pz.negro, 75, pz.rojo), 85, pz.blanco);
    // PointerInterceptor: en la versión web evita que el mapa (iframe) de debajo se quede con los clics
    return PointerInterceptor(
        child: Dialog(
      backgroundColor: Colors.transparent,
      insetPadding: const EdgeInsets.all(16),
      child: ClipRRect(
        borderRadius: BorderRadius.circular(14),
        child: Container(
          decoration: BoxDecoration(
            border: Border.all(color: borde),
            borderRadius: BorderRadius.circular(14),
            color: pz.negro,
          ),
          child: Stack(
            children: [
              // Fondo: el banner oscurecido
              Positioned.fill(child: ImagenRed(temaSitio.banner)),
              Positioned.fill(child: Container(color: pz.negro.withAlpha(227))),
              SingleChildScrollView(
                padding: const EdgeInsets.all(16),
                child: Column(
                  mainAxisSize: MainAxisSize.min,
                  crossAxisAlignment: CrossAxisAlignment.stretch,
                  children: [
                    Row(
                      children: [
                        Expanded(child: Text('Inicio de sesion', style: TextStyle(color: pz.amarillo, fontSi
ze: 20, fontWeight: FontWeight.w700))),
                        IconButton(
                          tooltip: 'Cerrar',
                          onPressed: () => Navigator.of(context).pop(false),
                          icon: Icon(Icons.close, color: pz.blanco),
                        ),
                      ],
                    ),
```

```
143                     Align(
144                       alignment: Alignment.centerLeft,
145                       child: Container(
146                         padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
147                         decoration: BoxDecoration(
148                           color: mezclar(pz.naranja, 12, Colors.transparent),
149                           border: Border.all(color: borde),
150                           borderRadius: BorderRadius.circular(999),
151                         ),
152                         child: Text.rich(
153                           TextSpan(
154                             text: 'Tiempo restante: ',
155                             style: TextStyle(color: pz.gris, fontSize: 13),
156                             children: [
157                               TextSpan(text: '${_segundos.clamp(0, _segundosIniciales)}', style: TextStyle(c
olor: pz.amarillo, fontWeight: FontWeight.w700)),
158                               const TextSpan(text: 's'),
159                             ],
160                           ),
161                         ),
162                       ),
163                     ),
164                     Divider(color: mezclar(mezclar(pz.negro, 85, pz.rojo), 85, pz.gris), height: 28),
165                     if (_alerta != null) Alerta(_alerta!, error: _alertaError),
166                     _etiqueta('Usuario'),
167                     TextField(
168                       controller: _usuario,
169                       maxLength: 200,
170                       autofillHints: const [AutofillHints.username],
171                       textInputAction: TextInputAction.next,
172                       decoration: _decoracion('Digite su usuario').copyWith(counterText: ''),
173                     ),
174                     const SizedBox(height: 14),
175                     _etiqueta('Contrasena'),
176                     TextField(
177                       controller: _password,
178                       maxLength: 200,
179                       obscureText: true,
180                       autofillHints: const [AutofillHints.password],
181                       onSubmitted: (_) => _entrar(),
182                       decoration: _decoracion('Digite su contrasena').copyWith(counterText: ''),
183                     ),
184                     const SizedBox(height: 18),
185                     BotonGradiente(
186                       texto: 'Entrar',
187                       cargando: _enviando,
188                       colores: [pz.rojo, mezclar(mezclar(pz.naranja, 80, pz.rojoProfundo), 60, pz.rojoViv
o)],
189                       borde: mezclar(mezclar(pz.naranja, 70, pz.rojoVivo), 90, pz.blanco),
190                       alPresionar: _entrar,
191                     ),
192                   ],
193                 ),
194               ),
195             ],
196           ),
197         ),
198       ),
199     ));
200   }
201 }
```